

Endelig Rapport

Gruppe 1

ITL25019-1 25H Big Data: lagring og bearbeiding

Andreas Isaksen
Jon Åsbjørn Hauge
Torgim Aune

6. mai 2026
Halden



Innhold

1	Introduksjon	1
1.1	Gruppen	1
1.1.1	Andreas Isaksen	1
1.1.2	Torggrim Aune	1
1.1.3	Jon Åsbjørn Hauge	2
2	Base Camp	3
2.1	Import av Data	3
2.1.1	TLC	3
2.1.1.1	Nedlasting	3
2.2	Analyse av TLC Datasett	4
2.2.1	Import av Pakker	4
2.2.2	Lesing og Inspeksjon av Data	5
2.2.2.1	Kontroll av Innhold på Utvalgte Kolonner før Type Casting	8
2.2.3	Type Casting og Validering av Datasett etter Behandling	12
2.2.3.1	Type Casting	12
2.2.3.2	Validering av Type Casting	13
2.2.4	Data Kvalitet og Overflødige Kolonner	15
2.2.4.1	Filtrering på Hastighet	15
2.2.4.2	Filtrering på Betalingsbeløp	18
2.2.5	Filtrering på Antall Passasjerer	20
2.2.5.1	Bygge Kolonnene 'trip_duration_min', 'hour_of_day' og 'day_of_week'	20
2.2.5.2	Kalkulere Daglige Turer	21
2.2.5.3	Kalkulere Median for Reisetid	23
2.2.5.4	Kalkulere Total Billettpris per Dag	24

2.2.6	Joins	25
2.2.6.1	Import av Taxi Soner	25
2.2.6.2	Left-Join Lokasjonnavn til Hoveddatasett	26
2.2.6.3	Aggregering: Top 10 Lokasjoner	27
2.2.7	Window Funksjonalitet	29
2.2.7.1	Top 5 Soner Daglig Billettpris Liste	29
2.2.7.2	7-dagers Glidende Gjennomsnitt av Turer per Bydel.	30
2.2.8	Join Fallgruver og Ytelse	31
2.2.8.1	Oppretter Måleverktøy	32
2.2.8.2	Demonstrasjons av 'non-broadcast' Join	33
2.2.8.3	Demonstrasjons av 'broadcast' Join	35
2.2.8.4	Salting	35
2.2.8.5	Caching	38
2.3	Analysis Using the Movielens Dataset	39
2.3.1	Import av Pakker	39
2.3.2	Load and Normalize	40
2.3.2.1	Import av Data	40
2.3.2.2	Konvertering av Timestamp til Dato	41
2.3.2.3	Vurderinger per Dag	42
2.3.2.4	Distribusjon av Vurderings Verdier	43
2.3.2.5	Brukere med $\geq N$ Vurderinger	44
2.3.3	Join ratings→movies to Compute	46
2.3.3.1	Top Movies Rangert på Antall Vurdering med Minst 100 Vurderinger	46
2.3.3.2	Vektet Bayesian Vurdering og Topp 20 Movie Liste med Både Gjennomsnitt og Bayesian Verdi	46
2.3.4	Tags and Texty Aggregation	47
2.3.4.1	For Hver Movie Tell Hvor Mange Ganger en Spesifikk Tag Finnes .	47

2.3.4.2	Top Tags for Hver Movie	49
2.3.5	User-level Windows and Cohorts	50
2.3.5.1	Først og Siste Dato en Bruker Registrerte en Vurdering	50
2.3.5.2	Månedlig Aktivitet med Vurdering per Bruker	50
2.3.5.3	Rullende 3 Måneders Telling av Vurdering per Bruker	51
2.3.5.4	Bruker Kohorter Oppdelt på Første Måned med en Registrert Vurdering	52
2.3.5.5	Aktive Brukere +1, +2, +3 Måneder etter Første Vurdering	53
2.4	Cassandra Database Design og Bruk	53
2.4.1	Modeling TLC in Cassandra	53
2.4.1.1	Target queries:	54
2.4.1.2	Activity	59
2.4.2	Modeling Movielens in Cassandra	62
2.4.2.1	Target Queries:	62
2.4.2.2	Schema Design Exercise	63
2.4.3	Spark-Cassandra Connector	64
2.4.3.1	Load a Subset of TLC Trips into Cassandra	64
2.4.3.2	Run a Spark Job That:	64
3	Capstone Prosjekt Beskrivelse	65
3.1	Intro	65
3.2	Datasettene	65
3.2.1	Trafikkdata	65
3.2.2	Om Trafikkdataene	67
3.2.3	Værdata	68
3.2.4	Om Værdataene	68
3.3	Konkluderende om datasettene	69

4	Utforskning og Validering av Prosjektemnet	70
4.1	Innledning	70
4.1.1	Trafikkdata	70
4.1.2	Værddata	70
4.1.3	Hypoteser	71
4.2	Importer og Klargjøring av Data	71
4.2.1	Overordnet Konfigurering av Zeppelin Notatbok	71
4.2.2	Data Kilder	71
4.2.3	Schema – Definerer av datastruktur	72
4.2.3.1	Trafikk Schema	72
4.2.3.2	Vær Schema	74
4.2.4	Import av Data	75
4.2.5	Fjerning av Duplikater	76
4.2.6	Sammensetting av Trafikk Dataframes	76
4.2.7	Behandling av Importert Data	77
4.2.8	Sammensetting av Trafikk og Værddata	78
4.2.9	Vurdering av Upassende eller Ukorrekt Data	79
4.2.10	Ytelse og Optimalisering	80
4.2.10.1	Regex Replace	80
5	Cassandra	83
5.1	Introduksjon	83
5.2	Keyspace (1_Capstone_CQL_DB_Build.zpln)	83
5.3	CQL Spørringer (3_Capstone_Cassandra_Queries.zpln)	83
5.3.1	Beskrivelse	83
5.3.2	Hypotese 1	84
5.3.2.1	Utdypelse	84

5.3.2.2	Løsning	84
5.3.3	Hypotese 2	84
5.3.3.1	Utdypelse	84
5.3.3.2	Løsning	84
5.3.4	Hypotese 3	84
5.3.4.1	Utdypelse	85
5.3.4.2	Løsning	85
5.3.5	Hypotese 4	85
5.3.5.1	Utdypelse	85
5.3.5.2	Løsning	85
5.3.6	Hypotese 5	85
5.3.6.1	Utdypelse	86
5.3.6.2	Løsning	86
5.4	Database/Tabell Design (1_Capstone_CQL_DB_Build.zpln)	86
5.4.1	Beskrivelse	86
5.4.2	Applikasjonsflyt	86
5.4.2.1		87
5.4.3	Hypotese 1	88
5.4.3.1	Utdypelse	88
5.4.3.2	Løsning	88
5.4.4	Hypotese 2	89
5.4.4.1	Utdypelse	89
5.4.4.2	Løsning	89
5.4.5	Hypotese 3	90
5.4.5.1	Utdypelse	90
5.4.5.2	Løsning	90

5.4.6	Hypotese 4	91
5.4.6.1	Utdypelse	91
5.4.6.2	Løsning	91
5.4.7	Hypotese 5	92
5.4.7.1	Utdypelse	92
5.4.7.2	Løsning	92
5.4.8	Datamodell	93
5.5	Dataeksport Spark → Cassandra (2_Capstone_Frame_to_Table.zpln)	93
5.5.1	Beskrivelse	93
5.5.2	Hypotese 1	94
5.5.2.1	Utdypelse	94
5.5.2.2	Løsning	94
5.5.3	Hypotese 2	95
5.5.3.1	Utdypelse	95
5.5.3.2	Løsning	95
5.5.4	Hypotese 3	96
5.5.4.1	Utdypelse	96
5.5.4.2	Løsning	96
5.5.5	Hypotese 4	97
5.5.5.1	Utdypelse	97
5.5.5.2	Løsning	97
5.5.6	Hypotese 5	98
5.5.6.1	Utdypelse	98
5.5.6.2	Løsning	99
5.6	Resultat	99
5.6.1	Beskrivelse	99

5.6.2	Hypotese 1	99
5.6.2.1	Utdypelse	99
5.6.2.2	Konklusjon	100
5.6.3	Hypotese 2	100
5.6.3.1	Utdypelse	100
5.6.3.2	Konklusjon	101
5.6.4	Hypotese 3	101
5.6.4.1	Utdypelse	101
5.6.4.2	Konklusjon	102
5.6.5	Hypotese 4	102
5.6.5.1	Utdypelse	102
5.6.5.2	Konklusjon	103
5.6.6	Hypotese 5	103
5.6.6.1	Utdypelse	104
5.6.6.2	Konklusjon	105
6	Retrospekt	106
6.1	Utfordringer	106
6.1.1	Tekniske Utfordringer	106
6.1.1.1	Docker på Windows Maskiner	106
6.1.1.2	Docker på Linux Maskiner	110
6.1.1.3	Cassandra, QCL	116
6.1.1.4	Applikasjonsflyt	116
6.2	Løsninger for Docker Utfordringer	121
6.2.1	Windows 11	122
6.2.2	YAML Konfigurasjon	122
6.2.3	Zeppelin Interpreter Oppsett	128

6.2.3.1	Spark	129
6.2.3.2	Alluxio	130
6.2.3.3	bigquery	130
6.2.3.4	Cassandra	130
6.2.3.5	Elasticsearch	131
6.2.3.6	File	131
6.2.3.7	flink	132
6.2.3.8	flink-cmd	132
6.2.3.9	Groovy	133
6.2.3.10	HBase	133
6.2.3.11	InfluxDB	133
6.2.3.12	Java	134
6.2.3.13	Jupyter	135
6.2.3.14	Markdown	135
6.2.3.15	MongoDB	136
6.2.3.16	Neo4j	136
6.2.3.17	Python	136
6.2.3.18	R	137
6.2.3.19	sh	137
6.2.3.20	Spark-Submit	137
6.2.3.21	SPARQL	138
6.2.4	Operativ Systemet	138
6.2.5	Tidsfordeling	140
6.2.6	Samarbeid i Teamet	140
6.3	Perfomance Tuning	140
6.3.1	Salting	140

6.3.2	Bucketing	140
6.3.3	Repartitions og coalesce	140
6.3.4	Serialisering	141
6.4	Møtereferater - Veileder	141
6.4.1	12.12.2025	141
6.4.2	01.12.2025	141
6.4.3	24.11.2025	142
6.4.4	10.11.2025	143
6.4.5	27.10.2025	143
6.4.6	13.10.2025	144
6.4.7	29.09.2025	145
6.4.8	22.09.2025	146

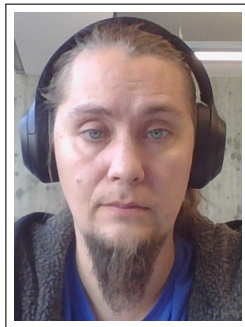
Introduksjon

1.1 Gruppen

Antall medlemmer i gruppen: 3

1.1.1 Andreas Isaksen

Født: 18.08.1990
Bosted: Halden
Mobil: (+47) 93821097
Discord: andreasisaksen
E-post: andreis@hiof.no



BIO

Andreas er en 35 år gammel mann med tidligere erfaring innenfor IT-drift i selskaper som IBM, Visma, Datamatrix og Atea.

Fritiden benyttes i givende stund til barn, hund og studier på fulltid.

Ambisjonene for denne utdanningen er å øke mulighetene for å kunne jobbe med utvikling av løsninger for store datamengder og/eller trene data modeller med maskinlæring i nærområdet (Østfold).

1.1.2 Torgrim Aune

Født: 12.04.1992
Bosted: Sarpsborg
Mobil: (+47) 40120590
Discord: grimtor_17607
E-post: torgria@hiof.no



BIO

Torgrim er 33 år gammel, og bor i Sarpsborg med kone og katt.

Han har jobbet innen bygg og anlegg samt som vaktmester/driftstekniker, og han studerer nå IT på fulltid.

På fritiden liker han å fikle med datamaskiner, spille dataspill, høre på musikk og leke med katten.

1.1.3 Jon Åsbjørn Hauge

Født: 17.101987

Bosted: Skjeberg

Mobil: 41687053

Discord: jon_aabjorn

E-post: jonahaug@hiof.no



BIO

Jon Åsbjørn er 38 år gammel, og bor i skjeberg.

Jon har tidligere gjennomført en mastergrad i idrettsfysiologi, men studere per nå informatikk og jobber deltid i sykkelbutikk. På fritiden driver Jon mye med trening og liker å holde seg fysisk aktiv og i god form.

Base Camp

2.1 Import av Data

2.1.1 TLC

2.1.1.1 Nedlasting

I sammenheng med at fagets tittel ble det tatt en avgjørelse om at det var behov for data av et brukbart magnitud. På grunnlag av dette ble det skrevet et lite script i forkant til å hente ned data fra TLC sine sider. Det foreligger nok bedre løsninger for dette, men siden dette var et kjent område gjorde dette det enkelt å kjøre flere iterative nedlastninger på en gang, slik at vi kunne få tilgang til data for alle typer data (yellow, green, fvhvh) som er lagt ut. For dette prosjektet har det blitt lastet ned alle *.parquet* filer for årene 2022 – 2025.

Kode 2.1: eksempel på script for å hente ned parquet filer fra TLC sine sider.

```
# Download all 2022 TLC trip record files listed on the official page.
param(
    [string]$OutDir = "..\2022"
)

$ErrorActionPreference = 'Stop'
$uri = 'https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page'

# Fetch page (compat for Windows PowerShell 5.1)
if ($PSVersionTable.PSVersion.Major -lt 6) {
    $resp = Invoke-WebRequest -UseBasicParsing -Uri $uri
} else {
    $resp = Invoke-WebRequest -Uri $uri
}
$html = $resp.Content

# Find all 202X cloudfront PARQUET links (yellow/green/fhv/fhvhv)
$pattern = 'https?://d37ci6vzurychx\.cloudfront\.net/trip-data/[a-z]+_tripdata_2022-(0[1-9]|1[0-2])\.parquet'
$links = [Regex]::Matches($html, $pattern) |
    ForEach-Object { $_.Value } |
    Sort-Object -Unique

if ($links.Count -eq 0) { throw "No 2022 file links found. Page structure or host changed?" }

New-Item -ItemType Directory -Force -Path $OutDir | Out-Null

foreach ($url in $links) {
    $file = Join-Path $OutDir ([IO.Path]::GetFileName($url))
    if (Test-Path $file) {
```

```
Write-Host "Already have $(Split-Path -Leaf $file) - skipping"
continue
}
Write-Host "Downloading $(Split-Path -Leaf $file)..."
Invoke-WebRequest -Uri $url -OutFile $file
}

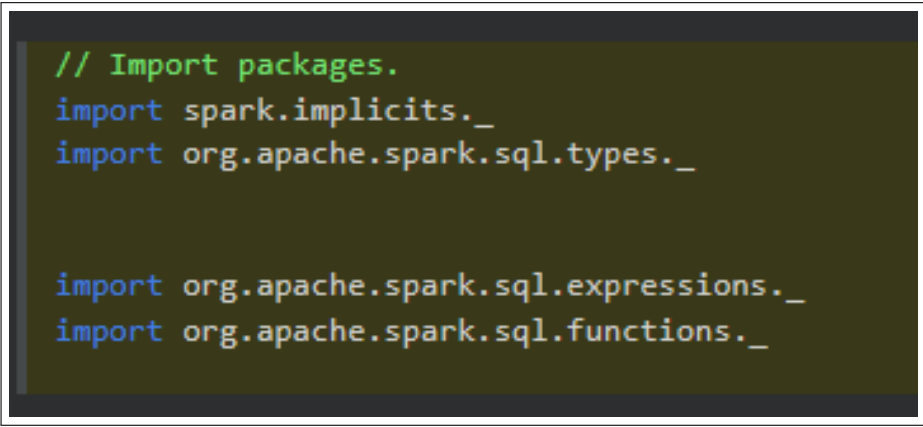
Write-Host "`nDone. Saved $($links.Count) files to $OutDir"
```

2.2 Analyse av TLC Datasett

2.2.1 Import av Pakker

For å løse denne oppgaven har følgende pakker blitt importert.

- `spark.implicit._`
Pakken importerer blant annet funksjonalitet som muliggjør det å konvertere **scala** objekter til *'dataframes'*, samt at pakken gir tilgang til bruk av enkle **SQL** uttrykk i **spark** kode. Slik som å referere til kolonner ved bruk av `$`, som blir brukt når det utføres **join** operasjoner.[\[1\]](#)
- `org.apache.spark.sql.types._`
Pakken importeres for å få tilgang til Spark sine *data type klasser*. Dette blir brukt til å lage skjematløsninger som definerer enkelte *'data frames'* i løsningen.[\[2\]](#)
- `org.apache.spark.sql.expressions._`
Pakken gir tilgang til **window** funksjonalitet i **spark**.[\[3\]](#)
- `org.apache.spark.sql.functions._`
Denne pakken inneholder kjernefunksjonalitet for **spark** uttrykk. Slik som **select**, **withColumn**, **agg**, **filter** og mye mer.[\[4\]](#)



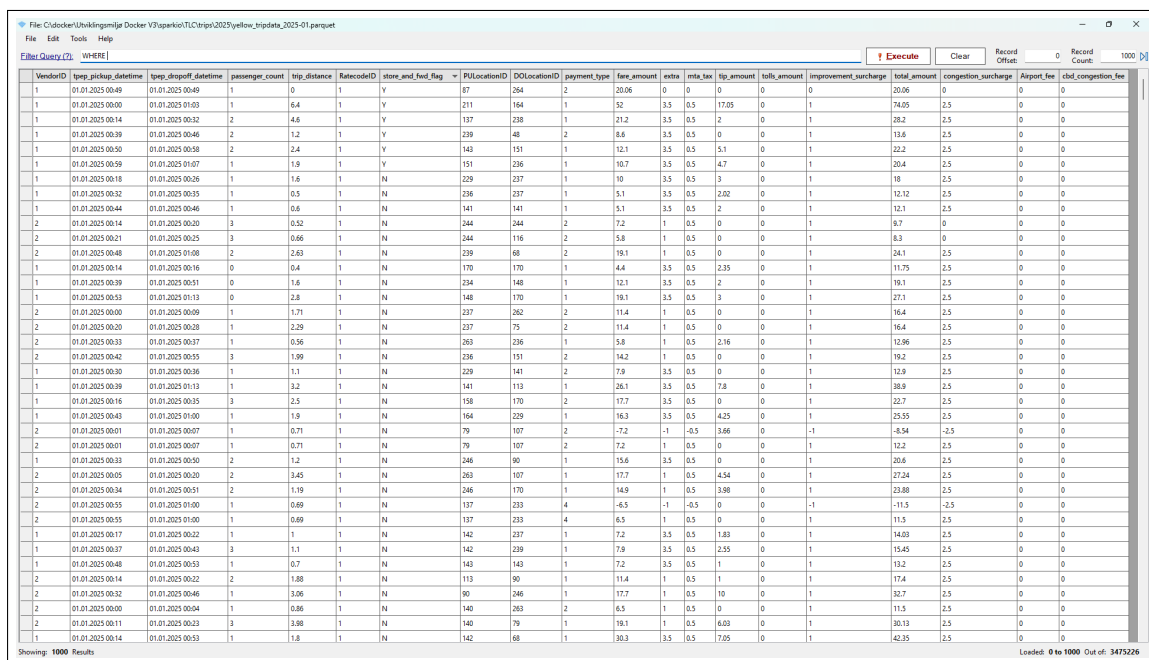
```
// Import packages.
import spark.implicit._
import org.apache.spark.sql.types._

import org.apache.spark.sql.expressions._
import org.apache.spark.sql.functions._
```

Figur 2.1: Utklipp fra Zeppelin celle: For pakke import

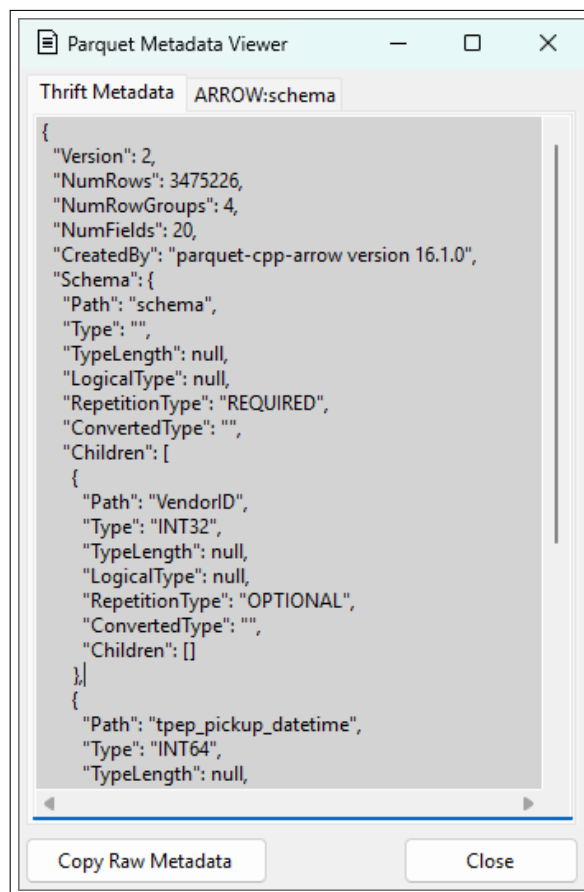
2.2.2 Lesing og Inspeksjon av Data

Under inspeksjonen av data har det blitt tatt i bruk en applikasjons løsning kalt *ParquetViewer*^[5] for å få en innsikt i datastrukturen til *.parquet* filene. Dette verktøyet muliggjør en enkel visuell fremstilling av innholdet i *.parquet* filer, samt at det gir mulighet for å inspisere metadata som er lagret i filene. Dette var nyttig for å få en forståelse av hvordan data var strukturert, samt hvilke datatyper som var definert for hver kolonne.



VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_ford_flag	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount	congestion_surcharge	Airport_fee	cdd_congestion_fee
1	01.01.2025 00:49	01.01.2025 00:49	1	0	1	Y	87	264	2	20.06	0	0	0	0	0	20.06	0	0	0
1	01.01.2025 00:50	01.01.2025 01:03	1	6.4	1	Y	211	164	1	52	3.5	0.5	17.05	0	1	74.05	2.5	0	0
1	01.01.2025 00:14	01.01.2025 00:32	2	4.8	1	Y	197	238	1	21.2	3.5	0.5	2	0	1	28.2	2.5	0	0
1	01.01.2025 00:39	01.01.2025 00:46	2	1.2	1	Y	239	48	2	8.8	3.5	0.5	0	0	1	13.6	2.5	0	0
1	01.01.2025 00:50	01.01.2025 00:58	2	2.4	1	Y	142	151	1	12.1	2.5	0.5	5.1	0	1	22.2	2.5	0	0
1	01.01.2025 00:59	01.01.2025 01:07	1	1.9	1	Y	151	236	1	10.7	3.5	0.5	4.7	0	1	20.4	2.5	0	0
1	01.01.2025 00:18	01.01.2025 00:26	1	1.6	1	N	229	237	1	10	3.5	0.5	3	0	1	18	2.5	0	0
1	01.01.2025 00:32	01.01.2025 00:35	1	0.5	1	N	236	237	1	5.1	3.5	0.5	2.02	0	1	12.12	2.5	0	0
1	01.01.2025 00:44	01.01.2025 00:46	1	0.6	1	N	141	141	1	5.1	3.5	0.5	2	0	1	12.1	2.5	0	0
2	01.01.2025 00:14	01.01.2025 00:20	3	0.52	1	N	244	244	2	7.2	1	0.5	0	0	1	9.7	0	0	0
2	01.01.2025 00:21	01.01.2025 00:25	3	0.66	1	N	244	116	2	5.8	1	0.5	0	0	1	8.3	0	0	0
2	01.01.2025 00:48	01.01.2025 01:08	2	2.63	1	N	239	68	2	19.1	1	0.5	0	0	1	24.1	2.5	0	0
1	01.01.2025 00:14	01.01.2025 00:16	0	0.4	1	N	170	170	1	4.4	3.5	0.5	2.35	0	1	11.75	2.5	0	0
1	01.01.2025 00:39	01.01.2025 00:51	0	1.6	1	N	234	148	1	12.1	3.5	0.5	2	0	1	19.1	2.5	0	0
1	01.01.2025 00:53	01.01.2025 01:13	0	2.8	1	N	148	170	1	19.1	3.5	0.5	3	0	1	27.1	2.5	0	0
2	01.01.2025 00:00	01.01.2025 00:09	1	1.71	1	N	237	262	2	11.4	1	0.5	0	0	1	16.4	2.5	0	0
2	01.01.2025 00:20	01.01.2025 00:26	1	2.29	1	N	237	75	2	11.4	1	0.5	0	0	1	16.4	2.5	0	0
2	01.01.2025 00:33	01.01.2025 00:37	1	0.56	1	N	263	236	1	5.8	1	0.5	2.16	0	1	12.96	2.5	0	0
2	01.01.2025 00:42	01.01.2025 00:55	3	1.99	1	N	236	151	2	14.2	1	0.5	0	0	1	19.2	2.5	0	0
1	01.01.2025 00:30	01.01.2025 00:36	1	1.1	1	N	229	141	2	7.9	3.5	0.5	0	0	1	12.9	2.5	0	0
1	01.01.2025 00:39	01.01.2025 01:13	1	3.2	1	N	141	113	1	26.1	3.5	0.5	7.8	0	1	38.9	2.5	0	0
1	01.01.2025 00:16	01.01.2025 00:35	3	2.5	1	N	158	170	2	17.7	3.5	0.5	0	0	1	22.7	2.5	0	0
1	01.01.2025 00:43	01.01.2025 01:00	1	1.9	1	N	164	229	1	16.3	3.5	0.5	4.25	0	1	25.55	2.5	0	0
2	01.01.2025 00:01	01.01.2025 00:07	1	0.71	1	N	79	107	2	-7.2	-1	-0.5	3.66	0	-1	-8.54	-2.5	0	0
2	01.01.2025 00:01	01.01.2025 00:07	1	0.71	1	N	79	107	2	7.2	1	0.5	0	0	1	12.2	2.5	0	0
1	01.01.2025 00:33	01.01.2025 00:50	2	1.2	1	N	246	90	1	15.6	3.5	0.5	0	0	1	20.6	2.5	0	0
2	01.01.2025 00:05	01.01.2025 00:20	2	3.45	1	N	263	107	1	17.7	1	0.5	4.54	0	1	27.24	2.5	0	0
2	01.01.2025 00:34	01.01.2025 00:51	2	1.19	1	N	246	170	1	14.9	1	0.5	3.98	0	1	23.88	2.5	0	0
2	01.01.2025 00:55	01.01.2025 01:00	1	0.69	1	N	137	233	4	-6.5	-1	-0.5	0	0	-1	-11.5	-2.5	0	0
2	01.01.2025 00:15	01.01.2025 01:00	1	0.69	1	N	137	233	4	6.5	1	0.5	0	0	1	11.5	2.5	0	0
1	01.01.2025 00:17	01.01.2025 00:22	1	1	1	N	142	237	1	7.2	2.5	0.5	1.82	0	1	14.09	2.5	0	0
1	01.01.2025 00:37	01.01.2025 00:43	3	1.1	1	N	142	239	1	7.9	3.5	0.5	2.55	0	1	15.45	2.5	0	0
1	01.01.2025 00:48	01.01.2025 00:53	1	0.7	1	N	143	143	1	7.2	3.5	0.5	1	0	1	13.2	2.5	0	0
2	01.01.2025 00:14	01.01.2025 00:22	2	1.88	1	N	113	90	1	11.4	1	0.5	1	0	1	17.4	2.5	0	0
2	01.01.2025 00:32	01.01.2025 00:46	1	3.06	1	N	90	246	1	17.7	1	0.5	10	0	1	32.7	2.5	0	0
2	01.01.2025 00:00	01.01.2025 00:04	1	0.86	1	N	140	263	2	6.5	1	0.5	0	0	1	11.5	2.5	0	0
2	01.01.2025 00:11	01.01.2025 00:23	3	3.98	1	N	140	79	1	19.1	1	0.5	6.03	0	1	30.13	2.5	0	0
1	01.01.2025 00:14	01.01.2025 00:53	1	1.8	1	N	142	68	1	30.3	3.5	0.5	7.05	0	1	42.35	2.5	0	0

Figur 2.2: ParquetViewer: Oversikt over innhold i en TLC *.parquet* fil

Figur 2.3: ParquetViewer: Metadata for en TLC *.parquet* fil

I starten av dette prosjektet ble det gjort en ambisiøs forsøk på å hente inn og kombinere data fra alle tre kategorier i *TLC* sine delte data (*yellow*, *green* og *utleie*) i perioden **2022 – 2025**. Dette førte til flere forskjellige utfordringer.

Første problemstilling var at data ble strukturert forskjellig i hver kategori, som videre ledet til en utfordring vedrørende å designe et **schema** som var passende for alle kategorier.

Videre var det også et problem med inkonsistent definering av datatyper for kolonner i forskjellige filer som tilhørte samme kategori (eksempelvis alle filer for gul taxi). Kolonner som var definert med en **int** verdi kunne ha en **double** verdi i en annen.

Dette er problemstillinger som er mulige å løse, men grunnet manglende erfaring med denne typen av databehandling ble det tatt en avgjørelse om å redusere datamengden denne gangen.

Etter en lengre elimineringsprosess av data ble til slutt data for *gule taxier* for perioden **2025** den endelige datamengden som har blitt jobbet med i dette prosjektet, da disse filene var konsise i henhold til oppførte datatyper på tilhørende kolonner.

```
// Import of TCL data:

// Define Schema for TCL.
val refPath = "/sparkio/TLC/trips/2025/yellow_tripdata_2025-01.parquet"
val refSchema = spark.read.parquet(refPath).schema

// Define path for datafiles.

val base = ("/sparkio/TLC/trips/2025/yellow*.parquet")

// Load data into a DataFrame.
val df = spark.read
  .option("recursiveFileLookup", true)
  .schema(refSchema)
  .parquet(base)

// Validate results.
print("Yellow df\n")
df.show(3, truncate = false)
```

Figur 2.4: Utklipp fra Zeppelin celle: For TLC data import

Her hentes først *metdata* fra den tidligste (januar) filen, som videre blir benyttet til å bygge opp et *schema* som dikterer datasettets initielle struktur.

Etter at data ble ferdig innlastet og 'dataframe' bygget, ble det gjort noen enkle grunnleggende handlinger for å inspisere importert data.

- **printSchema()**

Gjorde et kall på denne metoden for å få en oversikt over kolonnens datatyper og validere at oppsettet ser riktig ut.

```
// Get an overview of DataTypes
df.printSchema()

root
|-- VendorID: integer (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: long (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: long (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: integer (nullable = true)
|-- DOLocationID: integer (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- Airport_fee: double (nullable = true)
|-- cbd_congestion_fee: double (nullable = true)
```

Figur 2.5: Utklipp fra Zeppelin celle: For **printSchema()**.

Ved inspeksjon av datatyper som er definert i metadata til *.parquet* filen som vi har hentet *schema* fra, observeres det at det foreligger flere kolonner med uppassende datatyper i henhold til de data som blir representert. Følgende liste viser til endringer som vil bli gjort ved videre behandling av data. Dette er da steg som må utføres ved import av nye data for å sikre konsistitet i det endelige datasettet for TLC data. Det er blitt gjort forsøk på å manuelt definere et **schema** som var mer passende, men grunnet at det foreligger predefinerte datatyper i *.parquet* filene, førte dette til problemer ved import.

- **VendorID:**
Denne kolonnen er definert som **int**. Her vil det ikke nødvendigvis ha en stor innvirkning på lagringsplassen, men det ville vært mer passende å definere denne kolonnen som en **Byte** (Kan holde på tallverdier -128 til 127), da dette vil fint kunne representere de mulige verdiene i denne kolonnen.
- **tpep_pickup/dropoff_datetime:**
Disse kolonnene er definert som **timestamp_ntz** som er en datatype som ikke tar hensyn til tidssoner. Siden dataene representerer hendelser som skjer i New York, burde disse kolonnene vært definert som **timestamp** da alt forekommer innenfor samme tidssone.
- **passenger_count:**
Denne kolonnen er definert som **long**. Dette gir ikke mening i praksis, da en taxi normalt vil ha et begrenset antall seter. Det ville derfor vært mer passende å definere denne kolonnen som en **Byte** (Kan holde på tallverdier -128 til 127), da dette vil fint kunne representere antall passasjerer i en taxi. Dette vil bidra til unødvendig bruk av lagringsplass betraktelig for store datasett.
- **RatecodeID:**
Denne kolonnen er definert som **long**. Siden denne kolonnen representerer en kode for hvilken type rate som er brukt, vil det være mer passende å definere denne som en **Byte** (Kan holde på tallverdier -128 til 127). Dette vil bidra til unødvendig bruk av lagringsplass betraktelig for store datasett.
- **store_and_fwd_flag:**
Denne kolonnen er definert som **string**. Siden denne kolonnen kun inneholder to mulige verdier ('Y', 'N' og NULL), vil det gi mer mening å definere denne som en **boolean** datatype. Dette vil bidra til unødvendig bruk av lagringsplass betraktelig for store datasett.
- **payment_type:**
Denne kolonnen er definert som **long**. Kolonnen benytter tallverdier 1-5 til å representere forskjellige betalingsmetoder. Det vil derfor være mer passende å definere denne som en **Byte** (Kan holde på tallverdier -128 til 127). Dette vil bidra til unødvendig bruk av lagringsplass betraktelig for store datasett.

2.2.2.1 Kontroll av Innhold på Utvalgte Kolonner før Type Casting

Før type casting ble utført, ble det gjort en enkel kontroll av innholdet i kolonner som potensielt var aktuelle for type casting. Dette ble utført for å validere funnene vi gjorde i seksjon 2.2.2 er i tråd med hele datasettet som har blitt importert.

Måten dette ble utført på var ved å hente ut unike verdier i kolonner som var aktuelle for vurdering,

for så å inspisere disse manuelt.
Følgende kolonner ble inspisert:

– **VendorID**

```
df.select("VendorID").distinct().show()

+-----+
|VendorID|
+-----+
|      1|
|      6|
|      7|
|      2|
+-----+
```

Figur 2.6: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *VendorID*.

Bekreftet at denne kolonnen kan bli castet til **Byte** uten tap av data.

– **passenger_count**

```
df.select("passenger_count").distinct().show()

+-----+
|passenger_count|
+-----+
|              0|
|              7|
|              6|
|              9|
|              5|
|              1|
|              3|
|              8|
|              2|
|              4|
|             NULL|
+-----+
```

Figur 2.7: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *passenger_count*.

Bekreftet at denne kolonnen kan bli castet til **Byte** uten tap av data.

– **RatecodeID**

```
df.select("RatecodeID").distinct().show()
+-----+
|RatecodeID|
+-----+
|         6|
|         5|
|         1|
|         3|
|         2|
|         4|
|        99|
|        NULL|
+-----+
```

Figur 2.8: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *RatecodeID*.

Bekreftet at denne kolonnen kan bli castet til **Byte** uten tap av data.

– **store_and_fwd_flag**

```
df.select("store_and_fwd_flag").distinct().show()
+-----+
|store_and_fwd_flag|
+-----+
|                 Y|
|                 N|
|                NULL|
+-----+
```

Figur 2.9: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *store_and_fwd_flag*.

Grunnet at denne kolonnen kun inneholder verdiene 'Y', 'N' og NULL, ble det bekreftet at denne kolonnen kan bli castet til **boolean** uten tap av data.

– **payment_type**

```
df.select("payment_type").distinct().show()

+-----+
|payment_type|
+-----+
|          0|
|          1|
|          3|
|          2|
|          4|
|          5|
+-----+
```

Figur 2.10: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *payment_type*.

Bekreftet at denne kolonnen kan bli castet til **Byte** uten tap av data.

– **improvement_surcharge**

```
df.select("improvement_surcharge").distinct().show()

+-----+
|improvement_surcharge|
+-----+
|              0.0|
|             -1.0|
|              0.3|
|              1.0|
+-----+
```

Figur 2.11: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *improvement_surcharge*.

Da denne kolonnen inneholder desimaler og allerede er definert som **double**, ble det konkludert med at denne kolonnen allerede har en passende datatype og ikke trenger type casting.

– **cbd_congestion_fee**

```
df.select("cbd_congestion_fee").distinct().show()

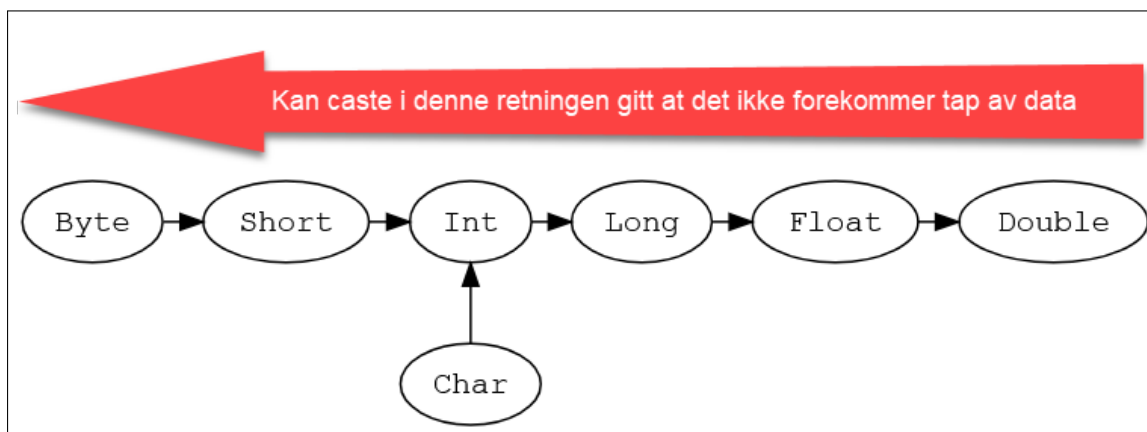
+-----+
|cbd_congestion_fee|
+-----+
|              0.0|
|             1.25|
|            -0.75|
|             0.75|
|             1.5|
|             0.25|
+-----+
```

Figur 2.12: Utklipp fra Zeppelin celle: Unike verdier i kolonnen *cbd_congestion_fee*.

Da denne kolonnen inneholder desimaler og allerede er definert som **double**, ble det konkludert med at denne kolonnen allerede har en passende datatype og ikke trenger type casting.

2.2.3 Type Casting og Validering av Datasett etter Behandling

Basert på funnene i seksjon 2.2.2 og 2.2.2.1 ble det utført type casting på de kolonner som ble ansett som upassende i henhold til datatyper. Dette ble utført ved å benytte **cast** funksjonen til å endre datatypene til ønsket format. Her er det tatt høyde for regler rundt 'Type Casting', som bekrefter at man kan caste en verdi 'bakover' gitt at dette ikke fører til tap av data[6].



Figur 2.13: Illustrasjon av mulige retninger for type casting.

2.2.3.1 Type Casting

Selve type castingen ble utført ved å lage en midlertidig kopi av hoveddatasettet, hvor det ble gjort et kall på **withColumn** funksjonen for hver kolonne som skulle endres. Her ble det spesifisert navnet på kolonnen som skulle endres, samt at det ble gjort et kall på **cast** funksjonen med ønsket datatype som parameter.

```
%spark
val typeFix = df
  .withColumn("VendorID", col("VendorID").cast("byte"))
  .withColumn("tpep_pickup_datetime", col("tpep_pickup_datetime").cast("timestamp"))
  .withColumn("tpep_dropoff_datetime", col("tpep_dropoff_datetime").cast("timestamp"))
  .withColumn("passenger_count", col("passenger_count").cast("byte"))
  .withColumn("RatecodeID", col("RatecodeID").cast("byte"))
  .withColumn("store_and_fwd_flag", (col("store_and_fwd_flag") === lit("Y")).cast("boolean"))
  .withColumn("payment_type", col("payment_type").cast("byte"))
```

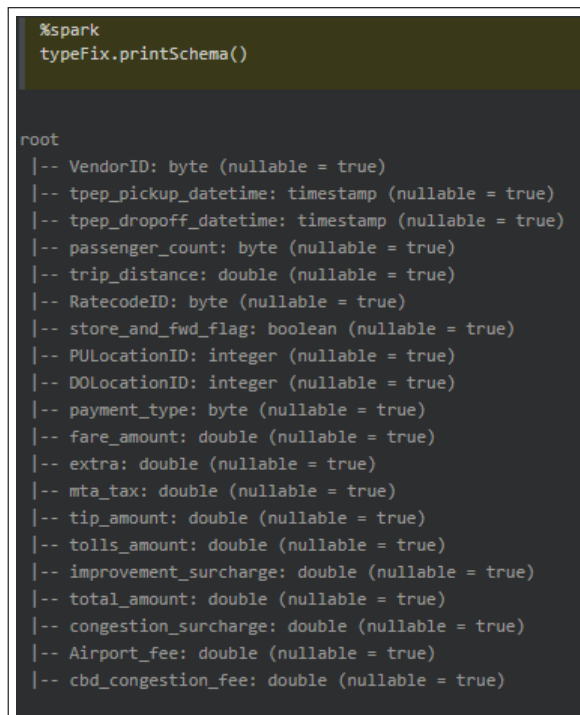
Figur 2.14: Utklipp fra Zeppelin celle: Type casting av kolonner i midlertidig kopi av hoveddatasettet.

2.2.3.2 Validering av Type Casting

Siden nye kolonner ble opprettet i en kopi av hoveddatasettet, kunne det utføres en enkel validering av at type castingen hadde blitt utført som forventet før dette ble overskrevet til hoveddatasettet.

- **Validering av datatyper:**

Først ble det gjort et kall på `printSchema()` funksjonen for å inspisere datatypene i den midlertidige kopien.



```
%spark
typeFix.printSchema()

root
|-- VendorID: byte (nullable = true)
|-- tpep_pickup_datetime: timestamp (nullable = true)
|-- tpep_dropoff_datetime: timestamp (nullable = true)
|-- passenger_count: byte (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: byte (nullable = true)
|-- store_and_fwd_flag: boolean (nullable = true)
|-- PULocationID: integer (nullable = true)
|-- DOLocationID: integer (nullable = true)
|-- payment_type: byte (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- Airport_fee: double (nullable = true)
|-- cbd_congestion_fee: double (nullable = true)
```

Figur 2.15: Utklipp fra Zeppelin celle: Validering av datatyper i midlertidig kopi av hoveddatasettet.

- **Validering av kolonnen 'store_and_fwd_flag':**

Siden denne kolonnen ble castet fra `string` til `boolean`, ble det gjort en ekstra validering for å sikre at verdiene i kolonnen fortsatt var intakte etter castingen. Dette ble utført ved å hente ut unike verdier for kolonnen i originalen og sammenligne med unike verdier i den midlertidige kopien.

```
%spark
print("Controll that 'Y'=='true':\n")
print(df.select("store_and_fwd_flag").filter(col("store_and_fwd_flag") === lit("Y")).count())
print("\n")
print(typeFix.select("store_and_fwd_flag").filter(col("store_and_fwd_flag") === lit("true")).count())
print("\nControll that 'N'=='false':\n")
print(df.select("store_and_fwd_flag").filter(col("store_and_fwd_flag") === lit("N")).count())
print("\n")
print(typeFix.select("store_and_fwd_flag").filter(col("store_and_fwd_flag") === lit("false")).count())
print("\n")
print("\nControll that 'NULL'=='NULL':\n")
print(df.filter(col("store_and_fwd_flag").isNull).count())
print("\n")
print(typeFix.filter(col("store_and_fwd_flag").isNull).count())

Controll that 'Y'=='true':
79145
79145
Controll that 'N'=='false':
21445846
21445846

Controll that 'NULL'=='NULL':
6457356
6457356
```

Figur 2.16: Utklipp fra Zeppelin celle: Validering av unike verdier i kolonnen *store_and_fwd_flag* etter type casting.

– **Validering av antall rader:**

Siden type casting ikke skal påvirke antall rader i et datasett, ble det gjort en enkel validering ved å telle antall rader i både originalen og den midlertidige kopien. Etter telling ble disse to verdiene sammenlignet ved å subtrahere antall rader i kopien fra antall rader i originalen. Resultatet av denne subtraksjonen skulle være 0 for å bekrefte at ingen rader hadde blitt tapt under type castingen.

```
%spark

df.count() - typeFix.count()

res70: Long = 0
```

Figur 2.17: Utklipp fra Zeppelin celle: Validering av antall rader etter type casting.

Nå som valideringene er fullført og bekreftet at type castingen har blitt utført som forventet, kan den midlertidige kopien overskrive hoveddatasettet. Det er med dette nye datasettet at videre behandling av data vil bli utført.

2.2.4 Data Kvalitet og Overflødige Kolonner

Her ble det gjort filtrering over flere steg for å kunne sikre en god filtreringmetodikk, samt muligheten for å validere at hoved datasettet fortsatt var intakt etter de forskjellige filtreringene av 'upassende' data. I denne kontekst omhandler *upassende data* verdier som ikke gir mening eller ikke er realistiske i praksis.

2.2.4.1 Filtrering på Hastighet

Først var det ønskelig å fjerne alle rader som inneholdt usannsynlige og umulige hastigheter for en kjøretur i New York. For å løse dette ble det gjort en undersøkelse på kjørehastigheter med GPT[7]. Etter denne undersøkelsen ble det antatt at over *80miles* i timen ble satt som øvre grense for realistisk kjørehastighet da dette var godt over øvre fartsgrense i New York (motorveier kunne ha opp til *50miles* i timen).

Siden datasettet ikke inneholdt informasjon om hastighet, måtte dette kalkuleres og legges inn som en ny kolonne før filtrering kunne utføres. Dette ble utført med følgende formel:

$$\frac{distanse}{tid} = hastighet$$

For å kunne utføre denne kalkulasjonen måtte data omformes til å passe inn i formelen. Det ble derfor tatt en kopi av datasettet til å omforme.

I kopien ble dateringer for når kunder ble 'plukket opp' og 'sluppet av' omformet til tallverdier i sekunder. Deretter ble de nye verdiene kalkulert ved å finne tidsintervallet mellom disse to, for så å gjøre en divisjon på **3600** som resulterte i en ny kolonne med reisetid i timer. Siden datasettet allerede inneholdt en kolonne om distanse kunne denne brukes til å finne kjørehastigheten per tur.

```
%spark
// Build trip_duration_min, hour_of_day, day_of_week.
val dfWithSpeed =
  df
    .withColumn("pickup_day", col("tpep_pickup_datetime"))
    .withColumn("dropoff_day", col("tpep_dropoff_datetime"))
    .withColumn("pickup_sec", col("tpep_pickup_datetime").cast("long"))
    .withColumn("dropoff_sec", col("tpep_dropoff_datetime").cast("long"))
    // duration in hours.
    .withColumn("duration_hours",
      (col("dropoff_sec") - col("pickup_sec")) / 3600.0
    )
    // speed = miles / hours.
    .withColumn(
      "Speed_miles_pr_hour",
      when(col("duration_hours") > 0,
        col("trip_distance") / col("duration_hours")
      ).otherwise(lit(null).cast("double"))
    )
    // Travel duration in minutes.
    .withColumn(
      "trip_duration_min",
      when(not(col("duration_hours").isNull),
        col("duration_hours") * 60
      ).otherwise(lit(null).cast("double"))
    )
    .drop("pickup_sec", "dropoff_sec", "duration_hours")
    .withColumn("Speed_miles_pr_hour", round(col("Speed_miles_pr_hour"), 2))
    .withColumn("trip_duration_min", round(col("trip_duration_min"), 2))
    .withColumn("hour_of_day", date_format(col("pickup_day"), "k"))
    .withColumn("pickup_day", dayofweek(col("pickup_day")))
    .withColumn("dropoff_day", dayofweek(col("dropoff_day")))
```

Figur 2.18: Utlipp fra Zeppelin celle: Definerings av hastighet, reisetid, klokkeslett i hele timer, dager for på-/avstigning.

Nå som datasettet inneholder de nødvendige verdier som trengs for å filtrer bort hastigheter som overstiger satt grenseverdi, kan dette nå filtreres ut av datasettet. Dette blir utført i kopi datasetet for å validere at filtreringen ble utført som forventet. Etter at ønsket resultat er oppnådd, blir kopi datasettet satt som det nye hoveddatasettet ved å overskrive det originale.

Først validerer jeg at kopien inneholder samme antall rader etter at behandlingen (Figur 2.5) er utført.

```
// Validate that numbers of rows are the same.
print(df.count())
print("\n")
print(dfWithSpeed.count())

27982347
27982347
```

Figur 2.19: Utklipp fra Zeppelin celle: Validerer rader i kopi datasett.

Deretter valideres det at kolonner har ønsket struktur i kopi datasettet.

```
// Validate content of new df.
dfWithSpeed.show(3, truncate = false)
```

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID
1	2025-01-01 00:18:38	2025-01-01 00:26:59	1	1.6	1	N	229	237
1	2025-01-01 00:32:40	2025-01-01 00:35:13	1	0.5	1	N	236	237
1	2025-01-01 00:44:04	2025-01-01 00:46:01	1	0.6	1	N	141	141

only showing top 3 rows

Figur 2.20: Utklipp fra Zeppelin celle: Validering av kolonner til kopi datasett.

Deretter utføres følgende handlinger:

- Teller antall verdier som skal fjernes. Dette er da verdier som er over *80miles* i timen eller der kolonnen har NULL verdi.

```
// Counting rows with higher speed than 80miler/hour.
dfWithSpeed.filter(col("Speed_miles_pr_hour") > 80 || col("Speed_miles_pr_hour").isNull()).count()

res7: Long = 260984
```

Figur 2.21: Utklipp fra Zeppelin celle: Teller antall verdier som skal fjernes

- Lager et midlertidig datasett hvor rader er ekskludert.

```
// Creating a new df excluding rows where speed > 80 or speed == NULL.
val dfWithSpeedRmHighSpeedAndNull = dfWithSpeed.filter(col("Speed_miles_pr_hour") <= 80)
dfWithSpeedRmHighSpeedAndNull.count()

dfWithSpeedRmHighSpeedAndNull: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [Ve
res8: Long = 27721363
```

Figur 2.22: Utlipp fra Zeppelin celle: Midlertidig datasett hvor upassende rader er ekskludert.

- Validerer at antallet fjernede verdier stemmer ved å sammenligne midlertidig datasett med det originale. Her kan vi se at antallet rader stemmer overens med antallet funnet i filtrering på antall rader som skal fjernes.

```
// Validate that exluded rows matches count on rows with higher speed than 80miler/hour.
dfWithSpeedRmHighSpeedAndNull.count() - df.count()

res9: Long = -260984
```

Figur 2.23: Utlipp fra Zeppelin celle: Validering av ekskluderinger.

- Etter validering blir det originale datasetet overskrevet slik at filtrerte verdier nå er ekskludert fra hoveddatasettet.

```
// Make df with exluded speed rows the new main df.
val df = dfWithSpeedRmHighSpeedAndNull
// Validate content of new main df.
df.show(3, truncate = false)
```

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip
1	2025-01-01 00:18:38	2025-01-01 00:26:59	1	1.6
1	2025-01-01 00:32:40	2025-01-01 00:35:13	1	0.5
1	2025-01-01 00:44:04	2025-01-01 00:46:01	1	0.6

Figur 2.24: Utlipp fra Zeppelin celle: Overskrivning av original til behandlet datasett.

2.2.4.2 Filtrering på Betalingsbeløp

For denne oppgaven foreligger det allerede relevante data i datasettet, slik at det ikke vil være behov for å manipulere datasettet før vi kan filtrere bort uønskede rader.

Her tar vi for oss kolonnen *total_amount* som inneholder det totale betalings beløpet for hver rad. Målet her er å få bort rader som inneholder verdier ≤ 0 . Dette blir løst på følgende måte:

- Teller rader i hoveddatasettet hvor betalinger er 0 eller lavere.

```
// Count rows where total_amount is 0 or lower.  
df.filter(col("total_amount") <= 0).count()  
  
res11: Long = 526220
```

Figur 2.25: Utklipp fra Zeppelin celle: Rader i hoveddatasettet hvor betalinger er 0 eller lavere.

- Lager et midlertidig datasett som holder på de radene vi ønsker å beholde.

```
// Create new df excluding rows where total_amount is 0 or lower.  
val PositiveFare = df.filter(col("total_amount") > 0)  
  
PositiveFare: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [
```

Figur 2.26: Utklipp fra Zeppelin celle: Midlertidig datasett.

- Validerer at antallet fjernede rader stemmer med hoveddatasettet. Her kan vi se at antallet rader stemmer overens med antallet funnet i filtrering på antall rader som skal fjernes.

```
// Validate that number of excluded rows match the number of 0 or negative values in total_amount.  
PositiveFare.count() - df.count()  
  
res12: Long = -526220
```

Figur 2.27: Utklipp fra Zeppelin celle: Filter validering.

- Overskriver hoveddatasettet med det midlertidige datasettet og validerer innholdet i det nye hoveddatasettet.

```
// Make df with excluded 0 or negative values in total_amount the new main df.
val df = PositiveFare
// Validate content of new main df.
df.show(3, truncate = false)
```

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	R
1	2025-01-01 00:18:38	2025-01-01 00:26:59	1	1.6	1
1	2025-01-01 00:32:40	2025-01-01 00:35:13	1	0.5	1
1	2025-01-01 00:44:04	2025-01-01 00:46:01	1	0.6	1

Figur 2.28: Utklipp fra Zeppelin celle: Overskrivning av original til behandlet datasett.

2.2.5 Filtrering på Antall Passasjerer

Her er målet å fjerne rader som inneholder 0 passasjerer, da dette ikke gir mening i praksis. Dette blir løst på følgende måte:

- Lager et midlertidig datasett som holder på de radene vi ønsker å beholde.
- Overskriver hoveddatasettet med det midlertidige datasettet.

```
%spark
val passengerMoreThanZero = df.filter(col("passenger_count") > 0)
passengerMoreThanZero: org.apache.spark.sql.Dataset[org.apache.spark
%spark
val df = passengerMoreThanZero
df: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [Vendor
```

Figur 2.29: Utklipp fra Zeppelin celle: Fjerning av rader med 0 passasjerer.

Nå som hoveddatasettet er ferdig behandlet, kan vi gå videre til å bruke dette til å løse de resterende oppgavene.

2.2.5.1 Bygge Kolonnene 'trip_duration_min', 'hour_of_day' og 'day_of_week'

Dette ble løst i seksjon [2.2.4.1](#) (~Figur 2.18).

Følgende løsninger ble gjort for å oppnå dette:

- **trip_duration_min**
Siden reisetid allerede hadde blitt kalkulert for hastighetproblemstillingen, var dette en enkel løsning ved å konvertere timer til minutter.

```

    )
    // Travel duration in minutes.
    .withColumn(
        "trip_duration_min",
        when(not(col("duration_hours").isNull),
            col("duration_hours") * 60
        ).otherwise(lit(null).cast("double")))
    )

```

Figur 2.30: Utklipp fra Zeppelin celle: Utklipp hvor reisetid blir konvertert til minutter.

· **hour_of_day**

Dette ble løst ved å benytte funksjonen `date_format`[8] med `k`[9] som parameter på kolonnen for hentetidspunktet. På denne måten ble det opprettet en kolonne som inneholdt verdiene 1 – 24 som videre representerte hver hele time i et døgn basert på en 24-timers klokke.

```

%spark
// Build trip_duration_min, hour_of_day, day_of_week.
val dfWithSpeed =
    df
        .withColumn("pickup_day", col("tpep_pickup_datetime"))
        .withColumn("hour_of_day", date_format(col("pickup_day"), "k"))

```

Figur 2.31: Utklipp fra Zeppelin celle: Utklipp hvor hentetidspunkt blir konvertert til 24-timers klokkeslett.

· **day_of_week**

Dette ble løst på tilnærmet samme måte som `'hour_of_day'`, men her ble funksjonen `dayofweek`[10] benyttet. Denne funksjonen returnerer en verdi mellom 1 – 7 som representerer ukedager. Her ble det opprettet to kolonner som representerer henting og levering.

```

// Build trip_duration_min, hour_of_day, day_of_week.
val dfWithSpeed =
    df
        .withColumn("pickup_day", col("tpep_pickup_datetime"))
        .withColumn("dropoff_day", col("tpep_dropoff_datetime"))
        .withColumn("pickup_day", dayofweek(col("pickup_day")))
        .withColumn("dropoff_day", dayofweek(col("dropoff_day")))

```

Figur 2.32: Utklipp fra Zeppelin celle: Utklipp hvor henting/levering blir konvertert til ukedager.

2.2.5.2 Kalkulere Daglige Turer

Når denne oppgaven ble utført, så ble det oppdaget to ganske ideelle løsninger som innfridde kravene.

Alternativ a: `.agg count`.

I denne løsningen dannes et nytt datasett som:

- Henter ut hentetidspunkt fra hoveddatasettet.
- Grupperer hentetidspunkt etter dato og bruker `to_date`[11] funksjonen for å endre til ønsket format.
- Aggregerer[12] gjennom datasettet og teller antallet rader per dato.
- Sorterer datasettet til en stigende rekkefølge etter dato kolonnen med `orderBy`[13] funksjonen.

```
%spark
// Option 1 (which I personally prefer)
// Create a df containing the date and the number of trips.
val daily_trips = df
  .withColumn("dates", col("tpep_pickup_datetime"))
  .groupBy(to_date(col("dates")).as("date"))
  .agg(count(lit(1)).as("number_of_trips"))
  .orderBy(col("date"))
daily_trips.show(truncate = false)
```

date	number_of_trips
2007-12-05	1
2009-01-01	2
2024-12-31	20
2025-01-01	71524
2025-01-02	78563

Figur 2.33: Utklipp fra Zeppelin celle: Første løsning på liste over daglige turer.

Alternativ b: `.groupBy count`.

Dette er en tilnærmet lik løsning som den første, men her blir `date_trunc`[14] funksjonen brukt til å definere dag og det blir gjort en `count()` direkte på grupperingene fremfor aggregeringen som blir utført i den første løsningen.

```
%spark
// Option 2
// Create a df containing the date and the number of trips.
val daily_trips2 = df
  .withColumn("pickup", col("tpep_pickup_datetime"))
  .withColumn("date", date_trunc("DAY", col("pickup")))
  .groupBy("date")
  .count().as("number_of_trips")
  .orderBy("date")

daily_trips2.show(truncate = false)
```

date	count
2007-12-05 00:00:00	1
2009-01-01 00:00:00	2
2024-12-31 00:00:00	20
2025-01-01 00:00:00	71524
2025-01-02 00:00:00	78563

Figur 2.34: Utlipp fra Zeppelin celle: Andre løsning på liste over daglige turer.

For fremtidige løsninger av lignende sort ville jeg nok kanskje forsøkt på en kombinert løsning av disse to.

Kode 2.2: Tenkt løsning for fremtidige prosjekter

```
val daily_trips = df
  .withColumn("dates", col("tpep_pickup_datetime"))
  .groupBy(to_date(col("dates")).as("date"))
  .count().as("number_of_trips")
  .orderBy(col("date"))
```

2.2.5.3 Kalkulere Median for Reisetid

Denne oppgaven ble utført ved å benytte `median`^[15] funksjonen til å finne medianen til kolonnen `'trip_duration_min'`.

```
// Create a df finding the median of each trip duration in minutes.
val median_duration = df.select("trip_duration_min")
median_duration.select(median(col("trip_duration_min")).as("median_trip_duration")).show()

+-----+
|median_trip_duration|
+-----+
|          12.17|
+-----+
```

Figur 2.35: Utklipp fra Zeppelin celle: Kalkulert median for reisetid i minutter.

2.2.5.4 Kalkulere Total Billettpris per Dag

Denne oppgaven ble utført ved å:

- Hente ut kolonnen for avreise.
- Gruppere datasettet etter avreise dager.
- Aggregere på grupperingen[16].
- Sortere gruppen etter avreise dager.

```
%spark
// Create a df finding the total fare for each date.
val total_fare_by_day = df
  .withColumn("dates", col("tpep_pickup_datetime"))
  .groupBy(to_date(col("dates")).as("date"))
  .agg(round(sum(col("total_amount")), 2).as("total_fare_by_day"))
  .orderBy(col("date"))

total_fare_by_day.show(truncate = false)

+-----+-----+
|date      |total_fare_by_day|
+-----+-----+
|2007-12-05|22.75           |
|2009-01-01|170.27          |
|2024-12-31|519.87          |
|2025-01-01|2133149.93      |
|2025-01-02|2342147.83      |
|2025-01-03|2400697.83      |
+-----+-----+
```

Figur 2.36: Utklipp fra Zeppelin celle: Kalkulert samlet billettpris per dag.

2.2.6 Joins

2.2.6.1 Import av Taxi Soner

For å kunne løse oppgavene i denne seksjonen var det først nødvendig å laste inn et datasett for 'Taxi Soner'. Siden dette er en csv fil måtte importen utføres med litt andre argumenter i forhold til importen av hoveddatasettet som ble opprettet med data fra *parquet* filer.

Det fremstod også et behov for å manuelt definere et **schema**^[17] for dette datasettet, da kolonnen *LocationID* ble definert som en string verdi, da det virket mer rasjonelt å ha dette som en *integer* basert på kolonnens innhold.

```
%spark
// Import of MapZone data:

//File location.
val base = ("/sparkio/TLC/mapZones/taxi_zone_lookup.csv")

// Custom schema (LocationID got defined as a string without)
val zoneSchema = StructType(Array(
  StructField("LocationID", IntegerType, true),
  StructField("Borough", StringType, true),
  StructField("Zone", StringType, true),
  StructField("service_zone", StringType, true)
))

// Import of data to df.
val dfZone = spark.read
  .option("header", "true")
  .schema(zoneSchema)
  .csv(base)

// Validating content of new df.
dfZone.show(3, truncate = false)
```

LocationID	Borough	Zone	service_zone
1	EWB	Newark Airport	EWB
2	Queens	Jamaica Bay	Boro Zone
3	Bronx	Allerton/Pelham Gardens	Boro Zone

Figur 2.37: Utklipp fra Zeppelin celle: Opprettelse av datasettet for taxi soner.

Etter opprettelse av dette datasettet, ble det foretatt noen enkle valideringer for å sikre at innholdet var som forventet.

```
dfZone.printSchema()

root
 |-- LocationID: integer (nullable = true)
 |-- Borough: string (nullable = true)
 |-- Zone: string (nullable = true)
 |-- service_zone: string (nullable = true)

dfZone.filter("LocationID = 1").select("Zone").show()

+-----+
|      Zone      |
+-----+
|Newark Airport|
+-----+
```

Figur 2.38: Utklipp fra Zeppelin celle: Validering av innhold i taxi soner datasettet.

2.2.6.2 Left-Join Lokasjonnavn til Hoveddatasett

Løser denne oppgaven ved å opprette en midlertidig datasett kopi av hoveddatasettet, hvor det blir utført to venstre orienterte `join`[\[18\]](#) operasjoner. Dette ble gjort for å få navn på lokasjon for på-/ og avstigning.

Her benyttes `alias`[\[19\]](#) funksjonen for å skape et skille mellom de to `join` operasjonene som utføres i samme celle.

Når handlingen var utført og innhold i midlertidig datasett var korrekt, ble hoveddatasettet overskrevet av den midlertidige, slik at hoveddatasettet nå hadde to nye kolonner for stedsnavn for på-/ og avstigning.

```
// Create new df to contain location names corresponding with locationIDs.
// Two left joins to add pickup and dropoff location names.
val dfWithLocationNames =
  df.alias("m")
    .join(dfZone.alias("zpu"),
      col("m.PULocationID") === col("zpu.LocationID"),
      "left")
    .join(dfZone.alias("zdo"),
      col("m.DOLocationID") === col("zdo.LocationID"),
      "left")
    .select(col("m.*"),
      col("zpu.Zone").as("pickup_location_name"),
      col("zdo.Zone").as("dropoff_location_name"))

// Validate content of new df.
dfWithLocationNames.show(3, truncate = false)
```

Figur 2.39: Utklipp fra Zeppelin celle: Opprettelse av datasetet med lokasjonnavn.

2.2.6.3 Aggregering: Top 10 Lokasjoner

a) Antall turer(volum)

Løser denne oppgaven ved å opprette et datasett som:

- Hente ut sonenavn fra hoveddatasettet.
- Grupper datasettet etter sonenavn.
- Aggregere en teller på tilsvarende kolonne fra hoveddatasettet og setter tilhørende telle verdier på grupperte soner.
- Sorterer datasettet etter utført telling.

```
// new df containing a decending list of trip volume per location.
val topTripVolume = df
  .withColumn("zone", col("pickup_location_name"))
  .groupBy(col("zone"))
  .agg(count(col("pickup_location_name")).as("trip_volume"))
  .orderBy(desc("trip_volume"))

topTripVolume.show(10, truncate = false)
```

zone	trip_volume
JFK Airport	1071452
Upper East Side South	1059958
Midtown Center	1043638

Figur 2.40: Utklipp fra Zeppelin celle: Opprettelse av top 10 lokasjoner for turer etter volum.

b) Summering av billettpriser

Løser denne oppgaven ved å opprette et datasett som:

- Hente ut sonenavn fra hoveddatasettet.
- Grupper datasettet etter sonenavn.
- Aggregere en `sum[20]` på `'total_amount'` kolonnen fra hoveddatasettet og setter tilhørende telle verdier på grupperte soner.
- Sorterer datasettet etter utført summering.

```
// new df containing a decending list of total fare per location.
val topTripTotalFare = df
  .withColumn("zone", col("pickup_location_name"))
  .groupBy(col("zone"))
  .agg(round(sum("total_amount"), 2).as("total_fare"))
  .orderBy(desc("total_fare"))

topTripTotalFare.show(10, truncate = false)
```

zone	total_fare
JFK Airport	8.745772521E7
LaGuardia Airport	4.902748448E7
Midtown Center	2.715074768E7
Upper East Side South	2.195396198E7

Figur 2.41: Utklipp fra Zeppelin celle: Opprettelse av top 10 lokasjoner for summerte billettpriser.

c) Snitt i tips

Løser denne oppgaven ved å opprette et datasett som:

- Hente ut sonenavn fra hoveddatasettet.
- Grupper datasettet etter sonenavn.
- Aggregere en `avg[21]` på `'tip_amount'` kolonnen fra hoveddatasettet og setter tilhørende telle verdier på grupperte soner.
- Sorterer datasettet etter kalkulert snitt i tips.

```
// new df containing a decending list of average tip per location.
val topTripAvgTip = df
  .withColumn("zone", col("pickup_location_name"))
  .groupBy(col("zone"))
  .agg(round(avg("tip_amount"), 2).as("average_tip"))
  .orderBy(desc("average_tip"))

topTripAvgTip.show(10, truncate = false)
```

zone	average_tip
West Brighton	33.72
Oakwood	22.57
Rossville/Woodrow	17.49
New Dorp/Midland Beach	16.25

Figur 2.42: Utlipp fra Zeppelin celle: Opprettelse av top 10 lokasjoner for snitt tips.

2.2.7 Window Funksjonalitet

2.2.7.1 Top 5 Soner Daglig Billettpris Liste

For å utføre denne oppgaven har følgende steg blitt tatt.

- Opprettet et datasett som med følgende data hentet fra hoveddatasettet:
 - Dato for påstigning (navngitt til 'day')
 - Dag for påstigning (navngitt til 'week_day')
 - Navn på soner (navngitt til 'zone')
 - Datasettet grupperes etter dag→ukedag→sone.
 - Aggregerer med summering på kolonnen 'total_amount' fra hoveddatasettet og navngir en ny kolonne som 'total_fare'.
 - Sorterer datasettet etter kolonnen 'day'.
- Oppretter en **window**[\[22\]](#) funksjon som partisjonerer etter 'day' og sorterer etter 'total_fare'.
- Gjør et kall på opprettet datasett med følgende handlinger:
 - Oppretter en ny kolonne under navnet 'rank', som blir definert til å utføre en **dense_rank()**[\[23\]](#) basert på **window** funksjonen via **over()**[\[24\]](#) definisjonen.
 - Filtrerer resultatet på top 5 i 'rank' kolonnen.

Det er verdt å nevne at siden `dense_rank()` blir benyttet i denne løsningen vil dette resultere i mer enn fem plasseringer på dager hvor det er topplasseringer som har like store beløper.

```
// A solution that rank the top 5 pickup zones by total fare.
// Implemented with window, partitionBy and dense_rank.

// Build a new df containing pickup dates, zone name and total fare.
val topFiveZonByTotFare = df
  .withColumn(
    "day",
    col("tpep_pickup_datetime").cast("timestamp")
  )
  .withColumn(
    "week_day",
    col("pickup_day")
  )
  .withColumn(
    "zone",
    col("pickup_location_name")
  )
  .groupBy(
    to_date(col("day")).as("day"),
    col("week_day"),
    col("zone")
  )
  .agg(round(sum("total_amount"), 2).as("total_fare"))
  .orderBy(col("day"))

// Build the window function to implement on topFiveZonByTotFare
val window = Window
  .partitionBy("day")
  .orderBy("total_fare")

// Implement the window function to create a scoreboard of best total fare per zone per day down to the fifth rank.
// Since we're using dense_rank, we will get more than five ranks on a day when total fare has the same sum on locations which are in the topFiveZonByTotFare
topFiveZonByTotFare
  .withColumn("rank", dense_rank().over(window))
  .filter(col("rank") <= 5)
  .show(50, truncate = false)
```

day	week_day	zone	total_fare	rank
2007-12-05	4	Lincoln Square East	22.75	1
2009-01-01	5	LaGuardia Airport	170.27	1
2024-12-31	3	Old Astoria	10.4	1
2024-12-31	3	Corona	10.4	1
2024-12-31	3	Central Park	15.7	2
2024-12-31	3	Clinton East	17.16	3
2024-12-31	3	Murray Hill	18.0	4
2024-12-31	3	Lincoln Square East	20.52	5
2025-01-01	4	Kingsbridge Heights	9.4	1
2025-01-01	4	Belmont	12.5	2
2025-01-01	4	Sunset Park East	17.0	3

Figur 2.43: Utklipp fra Zeppelin celle: Top 5 Soner Daglig Billettpris med window funksjonalitet.

2.2.7.2 7-dagers Glidende Gjennomsnitt av Turer per Bydel.

For denne oppgaven forekom det en del forvirring for hva som faktisk ble spurt etter. På grunnlag av dette har det forekommet en lengre dialog med chat-GPT[25] for å forstå oppgaven.

Første forsøkt på å løse denne oppgaven som ble gjort uten assistanse resulterte i en **window** funksjon som utførte en **count** aggregering på antall turer om dagen per område, som videre ble sortert etter uker. Siden ble det mer klart at snitt for antall turer skulle adderes for neste dag. Det ble også feil på eget forsøk, da window ble satt til **rank()** fremfor **round(avg()** som ville være en mer passende

løsning for denne oppgaven.

For illustrasjon, så har feilløsningen blitt beholdt i Zeppelin dokumentet, mens bildet under er løsningen som faktisk utfører det oppgaven ber om.

```
// 1) Join trips + zones to get borough, derive calendar day (localize if needed)
val tripsPerDay =
  df.alias("t")
    .join(broadcast(dfZone.alias("z")), $"t.PULocationID" === $"z.LocationID", "left")
    .withColumn("pickup_ts", col("t.tpep_pickup_datetime").cast("timestamp"))
    .withColumn("day", to_date($"pickup_ts"))
    .groupBy($"z.Borough".as("borough"), $"day")
    .agg(count(lit(1)).as("daily_trips"))

// 2) Fill missing days with zero per borough (so the average isn't biased)
val dayBounds =
  tripsPerDay.groupBy("borough")
    .agg(min($"day").as("min_day"), max($"day").as("max_day"))

val calendar =
  dayBounds.select(
    $"borough",
    explode(sequence($"min_day", $"max_day", expr("interval 1 day"))).as("day")
  )

val dailyFilled =
  calendar.join(tripsPerDay, Seq("borough", "day"), "left")
    .select($"borough", $"day", coalesce($"daily_trips", lit(0L)).as("daily_trips"))

// 3) 7-day trailing moving average (current day + previous 6)
val w = Window.partitionBy("borough").orderBy($"day").rowsBetween(-6, 0)

val tripsWithMA7 =
  dailyFilled
    .withColumn("trips_ma7", round(avg($"daily_trips").over(w), 2))
    .orderBy($"borough", $"day")

tripsWithMA7.show(50, truncate = false)
```

```
+-----+-----+-----+-----+
|borough|day      |daily_trips|trips_ma7|
+-----+-----+-----+-----+
|Bronx  |2025-01-01|137       |137.0    |
|Bronx  |2025-01-02|240       |188.5    |
|Bronx  |2025-01-03|246       |207.67   |
|Bronx  |2025-01-04|167       |197.5    |
|Bronx  |2025-01-05|145       |187.0    |
```

Figur 2.44: Utklipp fra Zeppelin celle: 7-dagers glidende gjennomsnitt av turer per bydel.

2.2.8 Join Fallgruver og Ytelse

Denne seksjonen omhandler de siste cellene som ligger i Zeppelin dokumentet for TLC datasettet. I denne oppgaven har det vært mye famling i blinde, med bruk av mange forskjellige kilder for utførelse. Hovedkilden for denne oppgaven har vært pensum book for *Spark* [26], men det har også blitt benyttet dokumentasjon fra *spark.apache.org* [27], samt dialoger med chat-GPT [25].

2.2.8.1 Oppretter Måleverktøy

I denne cellen opprettes flere **def** metoder som skal brukes til å utføre forskjellige tester vedrørende ytelse og tidsforbruk. Her er det blitt brukt **def** fremfor **val**, siden dette skal lede til at det blir opprettet et nytt datasett for hver test fremfor at det blir gjort kall på eksisterende datasett for å unngå kontaminerte resultater på målinger.

- **def timed[T]**
En metode som kan benyttes som en måleenhet for hvor lang tid det tok en prosess å fullføre. Denne metoden skal da kunne implementeres som et kall i **bench** metoden under.
- **def bench**
Skal regenerere et valgt datasett og benytter **timed** til å måle tidsforbruk på **count** av det gitte datasettet.
- **def printPlan**
Skal printe ut en *header*, samt en formatert utgave av **explain** som viser til logiske og fysiske plan i et lesbart utkast. Med denne metoden burde det være mulig å enklere se ytelse forskjeller mellom **BroadcastHashJoin** og **SortMergeJoin/ShuffledHashJoin**.
Man skal også kunne lese mer detaljert om partisjonering.

```
%spark
import org.apache.spark.sql.{DataFrame, SparkSession}
import org.apache.spark.storage.StorageLevel

def timed[T](label: String)(thunk: => T): T = {
  val t0 = System.nanoTime()
  val out = thunk
  val ms = (System.nanoTime() - t0) / 1e6
  println(f"🕒 $label took $ms%.1f ms")
  out
}

// Materialize and time a DF
def bench(df: DataFrame, label: String = "action"): Long =
  timed(label){ df.count() } // count() is a cheap "force" for joins

def printPlan(df: DataFrame, label: String): Unit = {
  println(s"\n==== $label ====")
  df.explain("formatted") // look for BroadcastHashJoin vs SortMergeJoin
}

import org.apache.spark.sql.{DataFrame, SparkSession}
import org.apache.spark.storage.StorageLevel
timed: [T](label: String)(thunk: => T)T
bench: (df: org.apache.spark.sql.DataFrame, label: String)Long
printPlan: (df: org.apache.spark.sql.DataFrame, label: String)Unit
```

Figur 2.45: Utlipp fra Zeppelin celle: Oppretter metoder for estimering.

2.2.8.2 Demonstrasjons av 'non-broadcast' Join

I denne cellen er utført et forsøk på en *non-broadcast left join* mellom **TLC** og **MapZone** datasettet. Her blir `autoBroadcastJoinThreshold`[\[27\]](#) satt med parameter `-1` som skruer av *broadcast* til nodene i klysteret. Dette betyr at det blir fremtvunget en *shuffle-based(SortMergeJoin)* join som gjør at vi kan gjøre målinger på en *non-broadcast baseline*.

Videre blir det printet en eksekveringsplan med `printPlan`(wrap av `df.explain(formatted)`) på det nye datasettet som utfører *join* operasjonen. Dette datasettet blir initialisert med et kall på `bench` metoden, som utfører operasjonen og til slutt printer ut resultatet på tidsforbruk.

```
%spark
// Turn off auto-broadcast
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)

val joinNoBroadcast =
  dfTuning
    .join(dfZoneTuning, dfTuning("PULocationID") === dfZoneTuning("LocationID"), "left")
    .select(dfTuning("*"), dfZoneTuning("Borough").as("PU_Borough"), dfZoneTuning("Zone").as("PU_Zone"))

printPlan(joinNoBroadcast, "NO BROADCAST PLAN")
bench(joinNoBroadcast, "no-broadcast join materialize")
```

==== NO BROADCAST PLAN ====

== Physical Plan ==

AdaptiveSparkPlan (10)

+-- Project (9)

 +- SortMergeJoin LeftOuter (8)

 :- Sort (3)

 : +- Exchange (2)

 : +- Scan parquet (1)

 +- Sort (7)

 +- Exchange (6)

 +- Filter (5)

 +- Scan csv (4)

(1) Scan parquet

Output [20]: [VendorID#8060, tpep_pickup_datetime#8061, tpep_dropoff_datetime#8062, passenger_count#8063L, Batched: true

Location: InMemoryFileIndex [file:/sparkio/TLC/trips/2025/yellow_tripdata_2025-01.parquet, ... 6 entries]

ReadSchema: struct<VendorID:int,tpep_pickup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,pas

(2) Exchange

Input [20]: [VendorID#8060, tpep_pickup_datetime#8061, tpep_dropoff_datetime#8062, passenger_count#8063L, Arguments: hashpartitioning(PULocationID#8067, 200), ENSURE_REQUIREMENTS, [plan_id=11708]

(3) Sort

Input [20]: [VendorID#8060, tpep_pickup_datetime#8061, tpep_dropoff_datetime#8062, passenger_count#8063L, Arguments: [PULocationID#8067 ASC NULLS FIRST], false, 0

(4) Scan csv

Output [3]: [LocationID#8100, Borough#8101, Zone#8102]

Batched: false

Location: InMemoryFileIndex [file:/sparkio/TLC/mapZones/taxi_zone_lookup.csv]

PushedFilters: [IsNotNull(LocationID)]

ReadSchema: struct<LocationID:int,Borough:string,Zone:string>

(5) Filter

Input [3]: [LocationID#8100, Borough#8101, Zone#8102]

Condition : isNotNull(LocationID#8100)

(6) Exchange

Input [3]: [LocationID#8100, Borough#8101, Zone#8102]

Arguments: hashpartitioning(LocationID#8100, 200), ENSURE_REQUIREMENTS, [plan_id=11709]

(7) Sort

Input [3]: [LocationID#8100, Borough#8101, Zone#8102]

Arguments: [LocationID#8100 ASC NULLS FIRST], false, 0

(8) SortMergeJoin

Left keys [1]: [PULocationID#8067]

Right keys [1]: [LocationID#8100]

Join type: LeftOuter

Join condition: None

(9) Project

Output [22]: [VendorID#8060, tpep_pickup_datetime#8061, tpep_dropoff_datetime#8062, passenger_count#8063L, Input [23]: [VendorID#8060, tpep_pickup_datetime#8061, tpep_dropoff_datetime#8062, passenger_count#8063L,

(10) AdaptiveSparkPlan

Output [22]: [VendorID#8060, tpep_pickup_datetime#8061, tpep_dropoff_datetime#8062, passenger_count#8063L, Arguments: isFinalPlan=false

no-broadcast join materialize took 41629.1 ms

joinNoBroadcast: org.apache.spark.sql.DataFrame = [VendorID: int, tpep_pickup_datetime: timestamp_ntz ... 2

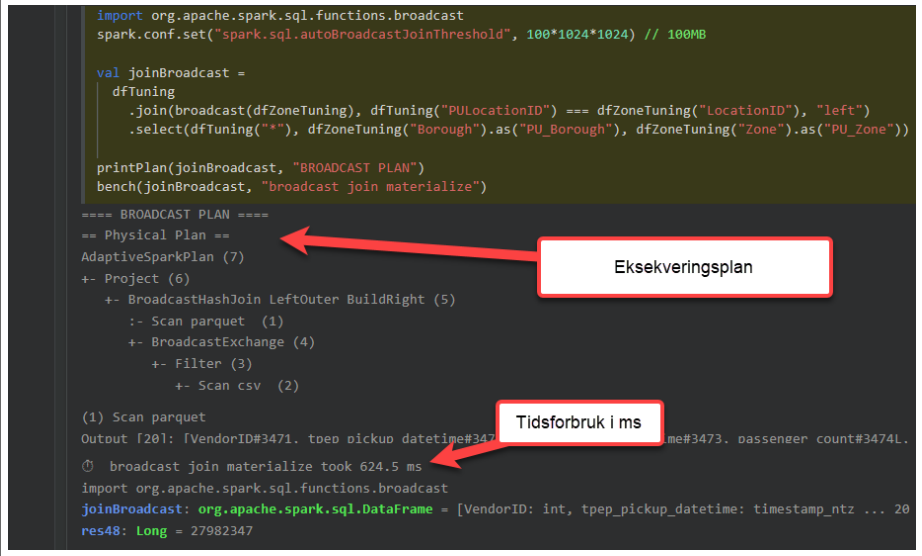
res170: Long = 27982347

Figur 2.46: Utklipp fra Zeppelin celle: Klippet bilde av *non-broadcast* test.

2.2.8.3 Demonstrasjons av 'broadcast' Join

Dette er tilsvarende test som i ovennevnte seksjon, men i dette tilfellet er det satt til en høyre *broadcast* verdi (100MB). Dette vil si at et ganske høyt antall med *bytes* av datasettet vil bli distribuert til arbeids noder i en *join* operasjon. Selve *join* operasjonen er identisk til *non-broadcast*, slik at tidsforbruket kan brukes til å sammenligne ytelse.

Resultatet viser til at *join* operasjonen er tilnærmet 10 ganger mer effektiv med en høy byte verdi på *broadcast* på kostnad av klysterets minne.



```
import org.apache.spark.sql.functions.broadcast
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 100*1024*1024) // 100MB

val joinBroadcast =
  dfTuning
    .join(broadcast(dfZoneTuning), dfTuning("PULocationID") === dfZoneTuning("LocationID"), "left")
    .select(dfTuning("*"), dfZoneTuning("Borough").as("PU_Borough"), dfZoneTuning("Zone").as("PU_Zone"))

printPlan(joinBroadcast, "BROADCAST PLAN")
bench(joinBroadcast, "broadcast join materialize")

==== BROADCAST PLAN ====
== Physical Plan ==
AdaptiveSparkPlan (7)
+- Project (6)
   +- BroadcastHashJoin LeftOuter BuildRight (5)
      :- Scan parquet (1)
      +- BroadcastExchange (4)
         +- Filter (3)
            +- Scan csv (2)

(1) Scan parquet
Output [201: [VendorID#3471, tpep_pickup_datetime#3471, ...], me#3473, passenger count#3474L, ...]
@ broadcast join materialize took 624.5 ms
import org.apache.spark.sql.functions.broadcast
joinBroadcast: org.apache.spark.sql.DataFrame = [VendorID: int, tpep_pickup_datetime: timestamp_ntz ... 20
res48: Long = 27982347
```

Eksekveringsplan

Tidsforbruk i ms

Figur 2.47: Utklipp fra Zeppelin celle: Klippet bilde av *high-broadcast* test.

2.2.8.4 Salting

Dataskjevhet er et vanlig problem i stordatabehandling. Det kan føre til ineffektiv ressursutnyttelse og lengre behandlingstider. *Apache Spark* tilbyr en teknikk kjent som *salting* for å håndtere skjeve data.

For å kunne utføre dette er vi nødt til å gjøre noen analytiske fortrinn av datasettet.

Her starter vi med å gruppere alle reiser på kolonnen *'PULocationID'* å gjør en telling på disse grupperingene for å se hvilke lokasjoner som har et så høyt antall at det vil påvirke ressursutnyttelse og behandlingstider. Resultatene for denne grupperingen vil da videre bli brukt til å velge hvilke grupperinger som har behov for *salting*.

```
val topPulocs =  
  dfTuning.groupBy($"PULocationID").count().orderBy(desc("count"))  
  topPulocs.show(10, truncate=false)
```

PULocationID	count
161	1237444
237	1213320
132	1161803
236	1080997
1370	1004473

Figur 2.48: Utklipp fra Zeppelin celle: Klippet bilde av 'tunge' grupper.

Her velger jeg de 4 tyngste toppgruppene og lager en variabel som holder på 'PULocationID' for disse. Denne kommer til å bli brukt for *salting*.

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)  
val heavyKeys = Seq(161, 237, 132, 236)
```

```
heavyKeys: Seq[Int] = List(161, 237, 132, 236)
```

Figur 2.49: Utklipp fra Zeppelin celle: Lagrer lokasjoner med høyt antall rader

Etter dette utfører jeg en ny prestasjons test med og uten *salting*

```
val joinedSkew =
  dfTuning
    .join(dfZoneTuning, $"PULocationID" === $"LocationID") // no broadcast, skewed key
    .select($"trip_distance", $"PULocationID", $"Zone")

printPlan(joinedSkew, "SKEWED JOIN (unsalted)")
bench(joinedSkew, "unsalted skewed join")

==== SKEWED JOIN (unsalted) ====
== Physical Plan ==
AdaptiveSparkPlan (11)
+- Project (10)
   +- SortMergeJoin Inner (9)
      :- Sort (4)
         : +- Exchange (3)
         :    +- Filter (2)
         :       +- Scan parquet (1)
      +- Sort (8)
         +- Exchange (7)
            +- Filter (6)
               +- Scan csv (5)

⏱ unsalted skewed join took 10395.6 ms
joinedSkew: org.apache.spark.sql.DataFrame = [trip_distance: double, PULocationID: int ...
res54: Long = 27982347
```

⏏ ⏴ ⏵ ☐ Highlight All ☐ Match Case ☐ Match Diacritics ☐ Whole Words 3 of 9

Figur 2.50: Utlipp fra Zeppelin celle: Join test usaltet

```
val saltBuckets = 16

// Left (big) side: assign salt for heavy keys; keep 0 for others
val dfSalted =
  dfTuning.withColumn(
    "salt",
    when(col("PULocationID").isin(heavyKeys:_*),
      pmod(hash(col("tpep_pickup_datetime")), lit(saltBuckets)) // deterministic spread
    ).otherwise(lit(0))
  )

// Right (small) side: replicate heavy keys across all salt buckets; keep 0 for others
val dfZoneSalted =
  dfZoneTuning
    .withColumn(
      "salt_array",
      when(col("LocationID").isin(heavyKeys:_*),
        sequence(lit(0), lit(saltBuckets - 1)) // 0..N-1
      ).otherwise(array(lit(0)))
    )
    .withColumn("salt", explode(col("salt_array")))
    .drop("salt_array")

val joinedSalted =
  dfSalted
    .join(dfZoneSalted,
      dfSalted("PULocationID") === dfZoneSalted("LocationID") &&
      dfSalted("salt") === dfZoneSalted("salt")
    )
    .drop(dfZoneSalted("salt"))

printPlan(joinedSalted, "SKEWED JOIN (salted)")
bench(joinedSalted, "salted skewed join")

① salted skewed join took 9714.4 ms
saltBuckets: Int = 16
dfSalted: org.apache.spark.sql.DataFrame = [VendorID: int, tpep_pickup_datetime: timestamp_ntz ... 19 more fields]
dfZoneSalted: org.apache.spark.sql.DataFrame = [LocationID: int, Borough: string ... 3 more fields]
joinedSalted: org.apache.spark.sql.DataFrame = [VendorID: int, tpep_pickup_datetime: timestamp_ntz ... 23 more fields]
res55: Long = 27982347
```

Figur 2.51: Utklipp fra Zeppelin celle: Join test saltet

Her kan vi se at prestasjonen har økt noe med salting, men at det kan vurderes å salte ytterligere flere av de øvre grupperingene, eller øke antallet med 'saltbuckets' for å potensielt øke prestasjonen videre.

2.2.8.5 Caching

Til slutt blir det gjort en måling av ytelse på lesing fra *cache* fremfor å lese direkte fra kildefilen. Her er jobben redusert til 3 kolonner for å lette på arbeidet som blir utført.

I dette tilfellet setter vi et skille på *trip_distance* for å se hvor langt tid den bruker på å lese øvre å nede grense for dette.

```
val base = dfTuning.select("PULocationID", "tpep_pickup_datetime", "trip_distance")
base.persist(StorageLevel.MEMORY_AND_DISK)
base.count() // materialize

val run1 = bench(base.filter($"trip_distance" > 3), "cached path 1")
val run2 = bench(base.filter($"trip_distance" <= 3), "cached path 2")

base.unpersist(true)

⌚ cached path 1 took 329.4 ms
⌚ cached path 2 took 246.6 ms
base: org.apache.spark.sql.DataFrame = [PULocationID: int, tpep_pickup_datetime: times
run1: Long = 8312574
run2: Long = 19669773
res58: base.type = [PULocationID: int, tpep_pickup_datetime: timestamp_ntz ... 1 more
```

Figur 2.52: Utklipp fra Zeppelin celle: Cache evaluering

2.3 Analysis Using the Movielens Dataset

2.3.1 Import av Pakker

Følgende pakker har blitt importert for å løse denne oppgave, i noen tilfeller har også pakker måtes importeres i den enkelte seksjonen i Zeppelin notebooken pga. problemer med kjøre når dette ikke har blitt gjort.

- **org.apache.spark.sql.types._**
Pakken importeres for å få tilgang til Spark sine data type klasser. Slik at vi kan bruke disse der det er behov for det.[2]
- **org.apache.spark.sql.DataFrame**
Denne pakken er importert for å få tilgang til funksjonalitet for å opprette dataframe objekter.[28]
- **org.apache.spark.sql.functions._**
Denne pakken inneholder kjernefunksjonalitet for sparkuttrykk. Slik som select, withColumn, agg, filter og mye mer.[4]
- **scala.math._**
Pakken gir tilgang til diverse matematiske operasjoner og er importert for å få tilgang til logaritmiske metoder.[29]
- **org.apache.spark.sql.expressions.Window**
Denne pakken ble importert for bruk av window funksjonalitet.[3]

2.3.2 Load and Normalize

2.3.2.1 Import av Data

For denne delen av oppgaven ble datasettene lastet ned via lenken oppgitt i oppgavedokumentet, videre ble de aktuelle datasettene lastet inn i spark som dataframes.

- **spark.read()**

Funksjon for å lese inn datasett i spark.[30]

```
val movies = spark.read
    .format("csv")
    .option("header", true)
    .option("inferSchema", true)
    .load("/sparkio/ml-32m/movies.csv")

val ratings = spark.read
    .format("csv")
    .option("header", true)
    .option("inferSchema", true)
    .load("/sparkio/ml-32m/ratings.csv")

val tags = spark.read
    .format("csv")
    .option("header", true)
    .option("inferSchema", true)
    .load("/sparkio/ml-32m/tags.csv")
```

Figur 2.53: Utklipp fra Zeppelin for import av datasett

For å validere at datasettene ble lastet inn riktig og å få en oversikt over datasettene, så ut ble det skrevet ut fem linjer for hver dataframe og gjort en telling av linjer.

- **.show()**

Funksjon for å skrive ut linjer fra en dataframe.[31]

- **.count()**

Funksjon for å telle linjer i en dataframe.[32]

```

+-----+-----+-----+-----+
|movieId|          title|          genres|
+-----+-----+-----+
|    1| Toy Story (1995)|Adventure|Animati...| |
|    2| Jumanji (1995)|Adventure|Childre...|
|    3|Grumpier Old Men ...| Comedy|Romance|
|    4|Waiting to Exhale...|Comedy|Drama|Romance|
|    5|Father of the Bri...|          Comedy|
+-----+-----+-----+
only showing top 5 rows

+-----+-----+-----+-----+
|userId|movieId|rating|timestamp|
+-----+-----+-----+-----+
|    1|    17|    4.0|944249077|
|    1|    25|    1.0|944250228|
|    1|    29|    2.0|943230976|
|    1|    30|    5.0|944249077|
|    1|    32|    5.0|943228858|
+-----+-----+-----+-----+
only showing top 5 rows

+-----+-----+-----+-----+
|userId|movieId|          tag| timestamp|
+-----+-----+-----+-----+
|    22|  26479|Kevin Kline|1583038886|
|    22|  79592|  misogyny|1581476297|
|    22| 247150|acrophobia|1622483469|
|    34|   2174|    music|1249808064|
|    34|   2174|    weird|1249808102|
+-----+-----+-----+-----+
only showing top 5 rows

(87585,32000204,2000072)

```

Figur 2.54: Utklipp fra Zeppelin for utskrift av dataframes og telling av linjer.

2.3.2.2 Konvertering av Timestamp til Dato

For å konvertere timestamps kolonnen i ratings til dato verdier lagde vi en ny dataframe, ratings_with_date hvor vi transformerte timestamp kolonnen til en ny kolonne rating_date før vi droppet den gamle timestamp kolonnen.

- **.withColumn()**
Funksjon for å lage en ny kolonne i en dataframe.[\[33\]](#)
- **.from_unixtime()**
Funksjon for å konvertere en timestamp verdi til en dato string.[\[34\]](#)
- **.to_date()**
Funksjon for å endre type fra en string til en dato.[\[35\]](#)
- **.drop()**
Funksjon for å fjerne en kolonne i en dataframe.[\[36\]](#)

```
// Parse timestamps to rating_date
val ratings_with_date = ratings
  .withColumn("rating_date", to_date(from_unixtime(col("timestamp"))))
  .drop("timestamp")

ratings_with_date.show(5)
```

userId	movieId	rating	rating_date
1	17	4.0	1999-12-03
1	25	1.0	1999-12-03
1	29	2.0	1999-11-22
1	30	5.0	1999-12-03
1	32	5.0	1999-11-22

only showing top 5 rows

Figur 2.55: Utklipp fra Zeppelin av kode for konvertering av timestamp til dato og test utskrift av dette.

2.3.2.3 Vurderinger per Dag

For å hente ut verdier for vurderinger per dag opprettet vi en ny dataframe, `ratings_per_date` hvor vi grupperte dataene etter vurderings dato og deretter tellet alle linjene. Videre sortere vi også linjene etter synkende rekkefølge og tilslutt for å sjekke at resultatet gjorde vi en test utskrift av den nye dataframen.

- **.groupBy()**

Funksjon for å grupper en datafram på en spesifikk kolonne.[\[16\]](#)

- **.orderBy()**

Funksjon for å sortere en dataframe etter en eller flere kolonner.[\[13\]](#)

```
// ratings per day
val ratings_per_date = ratings_with_date
  .groupBy("rating_date")
  .count()
  .orderBy(desc("count"))

ratings_per_date.show(5)
```

```
+-----+-----+
|rating_date|count|
+-----+-----+
| 2000-11-20|90717|
| 2005-03-22|74922|
| 1999-12-11|63064|
| 2008-10-29|53629|
| 2000-11-21|53514|
+-----+-----+
only showing top 5 rows
```

Figur 2.56: Utklipp fra Zeppelin av kode for vurdering per dag og test utskrift.

2.3.2.4 Distribusjon av Vurderings Verdier

Når vi skulle lage en distribusjon over de forskjellige vurderingsverdiene, grupperte vi `ratings_with_date` på `rating` kolonnen og tellet linje i hver av grupperingene og tilslutt brukte vi `sort` vi grupperingen i stigende rekkefølge før vi skrev ut distribusjonen.

```
// distribution of rating values
val rating_distribution = ratings_with_date
  .groupBy("rating")
  .count()
  .orderBy(asc("rating"))

rating_distribution.show()
```

rating	count
0.5	525132
1.0	946675
1.5	531063
2.0	2028622
2.5	1685386
3.0	6054990
3.5	4290105
4.0	8367654
4.5	2974000
5.0	4596577

Figur 2.57: Utklipp fra Zeppelin for kode for distribusjon av vurderings verdier og test utskrift.

2.3.2.5 Brukere med $\geq N$ Vurderinger

Vi løste denne oppgaven i to trinn. Først lagde vi en ny dataframe, `ratings_per_user` hvor vi tok `ratings_with_date` og grupperte på `userId` og deretter tellet antall linjer/vurderinger per `userId`. I det neste steget lagde vi en funksjon med input parameter, `N` på hvor mange vurderinger det skal sorteres etter, denne funksjonen returnerer en dataframe med alle brukerne som har lagt inn like mange eller flere vurderinger enn `N`. I denne funksjonen lager vi en nye retur dataframe ved å fjerne alle brukere i `ratings_per_user` som har mindre enn `N` vurderinger, deretter sortere vi brukerne i stigende rekkefølge ut ifra antall vurderinger. Tilslutt kjørte vi en test utskrift av denne funksjonen for å sjekke at den ga ønsket resultat.

- **.where()**

Funksjon for fjerning av linjer ut i fra spesifikke kriterier som settes i funksjonen.[37]

```
// users with ≥ N ratings
val ratings_per_user = ratings_with_date
  .groupBy("userId")
  .count()

def users_with_N_ratings(N: Int): DataFrame = {
  val returnDf = ratings_per_user.where($"count" >= N).orderBy(asc("count"))
  returnDf
}
users_with_N_ratings(19).show(5)
```

```
+-----+-----+
|userId|count|
+-----+-----+
|110327|  20|
|140611|  20|
|105339|  20|
|133949|  20|
|121283|  20|
+-----+-----+
only showing top 5 rows
```

Figur 2.58: Utklipp fra Zeppelin for kode og utskrift for brukere med $\geq N$ vurderinger.

2.3.3 Join ratings→movies to Compute

2.3.3.1 Top Movies Rangert på Antall Vurdering med Minst 100 Vurderinger

På denne oppgaven tok vi først å lagde en ny dataframe `ratings_with_count_per_movie` hvor vi tok ratings og grupperte på `movieId` og tellet antall vurderinger per `movieId`. Deretter lagde vi et join-expression som slår sammen `movies` med `ratings_with_count_per_movie` for alle linjer hvor `movieId` kolonnen for begge dataframene er like. Så gjorde vi selve sammenslåingen av dataframene til en ny dataframe, `movies_with_rating_count`. Videre fjernet vi alle `movies` som hadde mindre enn 100 vurderinger. For å unngå doble kolonner med samme verdi i den nye dataframe valgte vi spesifikt alle kolonnen vi ønsket å ta med oss videre etter sammenslåingen og tilslutt sorterte vi resultatet på antall vurderinger i synkende rekkefølge og gjorde en test utskrift.

- **.join()**

Funksjon for sammenslåing av dataframes.[18]

- **.select()**

Funksjon for å velge spesifikke kolonner i en dataframe.[38]

```
// Top movies by rating count (min 100 ratings)
val ratings_with_count_per_movie = ratings.groupBy("movieId").count()
val join_ex1 = movies.col("movieId") === ratings_with_count_per_movie.col("movieId")
val movies_with_rating_count = movies.join(ratings_with_count_per_movie, join_ex1)
    .where($"count" >= 100)
    .select(movies("movieId"), movies("title"), movies("genres"), ratings_with_count_per_movie("count"))
    .orderBy(desc("count"))

movies_with_rating_count.show(5)
```

movieId	title	genres	count
318	Shawshank Redempt...	Crime Drama	102929
356	Forrest Gump (1994)	Comedy Drama Roma...	100296
296	Pulp Fiction (1994)	Comedy Crime Dram...	98409
2571	Matrix, The (1999)	Action Sci-Fi Thr...	93808
593	Silence of the La...	Crime Horror Thri...	90330

only showing top 5 rows

Figur 2.59: Utklipp fra Zeppelin for kode og utskrift Top movies rangert på antall vurderinger.

2.3.3.2 Vektet Bayesian Vurdering og Topp 20 Movie Liste med Både Gjennomsnitt og Bayesian Verdi

Først lage vi en ny dataframe `ratings_with_avg` hvor vi tok `ratings_with_date` og grupperte på `movieId` før vi tok gjennomsnitt av alle vurderingene for hver `movieId` og tellet antall vurderinger per `movieId`. Videre lagde vi en variable med globalt gjennomsnittet av alle vurderingene for bruk i videre utregninger. Så lagde vi en ny dataframe `ratings_bay_score` hvor vi tok `ratings_with_avg` og lagde en ny kolonne med utregning for bayesian score for alle moviene. Deretter lagde vi et join-expression for å slå sammen `movies` og `rating_bay_score` hvor kolonnene for `movieId` er like, så brukte vi dette expressionet

til å slå sammen disse dataframene. Videre valgte vi de spesifikke kolonnen vi ønsket å ta med oss fra dataframene, slik at vi unngikk dobbelt kolonner med samme verdier, deretter sorterte vi resultatet i synkende rekkefølge først på gjennomsnittlig vurdering og deretter på bayesian score. Tilslutt begrenset vi dataframen til de topp 20 resultatene og gjorde en test utskrift.

- **.agg()**
Funksjon som tillater forskjellige utregninger over en kolonne i en dataframe.[16]
- **.as()**
Funksjon for å sette id for en ny kolonne til en spesifikk verdi.[16]
- **format_number()**
Funksjon for å formatere en tall verdi.[39]
- **.limit()**
Funksjon for å kun hente ut et gitt antall verdier for en dataframe.[40]

```
// top-20 table with both averages and the Bayesian score
val ratings_with_avg = ratings_with_date.groupBy("movieId")
    .agg(
        format_number(avg("rating"), 2).as("avg_rating"),
        count("rating").as("rating_count")
    )

val global_avg_rating = ratings_with_date.select(avg("rating")).first()
val ratings_bay_score = ratings_with_avg
    .withColumn("bayesian_rating", format_number(((($"rating_count" / ($"rating_count" + 100)) * $"avg_rating") + (lit(100) / ($"rating_count" + 100)) * global_avg_rating(0),2))
val join_ex2 = movies.col("movieId") === ratings_bay_score.col("movieId")
val movies_with_rating = movies.join(ratings_bay_score, join_ex2).where($"rating_count" >= 100)
    .select(movies("movieId"), movies("title"), movies("genres"), ratings_bay_score("rating_count"), ratings_bay_score("avg_rating"), ratings_bay_score("bayesian_rating"))
    .orderBy($"avg_rating".desc, $"bayesian_rating".desc)
    .limit(20)
movies_with_rating.show(30)
```

movieId	title	genres	rating_count	avg_rating	bayesian_rating
171011	Planet Earth II (...)	Documentary	1956	4.45	4.41
159817	Planet Earth (2006)	Documentary	2948	4.44	4.41
170705	Band of Brothers ...	Action Drama War	2811	4.43	4.40
318	Shawshank Redempt...	Crime Drama	102929	4.40	4.40
171495	Cosmos	(no genres listed)	615	4.33	4.22
850	Godfather, The (1...	Crime Drama	66440	4.32	4.32
202439	Parasite (2019)	Comedy Drama	11670	4.31	4.30
179135	Blue Planet II (2...	Documentary	1163	4.30	4.24
198185	Twin Peaks (1989)	Drama Mystery	1140	4.30	4.24
228528	Twelve Angry Men ...	Drama	449	4.29	4.15
1203	12 Angry Men (1957)	Drama	21863	4.27	4.27
50	Usual Suspects, T...	Crime Mystery Thr...	67750	4.27	4.27
1221	Godfather: Part I...	Crime Drama	43111	4.26	4.26
163809	Over the Garden W...	Adventure Animati...	1397	4.26	4.21
2019	Seven Samurai (Sh...	Action Adventure ...	16531	4.25	4.25
142115	The Blue Planet (...)	Documentary	1061	4.25	4.19
182723	Cosmos: A Spaceti...	(no genres listed)	594	4.25	4.15
527	Schindler's List ...	Drama War	73849	4.24	4.24
166024	Whiplash (2013)	(no genres listed)	3080	4.24	4.22
904	Rear Window (1954)	Mystery Thriller	24883	4.23	4.23

Figur 2.60: Utklipp fra Zeppelin for kode og utskrift av topp 20 movie liste sortert på gjennomsnitt og bayesian verdi.

2.3.4 Tags and Texty Aggregation

2.3.4.1 For Hver Movie Tell Hvor Mange Ganger en Spesifikk Tag Finnes

For å løse denne lagde vi en ny dataframe tags_with_count hvor vi tok tags og grupperte på kolonnen for movieId og tag, deretter tellet antall forekomster av en movieId og tag sammen. Tilslutt gjorde vi en test utskrift for å sjekke resultatet.

```
// For each movieId, count how often each tag appears
val tags_with_count = tags.groupBy("movieId", "tag").count()
tags_with_count.show(5)
```

```
+-----+-----+-----+
|movieId|      tag|count|
+-----+-----+-----+
|  61132|   parody|   40|
| 136020| espionage|   17|
|   5952|    magic|   66|
|  66659|tyler perry|    2|
|  59315|  military|   24|
+-----+-----+-----+
only showing top 5 rows
```

Figur 2.61: Utklipp fra Zeppelin for kode og utskrift av liste med antall forekomster av en tag for en movieId.

2.3.4.2 Top Tags for Hver Movie

Først lage vi en variabel for antall filmer i datasettet og konverterte denne til en double for senere utregninger. Videre lagde vi også en variabel for antall movies som har en eller flere tags, for å gjøre dette grupperte vi `movies_with_tags` på `movieId` og tellet denne grupperingen, deretter tellet vi antall linjer i dette utvalget for å få antall movies med minst en tag. Vi konverterte også denne verdien til en double. Så lagde vi en ny dataframe `top_tags` hvor vi tok `tags_with_count` og lagde en ny kolonne `min_tag_freq` med ønsket beregning for tag frekvens og sorterte denne i synkende rekkefølge for antall ganger en spesifikk tag forekommer for en `movieId`.

- **.toDouble()**

Funksjon for å konvertere typen til en verdi til en double.[\[41\]](#)

- **.log10()**

Funksjon for å ta 10'er logaritmen av en tallverdi.[\[29\]](#)

For å løse denne oppgaven måtte vi importere `scala.math._` for å kunne bruke `log10()`, vi prøvde i utgangspunktet å ha den importen på toppen i den første Zeppelin seksjonen, men vi fikk noen problemer med at dette ikke fungere hver gang vi kjørte hele Zeppelin filen samlet og vi fant ut at vi måtte ha importen på toppen av den aktuelle seksjonen for å sikre at det skulle fungere.

```
import scala.math._
// top tags per movie (min tag frequency)
val total_movies = movies.count().toDouble
val movies_with_tag = tags.groupBy("movieId").count().count().toDouble
val top_tags = tags_with_count.withColumn("min_tag_freq", format_number($"count" * (log10(total_movies / movies_with_tag)),2))
    .orderBy(desc("count"))

top_tags.show(5)
```

movieId	tag	count	min_tag_freq
260	sci-fi	596	138.34
79132	alternate reality	360	83.56
296	Quentin Tarantino	355	82.40
2959	twist ending	347	80.54
79132	thought-provoking	339	78.69

only showing top 5 rows

Figur 2.62: Utklipp fra Zeppelin for kode og utskrift for top tags for hver movie.

2.3.5 User-level Windows and Cohorts

2.3.5.1 Først og Siste Dato en Bruker Registrerte en Vurdering

For å løse denne oppgaven lagde vi en ny dataframe `user_first_last` hvor vi tok `ratings_with_date` og grupperte på `userId` og fant min og maks verdi for dato for hver enkelt bruker å lagde nye kolonner for disse.

For å løse denne oppgaven så har vi brukt funksjoner som krever import av `org.apache.spark.sql.functions._`, vi har i utgangspunktet en import av denne på toppen av den første seksjonen i Zeppelin filen da vi bruker denne andre steder også, men vi har hatt noen problemer med at denne seksjonen ikke vil kjøre hver gang vi starter opp notebooken uten at det ligger en egen import i denne seksjonen.

```
import org.apache.spark.sql.functions._
// Per-user: first/last rating date
val user_first_last = ratings_with_date.groupBy($"userId")
    .agg(
        min($"rating_date").as("first"),
        max($"rating_date").as("last")
    )

user_first_last.show(5)
```

```
+-----+-----+-----+
|userId|   first|   last|
+-----+-----+-----+
|  148|2016-08-21|2016-08-22|
|  463|2016-09-25|2016-09-25|
|  471|1996-08-20|1996-08-20|
|  496|2021-10-07|2021-10-07|
|  833|2006-05-17|2009-06-03|
+-----+-----+-----+
only showing top 5 rows
```

Figur 2.63: Utklipp fra Zeppelin for kode og utskrift for første og siste dato en bruker registrerte en vurdering.

2.3.5.2 Månedlig Aktivitet med Vurdering per Bruker

Siden vi på denne oppgaven er ute etter å hente ut månedlig aktivitet/vurderinger per bruker så tok vi å lagde en ny dataframe ut i fra `ratings_with_date` hvor vi først lagde en ny kolonne `month` for vi hentet ut kun måned og år fra `rating_date`, deretter grupperte vi på `userId` og `month` før vi tellet antall linjer for disse før vi sorterte den nye dataframen etter `userId` og `month`. Tilslutt skrev vi ut en test av den nye dataframe for å se at vi fikk ønsket resultat.

```
// Per-user: monthly activity
val monthly_activity = ratings.with_date
  .withColumn("month", date_format($"rating_date", "yyyy-MM"))
  .groupBy($"userId", $"month")
  .count()
  .orderBy($"userId", $"month")
monthly_activity.show()
```

userId	month	count
1	1999-11	60
1	1999-12	81
2	1996-07	52
3	2004-05	147
4	2000-06	27
5	1996-08	33
6	2004-11	26
7	1996-08	44
8	2019-03	31
9	2006-01	23
9	2006-02	18
9	2006-03	17
10	2007-01	175
10	2007-02	15
10	2007-03	8
10	2007-04	14
10	2007-05	5
10	2007-06	2
10	2007-07	8
10	2007-08	2

only showing top 20 rows

Figur 2.64: Utklipp fra Zeppelin for kode og utskrift for månedlig aktivitet med vurdering per bruker.

2.3.5.3 Rullende 3 Måneders Telling av Vurdering per Bruker

På denne oppgave prøvde vi mange forskjellige løsninger ved bruk av window funksjonalitet, men vi klarte ikke helt få til ønsket resultat, så vi måtte ty til litt hjelp fra GTP-Uio. Når vi testet løsningen GTP-Uio forslø så skjønnte vi at vi hadde prøvd å løse denne på en mer komplisert måte enn nødvendig ved bruk forskjellige konverteringer frem og tilbake mellom dato og timestamps. Siden vi i den foregående oppgaven alt hadde fått laget en dataframe med månedlig telling av vurderinger per bruker så kunne vi bruke denne videre å lage et rullende vindu basert på de to foregående radene/månedene og den nåværende raden/måneden, slik at vi fikk en løpende telling over 3 rader/måneder.[\[42\]](#)

- **.Window.rowsBetween()**

Funksjon for å lage et window som kan brukes til å lage et rullende vidu over en kolonne i en dataframe.[\[43\]](#)

- **.over()**

Funksjon for å implementere et window over en gitt kolonne i en dataframe.[\[24\]](#)

```
//rolling 3 month count (includes over time).
val user = ratings
  .groupBy($"userId")
  .orderBy($"month")
  .windowSpec(1, 3)

val monthly_activity_with_rolling_avg = monthly_activity.withColumn("rolling_3_month_count", user($"count").over(user($"userId")))
monthly_activity_with_rolling_avg.show()
```

userId	month	count	rolling_3_month_count
1	1999-11	60	60
1	1999-12	81	141
2	1996-07	52	52
2	1996-08	44	96
2	1996-09	37	133
2	1996-10	25	160
2	1996-11	26	186
2	1996-12	33	219
2	1997-01	33	252
2	1997-02	35	287
2	1997-03	40	327
2	1997-04	37	364
2	1997-05	31	395
2	1997-06	21	426
2	1997-07	8	457
2	1997-08	2	488
2	1997-09	2	519
2	1997-10	2	550
2	1997-11	2	581
2	1997-12	2	612
2	1998-01	2	643
2	1998-02	2	674
2	1998-03	2	705
2	1998-04	2	736
2	1998-05	2	767
2	1998-06	2	798
2	1998-07	2	829
2	1998-08	2	860
2	1998-09	2	891
2	1998-10	2	922
2	1998-11	2	953
2	1998-12	2	984
2	1999-01	2	1015
2	1999-02	2	1046
2	1999-03	2	1077
2	1999-04	2	1108
2	1999-05	2	1139
2	1999-06	2	1170
2	1999-07	2	1201
2	1999-08	2	1232
2	1999-09	2	1263
2	1999-10	2	1294
2	1999-11	2	1325
2	1999-12	2	1356
2	2000-01	2	1387
2	2000-02	2	1418
2	2000-03	2	1449
2	2000-04	2	1480
2	2000-05	2	1511
2	2000-06	2	1542
2	2000-07	2	1573
2	2000-08	2	1604
2	2000-09	2	1635
2	2000-10	2	1666
2	2000-11	2	1697
2	2000-12	2	1728
2	2001-01	2	1759
2	2001-02	2	1790
2	2001-03	2	1821
2	2001-04	2	1852
2	2001-05	2	1883
2	2001-06	2	1914
2	2001-07	2	1945
2	2001-08	2	1976
2	2001-09	2	2007
2	2001-10	2	2038
2	2001-11	2	2069
2	2001-12	2	2100
2	2002-01	2	2131
2	2002-02	2	2162
2	2002-03	2	2193
2	2002-04	2	2224
2	2002-05	2	2255
2	2002-06	2	2286
2	2002-07	2	2317
2	2002-08	2	2348
2	2002-09	2	2379
2	2002-10	2	2410
2	2002-11	2	2441
2	2002-12	2	2472
2	2003-01	2	2503
2	2003-02	2	2534
2	2003-03	2	2565
2	2003-04	2	2596
2	2003-05	2	2627
2	2003-06	2	2658
2	2003-07	2	2689
2	2003-08	2	2720
2	2003-09	2	2751
2	2003-10	2	2782
2	2003-11	2	2813
2	2003-12	2	2844
2	2004-01	2	2875
2	2004-02	2	2906
2	2004-03	2	2937
2	2004-04	2	2968
2	2004-05	2	2999
2	2004-06	2	3030
2	2004-07	2	3061
2	2004-08	2	3092
2	2004-09	2	3123
2	2004-10	2	3154
2	2004-11	2	3185
2	2004-12	2	3216
2	2005-01	2	3247
2	2005-02	2	3278
2	2005-03	2	3309
2	2005-04	2	3340
2	2005-05	2	3371
2	2005-06	2	3402
2	2005-07	2	3433
2	2005-08	2	3464
2	2005-09	2	3495
2	2005-10	2	3526
2	2005-11	2	3557
2	2005-12	2	3588
2	2006-01	2	3619
2	2006-02	2	3650
2	2006-03	2	3681
2	2006-04	2	3712
2	2006-05	2	3743
2	2006-06	2	3774
2	2006-07	2	3805
2	2006-08	2	3836
2	2006-09	2	3867
2	2006-10	2	3898
2	2006-11	2	3929
2	2006-12	2	3960
2	2007-01	2	3991
2	2007-02	2	4022
2	2007-03	2	4053
2	2007-04	2	4084
2	2007-05	2	4115
2	2007-06	2	4146
2	2007-07	2	4177
2	2007-08	2	4208
2	2007-09	2	4239
2	2007-10	2	4270
2	2007-11	2	4301
2	2007-12	2	4332
2	2008-01	2	4363
2	2008-02	2	4394
2	2008-03	2	4425
2	2008-04	2	4456
2	2008-05	2	4487
2	2008-06	2	4518
2	2008-07	2	4549
2	2008-08	2	4580
2	2008-09	2	4611
2	2008-10	2	4642
2	2008-11	2	4673
2	2008-12	2	4704
2	2009-01	2	4735
2	2009-02	2	4766
2	2009-03	2	4797
2	2009-04	2	4828
2	2009-05	2	4859
2	2009-06	2	4890
2	2009-07	2	4921
2	2009-08	2	4952
2	2009-09	2	4983
2	2009-10	2	5014
2	2009-11	2	5045
2	2009-12	2	5076
2	2010-01	2	5107
2	2010-02	2	5138
2	2010-03	2	5169
2	2010-04	2	5200
2	2010-05	2	5231
2	2010-06	2	5262
2	2010-07	2	5293
2	2010-08	2	5324
2	2010-09	2	5355
2	2010-10	2	5386
2	2010-11	2	5417
2	2010-12	2	5448
2	2011-01	2	5479
2	2011-02	2	5510
2	2011-03	2	5541
2	2011-04	2	5572
2	2011-05	2	5603
2	2011-06	2	5634
2	2011-07	2	5665
2	2011-08	2	5696
2	2011-09	2	5727
2	2011-10	2	5758
2	2011-11	2	5789
2	2011-12	2	5820
2	2012-01	2	5851
2	2012-02	2	5882
2	2012-03	2	5913
2	2012-04	2	5944
2	2012-05	2	5975
2	2012-06	2	6006
2	2012-07	2	6037
2	2012-08	2	6068
2	2012-09	2	6099
2	2012-10	2	6130
2	2012-11	2	6161
2	2012-12	2	6192
2	2013-01	2	6223
2	2013-02	2	6254
2	2013-03	2	6285
2	2013-04	2	6316
2	2013-05	2	6347
2	2013-06	2	6378
2	2013-07	2	6409
2	2013-08	2	6440
2	2013-09	2	6471
2	2013-10	2	6502
2	2013-11	2	6533
2	2013-12	2	6564
2	2014-01	2	6595
2	2014-02	2	6626
2	2014-03	2	6657
2	2014-04	2	6688
2	2014-05	2	6719
2	2014-06	2	6750
2	2014-07	2	6781
2	2014-08	2	6812
2	2014-09	2	6843
2	2014-10	2	6874
2	2014-11	2	6905
2	2014-12	2	6936
2	2015-01	2	6967
2	2015-02	2	6998
2	2015-03	2	7029
2	2015-04	2	7060
2	2015-05	2	7091
2	2015-06	2	7122
2	2015-07	2	7153
2	2015-08	2	7184
2	2015-09	2	7215
2	2015-10	2	7246
2	2015-11	2	7277
2	2015-12	2	7308
2	2016-01	2	7339
2	2016-02	2	7370
2	2016-03	2	7401
2	2016-04	2	7432
2	2016-05	2	7463
2	2016-06	2	7494
2	2016-07	2	7525
2	2016-08	2	7556
2	2016-09	2	7587
2	2016-10	2	7618
2	2016-11	2	7649
2	2016-12	2	7680
2	2017-01	2	7711
2	2017-02	2	7742
2	2017-03	2	7773
2	2017-04	2	7804
2	2017-05	2	7835
2	2017-06	2	7866
2	2017-07	2	7897
2	2017-08	2	7928
2	2017-09	2	7959
2	2017-10	2	7990
2	2017-11	2	8021
2	2017-12	2	8052
2	2018-01	2	8083
2	2018-02	2	8114
2	2018-03	2	8145
2	2018-04	2	8176
2	2018-05	2	8207
2	2018-06	2	8238
2	2018-07	2	8269
2	2018-08	2	8300
2	2018-09	2	8331
2	2018-10	2	8362
2	2018-11	2	8393
2	2018-12	2	8424
2	2019-01	2	8455
2	2019-02	2	8486
2	2019-03	2	8517
2	2019-04	2	8548
2	2019-05	2	8579
2	2019-06	2	8610
2	2019-07	2	8641
2	2019-08	2	8672
2	2019-09	2	8703
2	2019-10	2	8734
2	2019-11	2	8765
2	2019-12	2	8796
2	2		

2.3.5.4 Bruker Kohorter Oppdelt på Første Måned med en Registrert Vurdering

På denne oppgaven lagde vi en ny dataframe `cohort_first` ved å ta `user_first_last` og lage en ny kolonne `first_month` hvor vi konverterte dato verdien i `first` til en dato verdi kun med kun år og måned, deretter grupperte vi på `first_month` og tellet antall linjer for hver måned før vi tilslutt sorterte etter `first_month` og kjørte en test utskrift.

- **.date_format()**

Funksjon for formatere dato verdier til ønsket format.[\[8\]](#)

```
// kode for Cohorts by first-rating month.
val cohort_first = user_first_last
  .withColumn("first_month", date_format($"first", "yyyy-MM"))
  .groupBy($"first_month").count().orderBy($"first_month")
cohort_first.show(5)
```

```
+-----+-----+
|first_month|count|
+-----+-----+
| 1995-01|    2|
| 1996-01|    6|
| 1996-02|   20|
| 1996-03|  226|
| 1996-04| 1215|
+-----+-----+
only showing top 5 rows
```

Figur 2.66: Utklipp fra Zeppelin for kode og utskrift for kohorter oppdelt på første måned med en registrert vurdering.

2.3.5.5 Aktive Brukere +1, +2, +3 Måneder etter Første Vurdering

For å løse denne oppgaven tok vi først å lagde en ny dataframe retention. For å lage denne tok vi `user_first_last` og lagde en ny kolonne `months_activ` hvor vi tellet antall måneder mellom dato verdiene i kolonnene `first` og `last` og deretter konverterte denne verdien til en int slik at vi fikk en verdi for antall hele måneder mellom `first` og `last`. Etter dette fjernet vi alle brukere med `months_activ` verdi mindre enn 0, grupperte på `months_activ` og deretter tellet alle linjene for hver verdi av `months_activ`. Tilslutt lagde vi variabler for aktive brukere etter +1, +2 og +3 måneder ved å bruke retention dataframen til å summere verdiene til `months_activ` ut i fra riktige betingelser.

- **.cast()**
Funksjon for å type verdier i en kolonne til en spesifikk type.[\[44\]](#)
- **.first()**
Funksjon for å hente ut den første raden i en dataframe.[\[45\]](#)
- **.getLong()**
Funksjon for å hente long verdi med en gitt indeks.[\[46\]](#)

```
// kode for retention = share of users active in month +1,+2, +3.
val retention = user_first_last
    .withColumn("months_activ", months_between($"last", $"first"))
    .cast(IntegerType)
    .where($"months_activ" > 0)
    .groupBy($"months_activ")
    .count()
    .orderBy(asc("months_activ"))

val mon1 = retention.where($"months_activ" > 0).agg(sum($"count")).first().getLong(0)
val mon2 = retention.where($"months_activ" > 1).agg(sum($"count")).first().getLong(0)
val mon3 = retention.where($"months_activ" > 2).agg(sum($"count")).first().getLong(0)

println(s"Users activ after +1 month: $mon1, +2 months: $mon3, +3months: $mon3")
```

```
Users activ after +1 month: 55266, +2 months: 44897, +3months: 44897
```

Figur 2.67: Utklipp fra Zeppelin for kode og utskrift for Aktive brukere +1, +2, +3 måneder etter første vurdering.

2.4 Cassandra Database Design og Bruk

2.4.1 Modeling TLC in Cassandra

For å løse følgende TLC orienterte oppgaver under denne seksjonen, har vi opprettet tre separate Zeppelin filer. Dette er gjort for at det skal være enklere å kunne kjøre hele *notebook* filer uten å måtte tenke på hvilke celler som kun skal kjøre en gang.

- **TLC_CQL_DB_Build**
Denne notebook er et rent Cassandra dokument, som bygger en CQL database med *keyspace*

og tabeller som er strukturert til å passe med etterspurte spørringer i denne oppgaven. Dokumentet er ment til å kjøres kun en gang. Deretter kan neste dokument i denne listen ([TLC_Frame_to_Table](#)) kjøres for å legge inn data i CQL basens tabeller.

- **TLC_Frame_to_Table**

Dette er et **spark** dokument som henter ned og strukturerer en TLC *dataframe* med mye av samme logiske løsninger som vist i fra seksjon 2.2.2 til og med seksjon 2.2.4.1, samt seksjon 2.2.6. Siden vi allerede har gjennomgått validering prosessen i ovennevnte seksjoner, har vi i dette dokumentet laget mer kompakt løsning med færre valideringer, da hovedfokuset i dette dokumentet er å forme *dataframes* til å passe inn i opprettet database miljø fra *TLC_CQL_Table_Build* dokumentet, for deretter å eksportere data til sine respektive tabeller. Dette dokumentet er ment til å kjøres en gang etter at **CQL** tabeller er opprettet. Vi har valgt å forbehandle dataene i **spark** før eksport til **Cassandra**. Grunnlaget for dette ligger i manglende *AdHoc*[47][48] støtte i **CQL** da nødvendige datatransformasjoner og aggregeringer ville blitt teknisk utfordrende å utføre når data har blitt fordelt i partisjoner over flere noder i et **Cassandra** cluster.

- **TLC_Cassandra_Queries**

Dette er et rent *Cassandra* dokument og er strukturert rundt konkrete spørringer. Når tabeller er opprettet og data eksportert blir dette dokumentet som illustrerer resultatet. Her kommer spørringer i henhold til oppgaven å finne sted. Dette dokumentet kan kjøres flere ganger uten problem, da det ikke utføres noen skrivende handlinger her.

2.4.1.1 Target queries:

Trips by pickup date (e.g., all trips starting in zone X on day Y).

For denne spørringen ønsker vi å hente ut en telling av alle turer som startet i en spesifikk sone på en spesifikk dag. For å kunne gjøre dette effektivt med **CQL** velger vi å bruke '*pickup_zone*' som primærnøkkel og '*pickup_date*' som clustering nøkkel. Dette gjør at vi kan kjøre spørringer mot den relevante sonen, samt at resultatet vil være sortert etter datering[49].

I dette tilfellet burde det tatt en vurdering på hvordan spørringen skal brukes. Om det foreligger et ønske om fokusere mer på datering fremfor soner, kan det være mer hensiktsmessig å bytte om på primærnøkkel og clustering nøkkel.

Selve grupperingen vil bli utført i **spark** før dataene blir eksportert til **Cassandra**, slik at **CQL** spørringer kun orienteres rundt lesing av allerede aggregerte data.

- Opprett tabell i **CQL** ([TLC_CQL_DB_Build.zpln](#))

```
USE tlc_ks;
CREATE TABLE IF NOT EXISTS trips_by_day_zone(
    pickup_date DATE,
    pickup_zone TEXT,
    number_of_trips INT,
    PRIMARY KEY (pickup_zone, pickup_date)
) WITH CLUSTERING ORDER BY (pickup_date ASC);
```

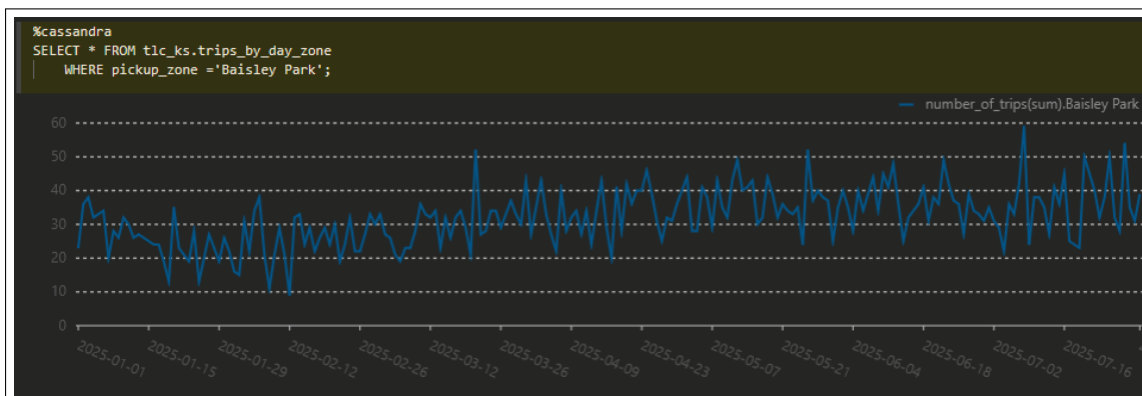
- Forbered data i **spark** ([TLC_Frame_to_Table.zpln](#))

```
val trips_by_day_zone_df = tlc_final
  .select(
    col("pickup_date"),
    col("pickup_location_name").as("pickup_zone")
  )
  .withColumn("pickup_date", to_date(col("pickup_date"), "yyyy-MM-dd"))
  .groupBy("pickup_date", "pickup_zone")
  .agg(count("*").as("number_of_trips"))
  .orderBy("pickup_date", "pickup_zone")

trips_by_day_zone_df.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "trips_by_day_zone",
    "keyspace" -> "tlc_ks"
  ))
  .mode("append")
  .save()
```

- Sluttresultat ved spørring ([TLC_Cassandra_Queries.zpln](#)):

Her illustreres hvordan man kan vise frem statistiske data for en gitt sone og dag. I dette eksempelet hentes antall turer per dag i *Baisley Park*, som videre blir visualisert i en graf.



Figur 2.68: Utklipp fra Zeppelin celle: CQL spørring mot *trips_by_day_zone* tabell

Top drop-off zones for a pickup zone (with counts).

For denne spørringen ønsker vi å hente ut de mest populære *drop-off* sonene for en gitt *pickup* sone med telling på antall mellom de to korrelerende sonene. Siden dette ikke er en dateringsorientert spørring gir det mening å bruke *pickup* sone som partisjonsnøkkel og *trips* (kolonne med **agg** telling)

sone som clustering nøkkel.

Vi bruker *trips* kolonnen som clustering nøkkel for å kunne sortere resultatet etter antall turer i synkende rekkefølge. Dette gjør at vi enkelt kan hente ut de mest populære *drop-off* sonene for en gitt *pickup* sone.

For å kun få ut den mest populære *drop-off* sonen for hver *pickup* sone, må vi implementere en **window** funksjon, slik at vi kan dele opp resultatet i partisjoner basert på *pickup* sone, og deretter rangere radene innen hver partisjon basert på antall turer. Siden vi benytter **rank()** her foreligger det en mulighet for at en *pickup* sone kan komme med i listen flere ganger om det finnes to eller fler top *dropoff* soner med samme antall turer. Det vil være mulig å implementere en strengere løsning ved å benytte **row_number()** istedet, men dette vil da føre til tap av data ved konkurrerende top kandidater.

Her bruker vi **spark** til å preprosessere data før vi eksporterer det til korresponderende Tabell, slik at vi unngår eventuelle utfordringer i henhold til svakheter i **CQL** spørringer.

- Opprett tabell i **CQL** ([TLC_CQL_DB_Build.zpln](#))

```
USE tlc_ks;
CREATE TABLE IF NOT EXISTS top_dropoff_zones_by_pickup_zone(
  pickup_zone TEXT,
  dropoff_zone TEXT,
  trips BIGINT,
  PRIMARY KEY (pickup_zone, trips)
)WITH CLUSTERING ORDER BY (trips DESC);
```

- Forbered data i **spark** ([TLC_Frame_to_Table.zpln](#))

```
val filter_dropof = tlc_final
  .select(
    col("pickup_location_name").as("pickup_zone"),
    col("dropoff_location_name").as("dropoff_zone")
  )
  .groupBy("pickup_zone", "dropoff_zone")
  .agg(
    count("*").as("trips")
  )
  .withColumn(
    "rank",
    rank().over(Window.partitionBy("pickup_zone").orderBy(desc("trips")))
  )
  .where(col("rank") === 1)
  .drop("rank")
  .orderBy(desc("trips"))

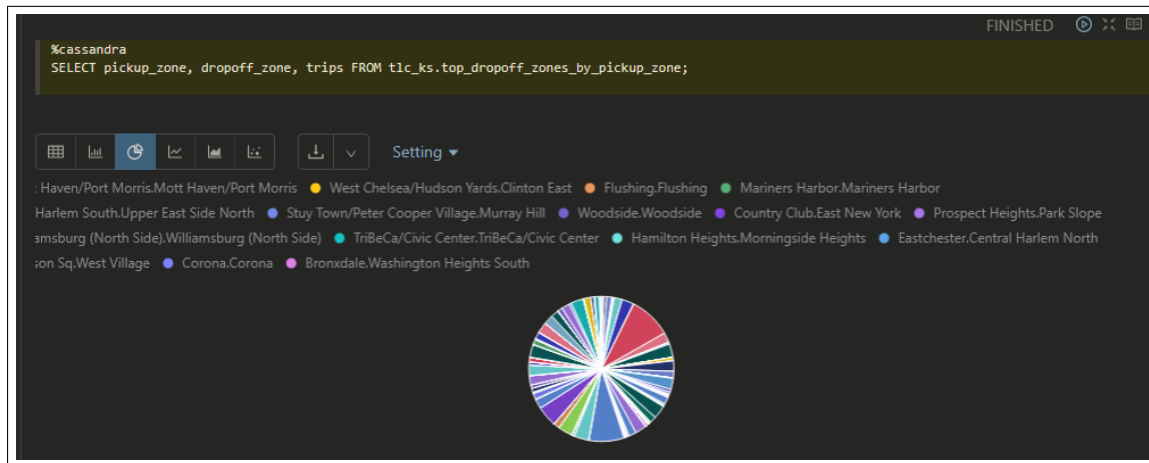
filter_dropof.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
```

```

    "table" -> "top_dropoff_zones_by_pickup_zone",
    "keyspace" -> "tlc_ks"
  ))
  .mode("append")
  .save()

```

- Sluttresultat ved spørring illustrert i et kakediagram.([TLC_Cassandra_Queries.zpln](#)):



Figur 2.69: Utklipp fra Zeppelin celle: CQL spørring mot `top_dropoff_zones_by_pickup_zone` tabell

Daily revenue per borough.

Her ønsker vi å hente ut den totale inntekten per bydel for hver enkel dag. For å kunne gjøre dette effektivt med **CQL** kan vi bruke `'borough'` som primærnøkkel og `'pickup_date'` som clustering nøkkel. Dette gjør at tabellen vil bli partisjonert på bydeler og sortert etter dateringer. Dermed kan vi kjøre spørringer mot bydeler, og resultatet vil være sortert etter datering.[49].

Selve grupperingen og summeringen av inntekter vil bli utført i **spark** før dataene blir eksportert til **Cassandra**, slik at **CQL** spørringer kun orienteres rundt lesing av allerede aggregerte data.

Når det kommer til hvordan vi allokere inntekter til bydeler, har vi valgt å bruke `pickup_borough` som indikator for hvilken bydel inntekten skal allokere til. Argumentet her er at den bydelen hvor turen starter er den bydelen som den gitte taxi bilen opererer i, og dermed er det naturlig å allokere inntekten til denne bydelen.

- Opprett tabell i **CQL** ([TLC_CQL_DB_Build.zpln](#))

```

USE tlc_ks;
CREATE TABLE IF NOT EXISTS daily_revenue_by_borough(
  borough TEXT,
  day DATE,
  revenue FLOAT,
  PRIMARY KEY (borough, day)
)WITH CLUSTERING ORDER BY (day ASC);

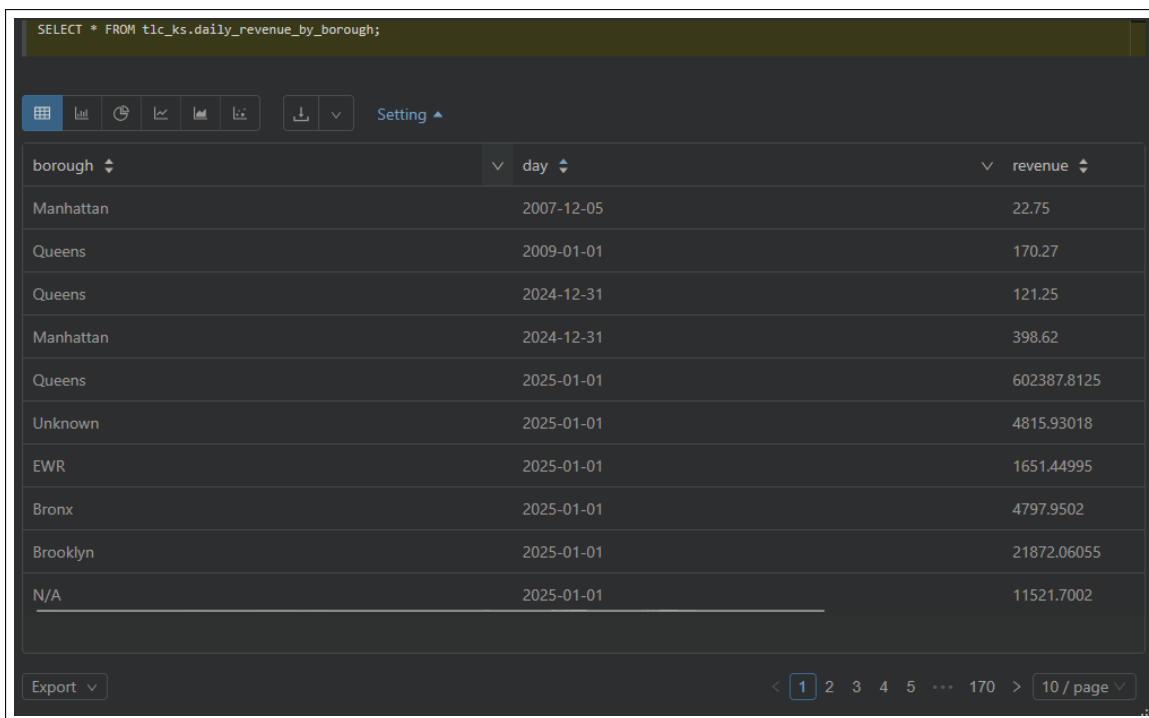
```

- Forbered data i `spark` ([TLC_Frame_to_Table.zpln](#))

```
val daily_revenue_by_borough_df = tlc_final
  .select(
    col("PU_Borough").as("borough"),
    col("pickup_date").as("day"),
    col("total_amount")
  )
  .withColumn("day", to_date(col("day"), "yyyy-MM-dd"))
  .groupBy("day", "borough")
  .agg(sum("total_amount").cast("float").as("revenue"))
  .orderBy("day")

daily_revenue_by_borough_df.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "daily_revenue_by_borough",
    "keyspace" -> "tlc_ks"
  ))
  .mode("append")
  .save()
```

- Sluttresultat ved spørring ([TLC_Cassandra_Queries.zpln](#)):



The screenshot shows a Zeppelin notebook cell with a CQL query and its results. The query is `SELECT * FROM tlc_ks.daily_revenue_by_borough;`. The results are displayed in a table with three columns: `borough`, `day`, and `revenue`. The table contains 11 rows of data, including entries for Manhattan, Queens, Unknown, EWR, Bronx, Brooklyn, and N/A.

borough	day	revenue
Manhattan	2007-12-05	22.75
Queens	2009-01-01	170.27
Queens	2024-12-31	121.25
Manhattan	2024-12-31	398.62
Queens	2025-01-01	602387.8125
Unknown	2025-01-01	4815.93018
EWR	2025-01-01	1651.44995
Bronx	2025-01-01	4797.9502
Brooklyn	2025-01-01	21872.06055
N/A	2025-01-01	11521.7002

Figur 2.70: Utklipp fra Zeppelin celle: CQL spørring mot `daily_revenue_by_borough_result` tabell

2.4.1.2 Activity

Choose partition keys (e.g., (pickup_date, pickup_zone)).

For de fleste naturlige spørringer som oppstår i kontekst av tilhørende data for TLC, vil det være hensiktsmessig å benytte *pickup_date* som partisjonsnøkkel i de alle fleste tabeller. Grunnlaget for dette er at de fleste relevante spørringer for disse typer av data vil være dato orienterte. Dette vil videre lede til effektive spørringer siden rader med samsvarende dateringer vil bli lagret på samme node i clustered og dermed kan systemet enkelt finne den noden som holder på partisjonen med de etterlyste radene.

Videre gir det mening å benytte soner orienterte kolonner som clustering nøkkel. Med denne løsningen vil noder sortere rader etter soner i stigende rekkefølge om ikke annen sortering blir definert med et 'WITH CLUSTERING ORDER BY...' uttrykk.

Det understrekes at ovennevnte valg av nøkkelstruktur kan variere veldig ut i fra hvilke spørringer som er tenkt for en tabell. I enkelte tabeller vil det mulig være mer logisk å bruke soner som partisjonsnøkkel om spørringene skulle være mer orientert rundt lokasjoner fremfor dateringer.

Use clustering columns for sorting by drop-off or fare.

For begge løsninger har vi valgt å bruke *pickup_date* som partisjonsnøkkel. Dette burde gi en god fordeling av dataene i clusteret, samtidig som det gir mening i forhold til naturlige spørringer.

Siden begge tabeller inneholder tilnærmet de samme kolonnene, kan vi lage et enkelt **spark** datasett hvor vi dropper kolonner som ikke er relevante for den gitte tabellen, og deretter eksportere det til de relevante tabellene.

I henhold til tabellenes endelige struktur, så foreligger det en del usikkerhet i forhold til hvilke spørringer ville bli mest brukt i en reell setting. Disse er dermed kun ment som eksempler på hvordan clustering nøkler kan brukes for å sortere data i henhold til spesifikke kolonner. I en reell setting ville det vært behov for et møte med interessenter for å kartlegge hvilke spørringer som ville vært mest relevante for de gitte tabellene, for deretter å strukturere tabellene i henhold til dette.

– **trips_by_day_zone_dropoff** (TLC_CQL_DB_Build.zpln).

I denne tabellen har vi valgt å bruke *pickup_date* som primærnøkkel, med *dropoff_zone* og *trips* som clustering nøkler. Dette gjør at vi kan kjøre spørringer mot *dateringer*, og resultatet vil være sortert etter *dropoff_zone* og antall turer.

Til slutt blir det spesifisert at tabellen skal sorteres i stigende rekkefølge i henhold til *dropoff_zone* kolonnen.

```
USE tlc_ks;
CREATE TABLE IF NOT EXISTS trips_by_day_zone_dropoff (
    pickup_date DATE,
    pickup_zone TEXT,
    dropoff_zone TEXT,
    trips INT,
    PRIMARY KEY (pickup_date, dropoff_zone, trips)
```

```
) WITH CLUSTERING ORDER BY (dropoff_zone ASC);
```

– **trips_by_day_zone_fare** ([TLC_CQL_DB_Build.zpln](#)).

For denne tabellen brukes *fare_amount* som clustering nøkler. Deretter blir det spesifisert at tabellen skal sorteres i synkende rekkefølge i henhold til *fare_amount* kolonnen. Dette gjør at vi enkelt kan hente ut de høyest betalende turene for en gitt periode.

Her ville det vært mulig å endre på tabell struktur ved å lage en sammensatt partisjonsnøkkel med både *pickup_date* og *pickup_zone*, slik at man kunne spesifisert både dato og sone i spørringer.

```
USE tlc_ks;
CREATE TABLE IF NOT EXISTS trips_by_day_zone_fare (
    pickup_date DATE,
    pickup_zone TEXT,
    fare_amount FLOAT,
    PRIMARY KEY (pickup_date, fare_amount)
) WITH CLUSTERING ORDER BY (fare_amount DESC);
```

– Forbered data i **spark** ([TLC_Frame_to_Table.zpln](#)).

```
%spark
val dropoff_and_fare_df = tlc_final
    .select(
        col("pickup_date"),
        col("pickup_location_name").as("pickup_zone"),
        col("dropoff_location_name").as("dropoff_zone"),
        col("total_amount").as("fare_amount")
    )
    .withColumn("pickup_date", to_date(col("pickup_date"), "yyyy-MM-dd"))
    .withColumn("fare_amount", col("fare_amount").cast("float"))

val dropoff_df = dropoff_and_fare_df
    .select("pickup_date", "pickup_zone", "dropoff_zone")
    .groupBy("dropoff_zone", "pickup_date", "pickup_zone")
    .agg(count("*").cast("Integer").as("trips"))

val fare_df = dropoff_and_fare_df.drop(col("dropoff_zone"))

dropoff_df.write
    .format("org.apache.spark.sql.cassandra")
    .options(Map(
        "table" -> "trips_by_day_zone_dropoff",
        "keyspace" -> "tlc_ks"
    ))
    .mode("append")
    .save()
```


Create trips_by_day_zone, daily_revenue_by_borough, etc.

For *Create trips_by_day_zone* så har denne oppgaven allerede blitt løst i seksjon [2.4.1.1](#).

For *daily_revenue_by_borough* så har denne oppgaven allerede blitt løst i seksjon [2.4.1.1](#).

2.4.2 Modeling Movielens in Cassandra**2.4.2.1 Target Queries:****Ratings given by a user (sorte by timestamps)**

Siden vi for denne spørringen ønsker å hente vurderinger gjort av en bruker sortert etter timestamps så må vi bruke `userId` som primærnøkkel og `rating_timestamp` som clustering nøkkel, dette gjøre at vi kan kjøre spørringer både på `userId` og `rating_timestamp`, samt at tabellen vi sorteres på `rating_timestamps` kolonnen.^[49]

```
CREATE TABLE user_ratings(  
    userId TEXT,  
    rating_timestamp TIMESTAMP,  
    movieId TEXT,  
    rating TEXT,  
    PRIMARY KEY (userId, rating_timestamp));
```

Average rating per movie

Hvis vi for denne spørringen kun ønsker å hente ut `avg_rating` ved å søke på en spesifikk `movieId` så vil det holde å bruke `movieId` som primærnøkkel, men hvis også ønsker å kunne sortere eller filtrer på `avg_rating` må vi også inkludere `avg_rating` som clustering nøkkel. Vi har satt opp et forslag med kun `movieId` som primærnøkkel.^[49]

```
CREATE TABLE avg_movie_rating(  
    movieId TEXT,  
    avg_rating TEXT,  
    rating_count TEXT,  
    PRIMARY KEY (movieId));
```

Top movies in a genre (by average rating, min N ratings)

For å løse denne spørringen må vi bruke `genre` som primærnøkkel og både `avg_rating` og `rating_count` som clustering nøkler. Dette gjør at vi kan kjøre spørringer mot alle disse kolonene og dermed også

sette en begrensning på minste verdi vi ønsker at rating_count kan ha når vi kjører en spørring mot tabellen. For denne tabellen vil det også være fornuftig å tilpasse sorterings rekkefølge på resultatet slik at denne har synkende verdier for avg_rating. Dette kan gjøres ved å legge til en WITH klausul på create table koden.^[49]

```
CREATE TABLE top_rating_in_genre(
    genre TEXT,
    movieId TEXT,
    avg_rating TEXT,
    rating_count TEXT,
    PRIMARY KEY (genre, avg_rating, rating_count))
WITH CLUSTERING ORDER BY (avg_rating DESC, rating_count DESC);
```

2.4.2.2 Schema Design Exercise

ratings_by_user(userId, rating_timestamp, movieId, rating)

For dette schema har vi valgt å bruke IntegerType for userId og movieId siden dette kun er id verdier som ikke skal brukes til andre operasjoner. Vi har også valgt å bruke nullable=false på userId siden dette er iden for radene for dette schema, samt at denne verdien ikke skal kunne være 0. For rating_timestamp har vi valgt å bruke TimestampType da dette vil bevare mest mulig informasjon direkte i kolonne verdiene, samt tillate direkte operasjoner for timestamps på kolonnen. For rating har vi valgt å bruke DoubleType da dette tillater desimal verdier og flere muligheter videre.

```
val ratings_by_user_schema = new StructType()
    .add("userId", IntegerType, nullable=false)
    .add("rating_timestamp", TimestampType, nullable=true)
    .add("movieId", IntegerType, nullable=true)
    .add("rating", DoubleType, nullable=true)
```

Figur 2.73: Utklipp fra Zeppelin for schema for ratings_by_user.

ratings_by_movie(movieId, userId, rating) (to support aggregates via Spark later)

På dette schema har vi også valgt å bruke IntegerType for movieId og userId da det vi dekke behovene for informasjon i disse kolonnen. For dette schema har vi satt nullable=false for movieId siden denne verdien ikke skal kunne være 0, samt at det er iden for schemaet. For rating har vi valgt å bruke DoubleType, dette viktig for denne kolonnen siden vi skal kunne støtte aggregering på et senere tidspunkt, for å kunne bruke aggregering på kolonner i spark så må kolonnen inneholde en numerisk type, i tillegg så vil en aggregering automatisk lagre resultat som samme type som kolonnen i utgangspunktet hadde. Derfor må vi velge en type som tillater desimal verdier for eks beregning av gjennomsnitt.^[50]

```
val rating_by_movie_schema = new StructType()  
  .add("movieId", IntegerType, nullable=false)  
  .add("userId", IntegerType, nullable=true)  
  .add("rating", DoubleType, nullable=true)
```

Figur 2.74: Utklipp fra Zeppelin for schema for ratings_by_movie.

top_movies_by_genre(genre, rating_avg, ...)

For dette schema har vi valgt å bruke StringType for genre da dette er tekst verdi for en enkelt genre. For movieId har vi valgt å bruke IntegerType på samme grunnlagt som tidligere. For avg_rating bruker vi også som i de tidligere schemaene DoubleType. For ratings_count har vi valgt å bruke IntegerType da dette skal være tilstrekkelig for den forventede datamengden.

```
val top_movies_by_genre_schema = new StructType()  
  .add("genre", StringType, nullable=true)  
  .add("movieId", IntegerType, nullable=true)  
  .add("avg_rating", DoubleType, nullable=true)  
  .add("rating_count", IntegerType, nullable=true)
```

Figur 2.75: Utklipp fra Zeppelin for schema for top_movies_by_genre.

2.4.3 Spark-Cassandra Connector

2.4.3.1 Load a Subset of TLC Trips into Cassandra

2.4.3.2 Run a Spark Job That:

Alle punkter under er løst og illustrert gjennom seksjon 2.4 i denne rapporten.

- Queries trips_by_day_zone for one borough.
- Aggregates results.
- Writes back to Cassandra in a reporting table.

Capstone Prosjekt Beskrivelse

3.1 Intro

I dette prosjektet skal det produseres en søkbar database med værdata, generelle trafikkdata, sykkeltrafikkdata og hvis mulig kollektivdata. Databasen er tenkt å kunne benyttes til å undersøke sammenhenger og avhengigheter mellom værdata og sykkeltrafikk i et avgrenset område med ambisjoner om å utvikle en modell som kan dekke flere byer og steder uten å måtte gjøre store endringer på databasen.

Datasettene som er valgt er omfattende værdata fra metrologisk institutt, trafikkdata fra statens vegvesen og sykkeltrafikkdata fra statens vegvesen. Disse datasettene er valgt de er enkelt skalerbare, offentlig tilgjengelige og er publisert av troverdige kilder. Vi kan selv velge hvilke tidsrom vi ønsker data for og kan relativt enkelt importere mer data om nødvendig.

I løpet av prosjektet skal det opprettes en database, relevant data fra datasettene skal importeres, sorteres og anvendes til videre analyser. Det skal blant annet undersøkes om det er en sammenheng mellom værdata og sykkelvaner.

På sikt kan databasen potensielt brukes til å se langtidseffekter av politiske vedtak og endringer ment til å påvirke sykkel og trafikkvaner. Vi kan også se etter endringer i trafikkmønstre og sykkelmønstre, både ved megde og tidspunkter. Denne databasen skal også relativt enkelt kunne utvides i geografisk omfang til å dekke flere byer og land.

3.2 Datasettene

3.2.1 Trafikkdata

Navn	Verdi	Forklaring
Trafikkregistreringspunkt	string	Id nummer på målestasjonen trafikkdataene er registrert.
Navn	string	Navnet på trafikkregistreringspunktet.
Vegreferanse	string	Hvilken vei stasjonen ligger ved. Gitt i statens vegvesens veinummerering.
Fra	string	Dato og tidspunkt for målingens start.
Til	string	Dato og tidspunkt for målingens slutt.
Dato	date	Dato for målingen.
Fra tidspunkt	timestamp	Tidspunkt for måleperiodens start.
Til tidspunkt	timestamp	Tidspunkt for måleperiodens slutt.

Felt	string	Hvilke kjørefelt målingen gjelder.
Trafikkmengde	string	Antall passeringer i måleperioden.
Dekningsgrad	string	Forklarer usikkerhet ved å angi andel av tidsintervall registreringen har data for, gitt i prosent.
Antall timer total	integer	Totalt antall timer for måleperioden.
Antall timer inkludert	integer	Totalt antall timer som har data i måleperioden.
Antall timer ugyldig	integer	Antall timer i perioden med ukjent kvalitet.
Ikke gyldig lengde	string	Antall kjøretøy med underkjent/ukjent lengde. Eks kjøretøy over 27,0 meter.
Lengdekvalitetsgrad	string	Andel kjøretøy med godkjent lengdemåling, gitt i prosent.
< 5,6m	string	Antall kjøretøy kortere enn 5,6 meter.
≥ 5,6m	string	Antall kjøretøy lengre enn eller lik 5,6 meter.
5,6m – 7,6m	string	Antall kjøretøy lenger enn eller lik 5,6 meter og kortere enn 7,6 meter.
7,6m – 12,5m	string	Antall kjøretøy lenger enn eller lik 7,6 meter og kortere enn 12,5 meter.
12,5m – 16,0m	string	Antall kjøretøy lenger enn eller lik 12,5 meter og kortere enn 16,0 meter.
≥ 16,0m	string	Antall kjøretøy lengre enn eller lik 16,0 meter.
16,0m – 24,0m	string	Antall kjøretøy lengre enn eller lik 16,0 meter og kortere enn 24,0 meter.
≥ 24,0m	string	Kjøretøy lengre enn eller lik 24,0 meter. Øvre grense for gyldig lengdemåling er på 27,0 meter.

3.2.2 Om Trafikkdataene

Alle trafikkdataen blir lest inn i dataframes for sykkel og bil, for å senere samles i en felles dataframe.

Navn	Bil	Sykkel	samlet DF Trafikk
Count	1247438	1071185	2189107
Mean Trafikkmengde	405.9	2.5	233.2
StDev Trafikkmengde	442.7	4.4	389.8
Min Trafikkmengde	0	0	0
Maks Trafikkmengde	2899	129	2899
Dekningsgrad (%)	99,9	94,4	99,9

Describe funksjonen i spark retunerer standardtall for alle kolonnene i datasettet, dette er antall innføringer (Count), gjennomsnitt (Mean), standardavvik (StDev), minimumsverdi (Min) og maksimumsverdi (Max).

Særlig relevant for analysen er standardtall for trafikkmengde da dette er kjernedataene i undersøkelsen. Standardtallene for verdier som kjøretøylengder er ikke inkludert da dette bare er kategorier for sortering av kjøretøy, så disse er ikke relevante for analysen. Det samme gjelder deskriptive kolonner som for eksempel Felt, som i noen innføringer angir kjørefelt nummer og i andre angir at innføringen gjelder totalen for en kjøreretning som tekst.

Lengdekvalitetsgrad sier noe om effektiviteten av lengdemålingen på kjøretøy, noe som ikke er relevant for analysen. Minimum dekningsgrad er som regel 0 og er derfor utelatt. Siden det er forskjellige datasett for sykkel og bil brukes ikke de ulike lengdemålene på kjøretøy. Noen av kolonnene vil alltid ha 1 som verdi, da dette er timer totalt for måleperioden, disse er også utelatt.

Trafikkdatasettene er hentet fra statens vegvesen. Vi har her tatt med flere målestasjoner der både sykkeldata og trafikkdata er registrert. Dataene er fra målestasjonene ved de store innfartsårene til Fredrikstad. Disse er fra perioden 01.01.2020 til 01.01.2025. De er timesmålinger for hver kjøreretning og totalt antall passeringer per time for begge retninger. Dataene er i formatet csv. For trafikkdataene var det ingen løsning for filtrering av data før nedlastning, og datasettet inkluderer derfor alle tilgjengelige variabler, også data som ikke er relevante for denne analysen. Det er hovedsakelig passeringsdata og mål på usikkerhet som er relevante for dette prosjektet. Ved videre analyse og implementasjon kan de forskjellige lengdene også utnyttes.

3.2.3 Værdata

Navn	Verdi	Forklaring
Navn	string	Navnet på målestasjon.
Stasjon	string	Stasjons ID.
Tid (norsk normaltid)	string	Registreringstidspunkt for måleperioden. Hver måleperiode er på én time.
Nedbør (1 t)	string	Nedbør siste time, gitt i millimeter.
Lufttemperatur	string	Temperatur på luft i måletidspunktet.
Antall minutter nedbør siste time	string	Antall minutter det er registrert nedbør den siste timen.
Høyeste middelvind (1 t)	string	Høyeste gjennomsnitt av vindhastighet over en ti-minutters periode den siste timen. Høyeste gjennomsnitt av seks stykk ti-minutters perioder.

3.2.4 Om Værdataene

Værdataene kommer fra metrologisk institutt og dekker et område i Fredrikstad. Datasettet er foreløpig bare fra Strømtangen fyr, da det er det eneste målepunktet i Fredrikstad med offentlig tilgjengelig data i aktuelle perioder. Dataene er timesmålinger fra 01.01.2005 til 01.01.2025(siste 20 år) og er på formatet csv.

Relevante standardtall for værdata er antall innføringer, gjennomsnitt, standardavvik, min og maks for nedbør, lufttemperatur, antall minutter nedbør siste time og høyeste middelvind. Tidsintervaller er alltid en time og standardtallene er derfor ikke relevante. Andre variabler gir ikke standardtall.

Det er totalt 177065 innføringer med værdata. Gjennomsnittsnedbøren de siste 20 årene var 0.007 mm med standardavvik på 0.16. Høyeste registrerte nedbørsmengde siste time er på 9,9 mm. Lufttemperaturen hadde et gjennomsnitt på 8.4 grader med standardavvik på 7,4, minstemperatur på -0,1 og makstemperatur på 9,9 grader. Antall minutter nedbør siste time var gjennomsnittlig på 6,2 minutter med standardavvik på 14,6 og maksverdi på 9 minutter. Høyeste middelvind i perioden var gjennomsnittlig 6,9 meter per sekund med standardavvik på 3,5. Minste registrerte var på 0,7 m/s og høyeste på 9,9 m/s.

Formålet med værdataene er se etter en korrelasjon mellom været og trafikkvaner særlig med tanke på hvilke vær fenomener som påvirker folk sine sykkelvaner. Av den grunn er det valg værvariabler som indikerer "dårlig vær" som nedbør, temperatur og vind. Solforhold er ikke tatt med da vi ikke kunne finne datasett som inkluderte dette. Som mål på vind ble høyeste middelvind valgt da dette indikerer høyeste vindmengde en potensiell syklist vil utsettes for i perioden. Ved senere analyse kan det se ut som værdata ikke inneholder tilfredstillende data om nedbør og antall minutter nedbør siste time.

3.3 Konkluderende om datasettene

Basert på de innleste dataene ser vi at det foreligger et behov for å sette opp schema for riktig innlesning av datatyper.

Utforsking og Validering av Prosjektemnet

4.1 Innledning

Hovedmålet våres for dette prosjektet er å lage en søk-bar database bestående av flere **CQL** tabeller for å kunne hente ut og analysere informasjon knyttet til generell trafikkdata, data for sykkeltrafikk og værdata for Fredrikstad. Som første ledd av dette ønsker vi i denne delen av prosjektet å hente inn trafikkdata for 4 hovedveier inn og ut av Fredrikstad sentrum.

For en mer utfyllende beskrivelse av de undenevnte datasettene, se [seksjon 3.2](#) i dette dokumentet.

4.1.1 Trafikkdata

Vi valgte å bruke følgende trafikk registrerings punkter.

I) Data for bil trafikk:

- Glemmen (Torsbekkveien).
- Rakkestadsvingen Øst (Haldenveien).
- Simo (Mosseveien).
- St.Hansfjellet (Riksvei 110, med registreringspunkt i Fredrikstad sentrum).

II) Data for sykkel trafikk:

- Rakkestadsvingen Øst (Haldenveien).
- Simo (Mosseveien).
- St. Hansfjellet (Riksvei Registreringspunkt i Fredrikstad sentrum ved siden av Riksvei 110).
- Wilbergjordet (Rolvøyveien).

I utgangspunktet ønsket vi å se på en tidsperiode 10 år tilbake i tid, men Vi møtte på problemer i forhold til tilgjengelig data for hele denne perioden. På grunnlag av dette endte vi opp med å bruke data fra perioden **2020 – 2025** siden dette var en periode alle punktene hadde registrert data for. I første omgang ønsker vi å analysere forskjeller i trafikkmønster og trafikkmengde for de forskjellige ukedagene og registrerings punktene. Vi kommer til å undersøke når de forskjellige ukedagene og trafikkpunktene har sine topper for trafikkmengde iløpet av en døgn periode, samt å undersøke forskjeller i den totale trafikkmengden. For å hente inn trafikkdata for disse registrerings punktene har vi bruk statens vegvesen sine nettsider for trafikkdata [\[51\]](#)[\[52\]](#)[\[53\]](#).

4.1.2 Værdata

Disse datasettene inneholder værdata for Fredrikstad området.

Intensjonen med å hente inn værdata er å kunne analysere om det finnes noen korrelasjon mellom

værforhold og trafikkmengde for både bil og sykkel trafikk.

Grunnet begrenset tilgang til værstasjoner er alle våre data hentet og basert på *Strømtangen Fyr*. Her har det blitt innhentet data for perioden 2005 – 2025 fra *Norsk Meteorologisk Institutt* sine nettsider.

4.1.3 Hypoteser

1. Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 og 16 : 00.
2. Den totale trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.
3. Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 og 14 : 00.
4. Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 og 14 : 00.
5. Data kan benyttes til å forutsi om det er blitt utført veiarbeid eller vedlikehold på et gitt trafikk registreringspunkt basert på uregelmessigheter i trafikkmålinger.

4.2 Importering og Klargjøring av Data

I denne seksjonen vil vi gå gjennom en detaljert beskrivelse av hvordan vi har importert og klargjort dataen vi har hentet inn for å kunne bruke den i vår database. Dette vil bli illustrert gjennom en *Zeppelin* notatbok som vi har laget for dette formålet ([CapstoneData_FrameHandling.zpln](#)). Det vil bli laget en kompakt versjon av denne notatboken som vil bli brukt til opprettelse og Cassandra tabeller i senere kapitler.

4.2.1 Overordnet Konfigurering av Zeppelin Notatbok

Siden all data vi har hentet inn er lokalisert innenfor samme tidssone, har vi valgt å sette tidssonen for hele notatboken til **Europe/Oslo**. Dette gjør vi for å unngå problemer med tidssoner når vi skal analysere dataen. Dette gjøres ved å sette følgende konfigurasjon i begynnelsen av notatboken:

```
spark.conf.set("spark.sql.session.timeZone", "Europe/Oslo")
```

4.2.2 Data Kilder

Alle *dataframes* som blir laget i notatboken blir bygget rekursivt på data som blir hentet inn fra separate datafiler. Derfor har vi definert variabler for filstier til alle datafilene vi har hentet inn, hvor hver filsti peker til alle datafiler for et spesifikt område. Dette gjør det enkelt å hente inn alle datafiler for et tema (bil, sykkel eller vær).

```
val bil_base = "/sparkio/trafikkdata/bil/*.csv"
val sykkel_base = "/sparkio/trafikkdata/sykkel/*.csv"
val weather_base = "/sparkio/værdata/*.csv"
```

4.2.3 Schema – Definerer av datastruktur

På grunnlag av at alle våre data er lagret i **CSV** format, har det vært nødvendig å definere et *schema* for datasettene vi ønsker å benytte. Siden sykkel og bil trafikk datasettene har samme struktur, har vi valgt å definere ett felles *schema* for disse. Værdataene har en annen struktur, og har derfor fått sitt eget *schema*.

Ved opprettelse av *schema* ble følgende felles avgjørelser tatt:

- Alle kolonnenavn er definert med små bokstaver for å unngå problemer med store og små bokstaver ved overføring til *Cassandra* tabeller.
- Kolonner som inneholder numeriske data med desimaler blir definert som **StringType** for å unngå tap av data.
Dette er grunnet at Norge bruker komma som desimal skiller, mens **Cassandra** og **Spark** forventer punktum som desimal skiller. Her foreligger det altså et behov for å gjøre en erstatning av komma til punktum før dataen kan overføres til **Cassandra**.
- Kolonner som inneholder numeriske heltallsverdier blir vurdert individuelt på verdienes rekkevidde. Basert på dette vil den omhandlede kolonne bli definert med en passende datatype. Dette blir gjort for å unngå unødvendig stort lagringsbehov.

4.2.3.1 Trafikk Schema

Under er en mer detaljert oversikt over *schema* for bil og sykkel trafikk datasettene. Denne seksjonen tar kun for seg hvilke datatyper de forskjellige kolonnene har blitt definert med, og en kort forklaring på hvorfor. For mer utfyllende informasjon om kolonnenes innhold, se [seksjon 3.2](#).

Kolonnenavn	Datatype	Begrunnelse
trafikkregistreringspunkt	StringType	Blanding av tall og bokstaver. Passer som <i>string</i> .
navn	StringType	Beskrivende tekst av lokasjon. Passer som <i>string</i> .
vegreferanse	StringType	Blanding av tall og bokstaver. Passer som <i>string</i> .
fra	TimestampType	ISO-8601[54] format satt til norsk tid. Passer som <i>timestamp</i>

Fortsetter på neste side

Kolonnenavn	Datatype	Begrunnelse
til	TimestampType	ISO-8601[54] format satt til norsk tid. Passer som <i>timestamp</i>
dato	DateType	Datering med format ' <i>dd-MM-yyyy</i> '. Passer som <i>date</i> .
fra tidspunkt	StringType	Klokkeslett med format ' <i>HH:MM</i> '. Defineres som <i>string</i> for å unngå datatap.
til tidspunkt	StringType	Klokkeslett med format ' <i>HH:MM</i> '. Defineres som <i>string</i> for å unngå datatap.
felt	StringType	Varierer mellom numeriske og string verdier i celler. Defineres som <i>string</i> for å unngå datatap.
trafikkmengde	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
dekningsgrad (%)	StringType	Tallverdi med komma som desimal. Defineres som <i>string</i> for å unngå datatap.
antall timer total	ByteType	Heltallsverdier med små verdier. Passer som <i>byte</i> (range: $-128 \rightarrow 127$)
antall timer inkludert	ByteType	Heltallsverdier med små verdier. Passer som <i>byte</i> (range: $-128 \rightarrow 127$)
antall timer ugyldig	ByteType	Heltallsverdier med små verdier. Passer som <i>byte</i> (range: $-128 \rightarrow 127$)
ikke gyldig lengde	ByteType	Heltallsverdier med små verdier. Passer som <i>byte</i> (range: $-128 \rightarrow 127$)
lengdekvalitetsgrad (%)	StringType	Tallverdi med komma som desimal. Defineres som <i>string</i> for å unngå datatap.
< 5,6m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
>= 5,6m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
5,6m - 7,6m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
7,6m - 12,5m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
12,5m - 16,0m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)

Fortsetter på neste side

Kolonnenavn	Datatype	Begrunnelse
>= 16,0m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
16,0m - 24,0m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)
>= 24,0m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: $-32768 \rightarrow 32767$)

Tabell 4.1: Definering av *schema* for bil og sykkel trafikk datasettene.

4.2.3.2 Vær Schema

Under er en mer detaljert oversikt over *schema* for værddata. Denne seksjonen tar kun for seg hvilke datatyper de forskjellige kolonnene har blitt definert med, og en kort forklaring på hvorfor. For mer utfyllende informasjon om kolonnenes innhold, se [seksjon 3.2](#).

Kolonnenavn	Datatype	Begrunnelse
navn	StringType	Beskrivende tekst av lokasjon. Passer som <i>string</i> .
stasjon	StringType	Blanding av tall og bokstaver. Passer som <i>string</i> .
tid(norsk normalt tid)	TimestampType	NS-ISO 8601[55], som er tidsformat etter norsk standard. Passer til <i>timestamp</i>
nedbør (1 t)	StringType	Tallverdi med komma som desimal. Defineres som <i>string</i> for å unngå datatap.
lufttemperatur	StringType	Tallverdi med komma som desimal. Defineres som <i>string</i> for å unngå datatap.
høyeste middelvind (1 t)	StringType	Tallverdi med komma som desimal. Defineres som <i>string</i> for å unngå datatap.

Tabell 4.2: Definering av *schema* for Værddata.

4.2.4 Import av Data

Med definerte *schema* på plass har vi nå mulighet til å importere dataene inn i *dataframes*. Under denne prosessen er det verdt å nevne at det gjort noen oppdagelser i datsetene som har blitt håndtert under importeringen:

- Flere steder er brukt som **NULL** verdi i datasettene. For å oppnå konsistens i dataene har vi valgt å erstatte alle forekomster hvor en celle kunn inneholder "med **NULL** verdier under importeringen.
- Det er brukt forskjellige daterings formater i trafikk og vær datasettene. Derfor har vi definert et tilpasset dato format for hver av disse under importeringen for å sikre korrekt tolkning av datoverdiene.
- Vi ønsket å lage en kombinert trafikk *dataframe* som inneholder både bil og sykkel trafikk data, men med mulighet til å skille mellom disse. For å oppnå dette har vi importert bil og sykkel trafikk dataene i separate *dataframes*, for deretter å legge til en ekstra kolonne i hver av disse som indikerer om det er bil eller sykkel.

0 – Bil.

1 – Sykkel.

Her vurderte vi å bruke en **BooleanType** kolonne, som hadde vært tilstrekkelig for å skille mellom bil og sykkel. Vi valgte likevel å bruke en **ByteType** kolonne for å åpne opp for muligheten til å legge til flere klassifiseringer i fremtiden.

Potesielle kandidater for dette kan være:

- Fotgjenger.
- Differensiering mellom ulike typer kjøretøy (lastebil, motorsykkel, buss, etc.).

```
%spark
val bil_raw_df = spark.read
  .option("header", true)
  .option("recursiveFileLookup", true)
  .schema(trafikk_Schema)
  .option("delimiter", ";")
  .option("nullValue", "-")
  .option("timestampFormat", "yyyy-MM-dd'T'HH:mmXXX")
  .csv(bil_base)

val sykkel_raw_df = spark.read
  .option("header", true)
  .option("recursiveFileLookup", true)
  .schema(trafikk_Schema)
  .option("delimiter", ";")
  .option("nullValue", "-")
  .option("timestampFormat", "yyyy-MM-dd'T'HH:mmXXX")
  .csv(sykkel_base)

val weather_raw_df = spark.read
  .option("header", true)
  .option("recursiveFileLookup", true)
  .schema(weather_Schema)
  .option("delimiter", ";")
  .option("nullValue", "-")
  .option("timestampFormat", "dd.MM.yyyy HH:mm")
  .csv(weather_base)

val bil_raw = bil_raw_df.withColumn("klasse", lit(0).cast(ByteType))
val sykkel_raw = sykkel_raw_df.withColumn("klasse", lit(1).cast(ByteType))
```

Figur 4.1: Utlipp fra [CapstoneData_FrameHandling.zpln](#) som viser importering av bil og sykkel trafikk data.

4.2.5 Fjerning av Duplikater

Under importeringen av de forskjellige datasettene har vi oppdaget at det finnes flere duplikate rader. For å sikre datakvalitet har vi derfor valgt å fjerne disse duplikatene ved hjelp av `dropDuplicates` metoden[56] i `Spark`.

4.2.6 Sammensetting av Trafikk Dataframes

Nå som vi har rådata for både bil og sykkel trafikk importert i egne *dataframes*, kan vi gå videre til å sette sammen disse til en kombinert trafikk *dataframe*. Dette gjøres ved å bruke `union` metoden[57] i `Spark`, som kombinerer rader fra begge *dataframes* inn i en ny *dataframe*.

```
val trafikk_raw_df = bil_raw.union(sykkel_raw)
```

4.2.7 Behandling av Importert Data

Før vi fullfører sammensettingen av trafikk og værdata, er det nødvendig å gjøre noen behandlinger av den importerte dataen for å sikre at kolonner har riktig format og innhold.

Her ønsker vi å behandle kolonner som tidligere har blitt definert som **StringType** for å unngå datatap.

Vi har benyttet metodene `withColumn` [33] og `regexp_replace` [58] for å utføre dette.

Følgende endringer blir gjort:

Trafikk data:

- Kolonner med numeriske desimalverdier blir behandlet ved å erstatte komma med punktum, for deretter å konvertere disse til **FloatType**. Vi har valgt **FloatType** fremfor **DoubleType** for å redusere lagringsbehovet, siden vi ikke trenger ekstremt høy presisjon i disse verdiene.
- For kolonner som inneholder klokkeslett har vi valgt å behandle disse ved å hente ut timen som en **ByteType** verdi fra korresponderende kolonner (eks: kolonnen *Fra* korresponderer med kolonnen *fra tidspunkt*). På denne måten kan vi overskrive de originale kolonnene med klokkeslett med en enklere representasjon som fortsatt bevarer nødvendig informasjon for våre analyser og åpner for bedre filtreringsløsninger.

Vær data:

- Kolonner med numeriske desimalverdier blir behandlet ved å erstatte komma med punktum, for deretter å konvertere disse til **FloatType**. Vi har valgt **FloatType** fremfor **DoubleType** for å redusere lagringsbehovet, siden vi ikke trenger ekstremt høy presisjon i disse verdiene.

```
%spark
val trafikk_df_formatFix = trafikk_raw_df
  .withColumn(
    "dekningsgrad (%)",
    regexp_replace(col("dekningsgrad (%)" ), ",", ".").cast(FloatType)
  )
  .withColumn(
    "lengdekvalitetsgrad (%)",
    regexp_replace(col("lengdekvalitetsgrad (%)" ), ",", ".").cast(FloatType)
  )
  .withColumn(
    "fra tidspunkt",
    date_format(col("fra"), "k").cast("byte")
  )
  .withColumn(
    "til tidspunkt",
    date_format(col("til"), "k").cast("byte")
  )

val weather_df_formatfix = weather_raw_df
  .withColumn(
    "nedbør (1 t)",
    regexp_replace(col("nedbør (1 t)" ), ",", ".").cast(FloatType)
  )
  .withColumn(
    "lufttemperatur",
    regexp_replace(col("lufttemperatur"), ",", ".").cast(FloatType)
  )
  .withColumn(
    "høyeste middelvind (1 t)",
    regexp_replace(col("høyeste middelvind (1 t)" ), ",", ".").cast(FloatType)
  )
```

Figur 4.2: Utlipp fra [CapstoneData_FrameHandling.zpln](#) som viser behandling av kolonner i trafikk og vær data.

4.2.8 Sammensetting av Trafikk og Værdata

Neste steg nå er å kombinere trafikk dataene med værdataene, slik at vi til slutt sitter med en *dataframe* som videre kan brukes til å bygge en *Cassandra* database med.

Dette har blitt utført med en `join` operasjon hvor vi kjører en sammeligning av datoer per rad og deretter legger inn værdata i korresponderende trafikk rader.

I [seksjon 3.2](#) har vi konkludert at værdata inneholder upålitelige data. På grunnlag av dette har vi kun inkludert kolonene 'lufttemperatur' og 'høyeste middelvind (1 t)' i den avsluttende dataframen. Resultatet er en *dataframe* som inneholder trafikk og værdata for både biler og sykkel, med mulighet til å filtrere på alle klassifiseringer etter behov.

```
%spark
val complete_df = trafikk_df_formatFix.alias("c")
    .join(weather_df_formatfix.alias("w"),
        col("c.fra") === col("w.tid(norsk normaltid)"), "left")
    .select(
        col("c.*"),
        col("w.lufttemperatur"),
        col("w.høyeste middelvind (1 t)").as("høyeste middelvind (m/s)")
    )
```

Figur 4.3: Utklipp fra [CapstoneData_FrameHandeling.zpln](#) som viser sammensetting av trafikk og vær data.

4.2.9 Vurdering av Upassende eller Ukorrekt Data

I løpet av analytisk arbeid utført i [seksjon 3.2](#), samt oppdagelser gjort under importering og klargjøring av data i [seksjon 4.2](#), har det blitt identifisert avvikende data som videre har blitt tatt opp til vurdering på om det skal fjernes eller beholdes i datasettene.

Trafikkdata

- Dekningsgrad (%)

Denne kolonnen er ment å indikere påliteligheten av trafikkdataen for en gitt tidsperiode. Ved gjennomgang av dataene det blitt oppdaget flere tilfeller hvor dekningsgraden er 0%, men likevel er det registrert en trafikkmengde større enn 0. Dette indikerer at dataen ikke er pålitelig, og vi har derfor valgt å fjerne alle rader hvor dekningsgraden er 0% og det finnes verdier i *trafikkmengde* kolonnen.

Vi har valgt å beholde rader hvor dekningsgraden er 0% og trafikkmengden er 0, siden dette kan potensielt indikere perioder hvor registreringspunktet har vært inaktivt på grunn feil eller vedlikehold.

```
%spark
val validation = complete_df_raw
    .filter( ! (col("dekningsgrad (%)") === 0 && col("trafikkmengde").isNotNull))
```

Figur 4.4: Utklipp fra [CapstoneData_FrameHandeling.zpln](#) som viser filtrering av rader basert på dekningsgrad.

- Lengdekvalitetsgrad (%)

Denne kolonnen er ment å indikere påliteligheten av lengde klassifiseringen av kjøretøy for en gitt tidsperiode.

Ved gjennomgang av dataene det blitt oppdaget flere tilfeller hvor lengdekvalitetsgraden er 0%, men likevel er det registrert en trafikkmengde større enn 0 i en eller flere lengde

klassifiserings kolonner. Dette indikerer at dataen ikke er pålitelig, og vi har derfor valgt å fjerne alle rader hvor lengde kvalitetsgraden er 0% og det finnes verdier i noen av lengde klassifiserings kolonnene.

I denne filtreringen har vi satt fokuset på rader som er klassisert som bil trafikk, siden lengde klassifiseringene for sykkel trafikk ikke er relevante.

```
%spark
val complete_df = validation
  .filter( ! (col("Lengdekvalitetsgrad (%)") === 0 && col("trafikkmengde").isNull && col("klasse") === 0))
```

Figur 4.5: Utklipp fra [CapstoneData_FrameHandling.zpln](#) som viser filtrering av rader basert på lengde kvalitetsgrad.

Værdata

- I værdata har vi allerede fjernet kolonner som inneholder upålitelige data i sammensettingen av trafikk og værdata i [underseksjon 4.2.8](#). På grunnlag av dette har vi ikke gjort noen ytterligere vurderinger av værdata.

4.2.10 Ytelse og Optimalisering

Denne seksjonen tar for seg noen av de ytelsesforbedrende tiltakene vi har implementert i vår *Zeppelin* notatbok for å sikre effektiv behandling av store datamengder.

Dette er arbeid som er utført etter fullføring av tidligere utført arbeid med importering og klargjøring av data, etter forespørsel fra veileder.

4.2.10.1 Regex Replace

Under behandlingen av importert data har vi benyttet **regexp_replace** funksjonen for å erstatte komma med punktum i kolonner som inneholder numeriske desimalverdier, samt noen string verdier for å sikre konsistitet mellom bil og sykkel data.

Etter samtale med veileder ble vi informert om at denne funksjonen er relativt treg når den skal behandle store datamengder, og ble videre oppfordret til å benytte **replace** der det var mulig. Siden operasjonene vi utfører ikke belaster seg på *regex* funksjonalitet, men heller enkel string erstatning, har vi valgt å bytte ut **regexp_replace** med en vanlig **replace**, som er optimalisert for denne typen operasjoner.

replace VS. regexp_replace

For å kunne forsvare denne endringen, har det blitt tatt en nærmere titt på disse to funksjonene for å forstå grunnen til ytelsesforskjellen.

Listen under oppsummerer de viktigste forskjellene mellom **regexp_replace** og **replace**:

	replace	regexp_replace
Fundamentalt	Bokstavelig erstatning av delstreng.	Tolker mønsteret som et regulært uttrykk.
	Ingen forståelse av ordgrenser eller tegnklasser	Benytter ordgrenser, grupper, tegnklasser, kvatifikatorer og lignende.
	Enkel algoritme: Skann streng → Se etter nøyaktig delstreng → erstatt.	Kompleks logikk: analyserer <i>regexen</i> → bygge en matcher → utfører så skann, finn delstreng og erstatt.
Minne	Mindre midlertidig allokering.	Flere midlertidige objekter.
	Færre hjelpeobjekter, stort sett bare nye strenger når noe erstattes	Benytter matchere, gruppetilstander og annen regex logikk for å utføre oppgaven.
Ytelse	Skaleres hovedsaklig med strenglengde og antall ganger den nøyaktige delstrengen forekommer.	Skaleres med strenglengde og regex kompleksitet, som kan føre til unødvendig komplekse og krevende patologiske mønstre.
Forbruk	Tyggere og billigere når det ikke er behov for mønsterlogikk.	Bedre for jobber som inneholder separerte bokstavelige endringer, eller ved behov for mer kompleks rengjøringslogikk.

Tabell 4.3: Sammenligning av **replace** og **regexp_replace** funksjonene i **Spark**.

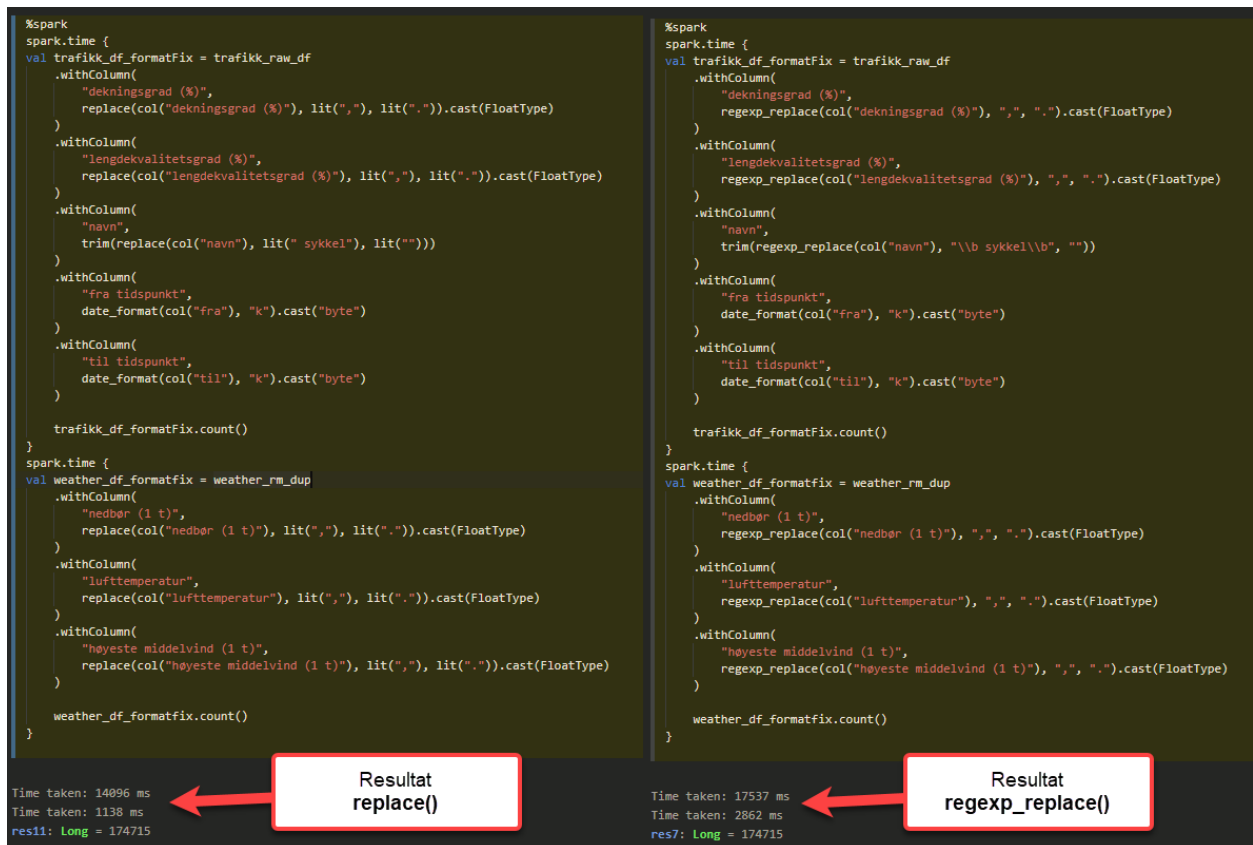
Sammenligning av Ytelse

For å kvantifisere ytelsesforskjellen mellom **replace** og **regexp_replace**, har vi utført en enkel benchmark test.

I *zeppelin* dokumentet [CapstoneData_FrameHandling.zpln](#) er det blitt implementert en timer på cellen som utfører streng erstatning i våre data.

For å sikre konsistente resultater har vi kjørt hver funksjon 5 ganger og tatt gjennomsnittet av kjøretidene. Det er også blitt lagt på en **count** for å tvinge utførelse av operasjonen, siden **Spark** benytter *lazy evaluation*^[59] som standard.

I bildet under kan det observeres at **replace** funksjonen presterer bedre enn **regexp_replace** i vår test.

Figur 4.6: Sammensatt utklipp fra [CapstoneData_FrameHandling.zpln](#)

Videre observeres det at begge funksjoner har en relativt tilsvarende kjøretid ved måling av gjennomsnitt over 5 kjøringene. Dette kan tyde på at datamengden som blir behandlet i denne testen er for liten til å kunne observere en tydelig forskjell i ytelse mellom de to funksjonene. Her ville det vært aktuelt å øke datamengden for å kunne observere en tydeligere forskjell i ytelse.

Funksjon	replace	regexp_replace
Tid i ms	21577	21122
	18335	18475
	19282	18922
	21093	20605
	18089	17310
Snitt	19675.2	19286.8

Tabell 4.4: Sammeligning av kjøretider for `replace` og `regexp_replace` funksjonene i `Spark` over 5 kjøringene.

Cassandra

5.1 Introduksjon

I denne kapittelet går gjennom den helhetlige prosessen med å designe, opprette og fylle en *Cassandra* database basert på den ferdig behandlede *dataframen* som ble laget i [kapittel 4](#).

- [seksjon 5.3](#) dekker opprettelse av **CQL** spørringer som er formet rundt hypoteser oppført i [underseksjon 4.1.3](#).
- [seksjon 5.4](#) tar for seg opprettelse av tabeller i *Cassandra* og databasens struktur for å imøtekomme spørringene.
- I [seksjon 5.5](#) utføres dataeksporten fra *Spark* til *Cassandra* databasen.
- [seksjon 5.6](#) illustrerer resultatene oppnådd ved å kjøre spørringene mot databasen. Her benyttes grafiske fremstillinger for å visualisere resultatene, samt gjøre analytiske vurderinger av funn.

5.2 Keyspace ([1_Capstone_CQL_DB_Build.zpln](#))

Før opprettelse av tabeller i *Cassandra* må en *keyspace* opprettes. En *keyspace* fungerer som en beholder for tabeller og definerer replikasjonsstrategien for dataene.

I dette prosjektet opprettes en *keyspace* kalt **capstone_ks** med en enkel replikasjonsstrategi som replikerer dataene på én node, passende for utviklings- og testmiljøer.

```
CREATE KEYSPACE IF NOT EXISTS capstone_ks
  WITH replication = {
    'class' : 'SimpleStrategy',
    'replication_factor' : 1
  };
```

5.3 CQL Spørringer ([3_Capstone_Cassandra_Queries.zpln](#))

5.3.1 Beskrivelse

Denne seksjonen tar for seg opprettelse av **CQL** spørringer som er formet rundt hypoteser oppført i [underseksjon 4.1.3](#).

Spørringene er designet for å hente ut relevant informasjon fra *Cassandra* databasen for å kunne validere eller avkrefte hypotesene.

Listen her består av hypotesen etterfulgt av tilhørende **CQL** spørringen.

5.3.2 Hypotese 1

Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 og 16 : 00.

5.3.2.1 Utdypelse

Denne spørringen er designet for å hente ut trafikkdata for *Glemmen* området over en ukes periode. Den vil hente ut alle time registreringer innefor denne perioden slik at trafikkmengden kan analyseres for å identifisere rushtidsmønstre.

5.3.2.2 Løsning

```
SELECT * FROM capstone_ks.rushtid
  WHERE navn = 'Glemmen'
  AND dato > '2024-03-04'
  AND dato < '2024-03-08';
```

5.3.3 Hypotese 2

Den totale trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.

5.3.3.1 Utdypelse

Denne spørringen er designet for å hente ut en månedlig telling av den totale bil trafikkmengden ved *Glemmen* for årene 2022 og frem til siste registrerte punkt. Dette vil tillate en sammenligning av trafikkmengden i juli mot andre måneder i året for å validere hypotesen.

5.3.3.2 Løsning

```
SELECT * FROM capstone_ks.trafikkmengde_mnd
  WHERE navn = 'Glemmen'
  AND yr >= 2022;
```

5.3.4 Hypotese 3

Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.3.4.1 Utdypelse

Med denne spørringen hentes data for sykkeltrafikk og tilhørende lufttemperatur ved registreringspunktet *Wilbergjordet* over en definert tidsperiode. Dette vil tillate analyse av korrelasjonen mellom temperatur og sykkeltrafikk i det angitte tidsrommet.

5.3.4.2 Løsning

```
SELECT navn, trafikkmengde, lufttemperatur FROM capstone_ks.sykkel_temp_corr
WHERE navn = 'Wilbergjordet'
AND dato >= '2022-02-01'
AND dato < '2025-10-01';
```

5.3.5 Hypotese 4

Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.3.5.1 Utdypelse

Spørringen henter data for biltrafikk og tilhørende lufttemperatur ved registreringspunktet *Stabburet* over en definert tidsperiode. Dette vil tillate analyse av korrelasjonen mellom temperatur og biltrafikk i det angitte tidsrommet.

5.3.5.2 Løsning

```
SELECT navn, trafikkmengde, lufttemperatur FROM capstone_ks.bil_temp_corr
WHERE navn = 'Stabburet'
AND dato >= '2022-02-01'
AND dato < '2025-10-01';
```

5.3.6 Hypotese 5

Data kan benyttes til å forutsi om det er blitt utført veiarbeid eller vedlikehold på et gitt trafikk registreringspunkt basert på uregelmessigheter i trafikkmålinger.

5.3.6.1 Utdypelse

Denne spørringen er designet for å hente ut alle eksisterende registreringer over tid hvor *dekningsgrad*(%) = 0 ved *Simo* sitt trafikk registreringspunkt. Har kan vi analysere mønstre i dataene for å identifisere potensielle uregelmessigheter som kan indikere veiarbeid eller vedlikehold.

5.3.6.2 Løsning

```
SELECT * FROM capstone_ks.veiarbeid
WHERE navn = 'Simo';
```

5.4 Database/Tabell Design ([1_Capstone_CQL_DB_Build.zpln](#))

5.4.1 Beskrivelse

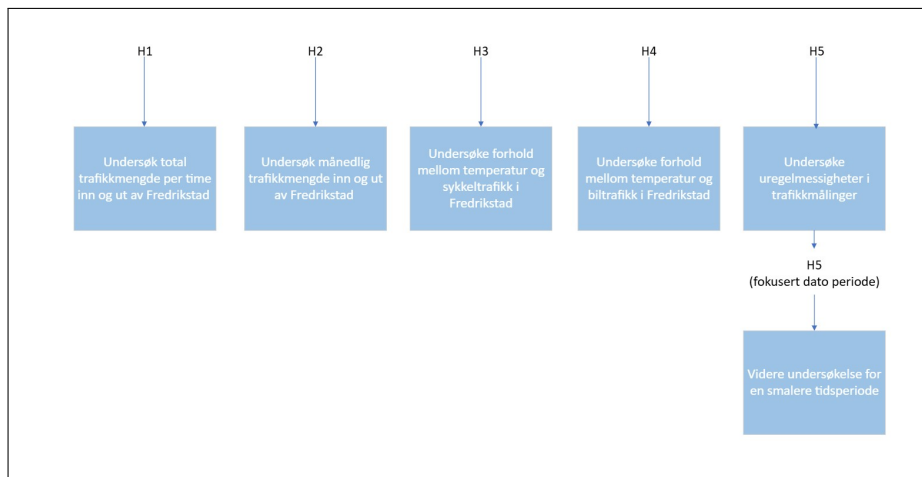
Denne seksjonen tar for seg designet av *Cassandra* databasen.

Tabeller som blir opprettet her er basert på den ferdig behandlede *dataframen* som ble laget i [seksjon 4.2](#).

Tabellenes design og struktur er formet til å passe spørringer fra [seksjon 5.3](#), som videre er orientert rundt hypoteser oppført i [underseksjon 4.1.3](#).

5.4.2 Applikasjonsflyt

Vi har valgt å svare på hypotesene våre med en løsning med som gir oss en enkel applikasjonsflyt (se figur [5.1](#)). Istedenfor å lage mere komplekse nestede spørringer for å svare på hypotesene våre har vi valgt å gjøre mye av denne jobben gjennom databehandlingen i spark, før overføring av data til tabellene. For å løse dette har vi laget spørringer som er tilpasset den enkelte hypotesen og deretter designet tabeller som er tilpasset disse spørringene, før vi har behandlet dataene spark for overføring til den enkelte tabellen.

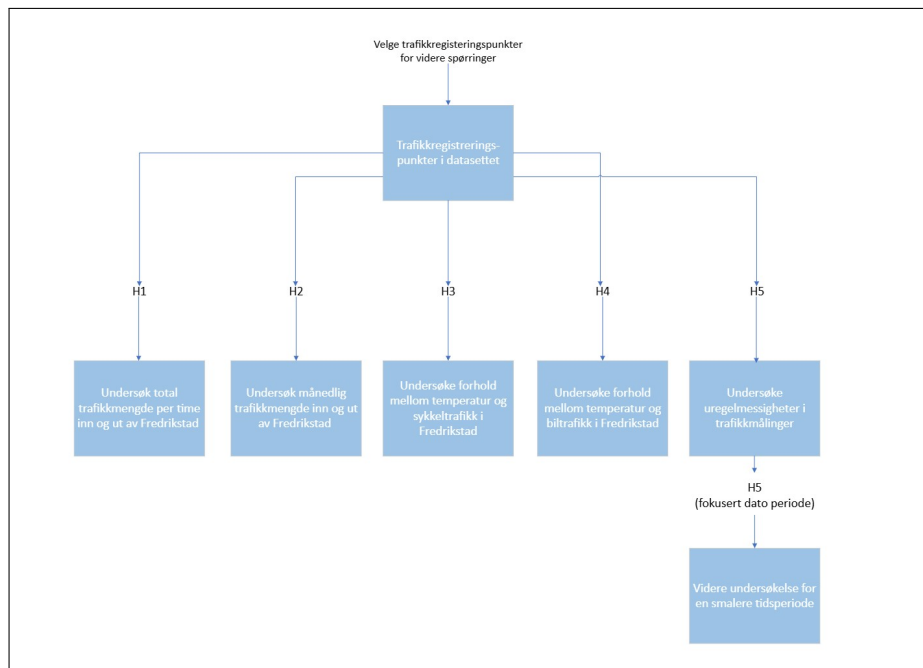


Figur 5.1: Figuren viser en enkel illustrasjon for applikasjonsflyt.

5.4.2.1

Mot slutten av prosjektet ønsket vi også å inkludere dynamiske/interaktive spørringer mot tabellene våre, men pga. tekniske utfordringer med koblingen mellom Spark og Cassandra har vi dessverre ikke fått inkludert dette. Vi har beskrevet flere av de tekniske utfordringene vi har hatt nærmere [under-underseksjon 6.1.1.4](#).

For å lage de interaktive spørringene var planen vår å opprette en tabell som inneholdt navnet til alle trafikkmålepunktene, samt klassen for kjøretøy for å skille mellom bil og sykkel. Videre var tanken å bruke en spørring mot denne tabellen for å lage en interaktiv liste med tilgjengelige trafikkmålepunkter som det ville hvert mulig å gjennomføre spørringer mot, valg av kjøretøy, samt spesifisering av andre kolonneverdier hvor hensiktsmessig, se figur [5.2](#) for en illustrasjon av dette.



Figur 5.2: Figuren viser en illustrasjon av alternativ applikasjonsflyt.

5.4.3 Hypotese 1

Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 og 16 : 00.

5.4.3.1 Utdypelse

Hensikten med denne tabellen er å lagre trafikkdata for hver time i løpet av en dag, slik at vi kan analysere trafikkmønstre i rushtiden.

Her er vi interessert i lokasjon, datering, ukedag, tidspunkt og trafikkmengde.

Innholdet i denne tabellen vil bli partisjonert basert på **navn** for å effektivt hente ut data for spesifikke trafikk registreringspunkter.

Hver partisjon vil deretter bli sortert etter **dato** og **fra** tidspunkt for å muliggjøre tidsbaserte spørringer innenfor hver lokasjon uten å behøve ytterligere filtrering i spørringene.

5.4.3.2 Løsning

```

CREATE TABLE IF NOT EXISTS rushtid(
    navn TEXT,
    dato DATE,
    ukedag TEXT,
    fra TINYINT,
  
```

```

    til TINYINT,
    trafikkmengde SMALLINT,
PRIMARY KEY (navn, dato, fra, trafikkmengde)
) WITH CLUSTERING ORDER BY (dato ASC, fra ASC);

```



Figur 5.3: Figuren viser Chebotko diagram for rushtid tabellen.

5.4.4 Hypotese 2

Den totale trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.

5.4.4.1 Utdypelse

Denne tabellen har lignende struktur som tabellen i Hypotese 1, men er designet for å lagre aggregerte måntelige trafikkdata.

Her er vi interessert i år, måned, lokasjon og trafikkmengde.

Innholdet i denne tabellen vil bli partisjonert basert på **navn** for å effektivt hente ut data for spesifikke trafikk registreringspunkter.

Hver partisjon vil deretter bli sortert etter **år** og **måned** for å muliggjøre tidsbaserte spørringer innenfor hver lokasjon uten å behøve ytterligere filtrering i spørringene.

Verdier for **trafikkmengde** blir aggregert under dataeksporten fra *Spark* til *Cassandra*.

5.4.4.2 Løsning

```

CREATE TABLE IF NOT EXISTS trafikkmengde_mnd(
    yr INT,
    mnd TINYINT,
    navn TEXT,
    trafikkmengde BIGINT,
PRIMARY KEY (navn, yr, mnd, trafikkmengde)
) WITH CLUSTERING ORDER BY (yr ASC, mnd ASC);

```

trafikkmengde_mnd			
navn	pk,PK	text	*
yr	pk,CK, ↑	integer	*
mnd	pk,CK, ↑	tinyint	*
trafikkmengde	pk	bigint	*

Figur 5.4: Figuren viser Chebotko diagram trafikkmengde_mnd tabellen.

5.4.5 Hypotese 3

Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.4.5.1 Utdypelse

Her er tabellens hensikt å lagre sykkeltrafikkdata sammen med tilhørende lufttemperatur for å analysere korrelasjonen mellom disse variablene.

Her er vi interessert i lokasjon, datering, tidspunkt, sykkeltrafikkmengde og lufttemperatur.

Innholdet i denne tabellen vil bli partisjonert basert på **navn** for å effektivt hente ut data for spesifikke trafikk registreringspunkter.

For å kunne svare på hypotesen og analysere korrelasjonen, vil vi trenge å hente ut timevise data over hele dager, slik at vi gjøre sammenligninger mellom hypotesens referanseperiode og andre tidsrom.

Hver partisjon vil deretter bli sortert etter **dato** for å muliggjøre tidsbaserte spørringer innenfor hver lokasjon uten å behøve ytterligere filtrering i spørringene.

5.4.5.2 Løsning

```
CREATE TABLE IF NOT EXISTS sykkel_temp_corr(
    navn TEXT,
    dato DATE,
    fra TINYINT,
    til TINYINT,
    trafikkmengde SMALLINT,
    lufttemperatur FLOAT,
    PRIMARY KEY (navn, dato, fra)
)WITH CLUSTERING ORDER BY (dato ASC);
```

sykkel_temp_corr				
::	navn	pk, PK	text	*
::	dato	pk, CK, ↑	dt	*
::	fra	pk, CK, ↑	tinyint	*
::	til		tinyint	
::	trafikkmengde		smallint	
::	lufttemperatur		float	

Figur 5.5: Figuren viser Chebotko diagram for sykkel_temp_corr tabellen.

5.4.6 Hypotese 4

Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.4.6.1 Utdypelse

Denne tabellen har tilsvarende struktur og formål som tabellen i Hypotese 3, men er designet for å lagre biltrafikkdata.

Hensikten med denne tabellen er å lagre biltrafikkdata sammen med tilhørende lufttemperatur for å analysere korrelasjonen mellom disse variablene.

For å kunne svare på hypotesen og analysere korrelasjonen, vil vi trenge å hente ut timevise data over hele dager, slik at vi gjøre sammenligninger mellom hypotesens referanseperiode og andre tidsrom.

Innholdet i denne tabellen vil bli partisjonert basert på **navn** for å effektivt hente ut data for spesifikke trafikk registreringspunkter.

Hver partisjon vil deretter bli sortert etter **dato** for å muliggjøre tidsbaserte spørringer innenfor hver lokasjon uten å behøve ytterligere filtrering i spørringene.

5.4.6.2 Løsning

```
CREATE TABLE IF NOT EXISTS bil_temp_corr(
    navn TEXT,
    dato DATE,
    fra TINYINT,
    til TINYINT,
    lufttemperatur FLOAT,
    trafikkmengde SMALLINT,
    PRIMARY KEY (navn, dato, fra)
)WITH CLUSTERING ORDER BY (dato ASC);
```

bil_temp_corr				
::	navn	pk,PK	text	*
::	dato	pk,CK, ↑	dt	*
::	fra	pk,CK, ↑	tinyint	*
::	til		tinyint	
::	lufttemperatur		float	
::	trafikkmengde		smallint	

Figur 5.6: Figuren viser Chebotko diagram for bil_temp_corr tabellen.

5.4.7 Hypotese 5

Data kan benyttes til å forutsi om det er blitt utført veiarbeid eller vedlikehold på et gitt trafikk registreringspunkt basert på uregelmessigheter i trafikkmålinger.

5.4.7.1 Utdypelse

Hensikten med denne tabellen er å lagre alle registreringer hvor *dekningsgrad*(%) = 0 for å analysere potensielle uregelmessigheter som kan indikere veiarbeid eller vedlikehold.

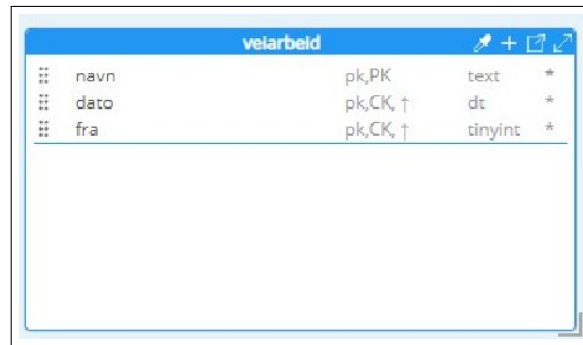
Her er vi kun interessert i loaksjon, datering og tidspunkt for registreringen.

Innholdet i denne tabellen vil bli partisjonert basert på **navn** for å effektivt hente ut data for spesifikke trafikk registreringspunkter.

Hver partisjon vil deretter bli sortert etter **dato** og **fra** tidspunkt for å muliggjøre tidsbaserte spørringer innenfor hver lokasjon uten å behøve ytterligere filtrering i spørringene.

5.4.7.2 Løsning

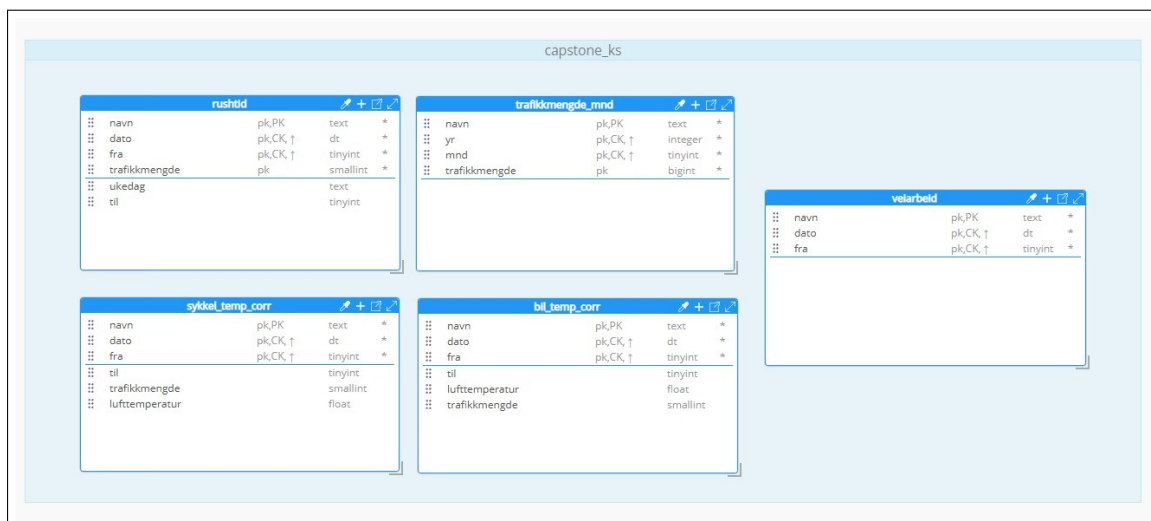
```
CREATE TABLE IF NOT EXISTS veiarbeid(
    navn TEXT,
    dato DATE,
    fra INT,
    PRIMARY KEY (navn, dato, fra)
)WITH CLUSTERING ORDER BY (dato ASC, fra ASC);
```



Figur 5.7: Figuren viser Chebotko diagram for veiarbeid tabellen.

5.4.8 Datamodell

Under ser vi en figur (5.8) av datamodellen til løsningen vår, figuren viser keyspace, tabellene og beskrivelser av kolonnen i tabellene med primærnøkler, partisjonsnøkler og klyngekolonner, samt beskrivelse av datatyper.



Figur 5.8: Bilde viser fysisk datamodell for Cassandra databasen.

5.5 Dataeksport Spark → Cassandra (2_Capstone_Frame_to_Table.zpln)

5.5.1 Beskrivelse

Denne seksjonen tar for seg prosessen med å forme *sub-dataframes*, samt eksportere de ferdig behandlede *dataframene* fra *Spark* til *Cassandra* databasen sine respektive tabeller ved hjelp av `DataFrameWriter`[60] funksjonen i *Spark*.

5.5.2 Hypotese 1

Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 og 16 : 00.

5.5.2.1 Utdypelse

For å kunne eksportere relevante data for tilhørende tabell i *Cassandra*, formes en ny *dataframe* kalt **df_rushtid**.

Denne *dataframen* inneholder kun kolonnene som passer til tabellens design.

Før vi eksporterer dataene, filtreres det på rader hvor **trafikkmengde** ikke er null, da disse radene ikke er relevante for analysen.

I tillegg filtreres det på rader hvor **felt** er lik **'Totalt'**, da kun total trafikkmengde er av interesse for denne hypotesen. På denne måten unngår vi potensielle duplikatverider av trafikkmengde fra målinger på separate felter i veibanen.

Rader som inneholder helgedager (lørdag og søndag) blir også filtrert ut, da hypotesen kun omhandler hverdager.

5.5.2.2 Løsning

```
val df_rushtid = complete_df
  .select(
    col("navn"),
    col("dato"),
    col("ukedag"),
    col("fra tidspunkt"),
    col("til tidspunkt"),
    col("trafikkmengde")
  )
  .filter(!(col("trafikkmengde").isNull))
  .filter(col("felt") === "Totalt")
  .filter(!col("ukedag").contains("Saturday"))
  .filter(!col("ukedag").contains("Sunday"))
  .withColumnRenamed("fra tidspunkt", "fra")
  .withColumnRenamed("til tidspunkt", "til")

df_rushtid.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "rushtid",
    "keyspace" -> "capstone_ks"
  ))
  .mode("append")
```

```
.save()
```

5.5.3 Hypotese 2

Den totale trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.

5.5.3.1 Utdypelse

Dataframen `df_trafikkmengde_mnd` formes for å inneholde kolonnene som er relevante for tilhørende tabell i *Cassandra*.

Før eksporten filtreres det på rader hvor `trafikkmengde` ikke er null, da disse radene ikke er relevante for analysen.

I tillegg filtreres det på rader hvor `felt` er lik **'Totalt'**, da kun total trafikkmengde er av interesse for denne hypotesen. På denne måten unngår vi potensielle duplikatverider av trafikkmengde fra målinger på separate felter i veibanen.

Deretter opprettes to nye kolonner, `mnd` og `yr`, som henter ut måneds- og årsdelen fra `dato` kolonnen. Disse kolonnene blir omformet til numeriske datatyper for å samsvare med tabellens design, da dette simplifiserer fremtidige filtreringer i spørringer.

Deretter grupperes dataene etter `yr`, `mnd` og `navn`, hvor `trafikkmengde` aggregeres ved å summere verdiene innenfor hver gruppe. Til slutt sorteres resultatet etter `yr` og `mnd` for å sikre en kronologisk rekkefølge i eksporten.

5.5.3.2 Løsning

```
val df_trafikkmengde_mnd = complete_df
  .select(
    col("navn"),
    col("dato"),
    col("ukedag"),
    col("trafikkmengde")
  )
  .filter(col("trafikkmengde").isNotNull)
  .filter(col("felt") === "Totalt")
  .withColumn("mnd", month(col("dato")).cast("byte"))
  .withColumn("yr", date_format(col("dato"), "yyyy").cast("int"))
  .groupBy("yr", "mnd", "navn")
  .agg(sum(col("trafikkmengde")).as("trafikkmengde"))
  .orderBy("yr", "mnd")

df_trafikkmengde_mnd.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "trafikkmengde_mnd",
```

```

    "keyspace" -> "capstone_ks"
  ))
  .mode("append")
  .save()

```

5.5.4 Hypotese 3

Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.5.4.1 Utdypelse

Dataframen `df_sykkel_temp_corr` formes for å inneholde kolonnene som er relevante for tilhørende tabell i *Cassandra*.

Før eksporten filtreres det på rader hvor `trafikkmengde` ikke er null, da disse radene ikke er relevante for analysen.

I tillegg filtreres det på rader hvor `felt` er lik **'Totalt'**, da kun total trafikkmengde er av interesse for denne hypotesen. På denne måten unngår vi potensielle duplikatverider av trafikkmengde fra målinger på separate felter i veibanen.

Det filtreres også på rader hvor `klasse` er lik 1, som representerer sykkeltrafikk i datasettet.

Deretter opprettes to nye kolonner, `fra` og `til`, som henter ut verdiene fra `fra tidspunkt` og `til tidspunkt` kolonnene, som er numeriske verdier for timeregistreringer.

For å fokusere på hypotesens referanseperiode, filtreres det på rader hvor `fra` ligger mellom 10 og 13, som representerer tidsrommet 10 : 00 til 14 : 00. På denne måten inkluderes alle timeintervaller på daglig basis innenfor dette tidsrommet, som vider muliggjør det å svare på hypotesen.

Før data eksporten fjernes unødvendige kolonner som ikke er relevante for tabellens design.

5.5.4.2 Løsning

```

val df_sykkel_temp_corr = complete_df.select(
  "navn",
  "dato",
  "klasse",
  "fra tidspunkt",
  "til tidspunkt",
  "felt",
  "trafikkmengde",
  "lufttemperatur")
  .filter(!(col("trafikkmengde").isNull))
  .filter(col("felt") === "Totalt")
  .filter(col("klasse") === 1)
  .withColumnRenamed("til tidspunkt", "til")
  .withColumnRenamed("fra tidspunkt", "fra")

```

```

    .filter(col("fra").between(10,13))
    .drop(col("felt"))
    .drop(col("klasse"))
    .drop(col("fra tidspunkt"))
    .drop(col("til tidspunkt"))

df_sykkel_temp_corr.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "sykkel_temp_corr",
    "keyspace" -> "capstone_ks"
  ))
  .mode("append")
  .save()

```

5.5.5 Hypotese 4

Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.5.5.1 Utdypelse

Denne *dataframen* har tilsvarende struktur og formål som **df_sykkel_temp_corr** i Hypotese 3, men er designet for å hente ut biltrafikldata.

Først hentes relevante kolonner for tabellens design.

Deretter filtreres det på rader hvor **trafikkmengde** ikke er null, da disse radene ikke er relevante for analysen.

I tillegg filtreres det på rader hvor **felt** er lik **'Totalt'**, da kun total trafikkmengde er av interesse for denne hypotesen. På denne måten unngår vi potensielle duplikatverider av trafikkmengde fra målinger på separate felter i veibanen.

Det filtreres også på rader hvor **klasse** er lik 0, som representerer biltrafikk i datasettet.

Deretter opprettes to nye kolonner, **fra** og **til**, som henter ut verdiene fra **fra tidspunkt** og **til tidspunkt** kolonnene, som er numeriske verdier for timeregistreringer.

For å fokusere på hypotesens referanseperiode, filtreres det på rader hvor **fra** ligger mellom 10 og 13, som representerer tidsrommet 10 : 00 til 14 : 00. På denne måten inkluderes alle timeintervaller på daglig basis innenfor dette tidsrommet, som vider muliggjør det å svare på hypotesen.

Før data eksporten fjernes unødvendige kolonner som ikke er relevante for tabellens design.

5.5.5.2 Løsning

```

val df_bil_temp_corr = complete_df.select(
  "navn",
  "dato",

```

```

      "klasse",
      "fra tidspunkt",
      "til tidspunkt",
      "felt",
      "trafikkmengde",
      "lufttemperatur")
    .filter(!(col("trafikkmengde").isNull))
    .filter(col("felt") === "Totalt")
    .filter(col("klasse") === 0)
    .withColumnRenamed("til tidspunkt", "til")
    .withColumnRenamed("fra tidspunkt", "fra")
    .filter(col("fra").between(10,13))
    .drop(col("felt"))
    .drop(col("klasse"))
    .drop(col("fra tidspunkt"))
    .drop(col("til tidspunkt"))

df_bil_temp_corr.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "bil_temp_corr",
    "keyspace" -> "capstone_ks"
  ))
  .mode("append")
  .save()

```

5.5.6 Hypotese 5

Data kan benyttes til å forutsi om det er blitt utført veiarbeid eller vedlikehold på et gitt trafikk registreringspunkt basert på uregelmessigheter i trafikkmålinger.

5.5.6.1 Utdypelse

Siden den ferdigstilte *dataframen* har filtrert ut de relevante radene for denne løsningen, har **complete_df_raw** blitt benyttet for denne eksporten.

Denne *dataframen* inneholder rader med *dekningsgrad*(%) = 0 før de blir filtrert ut i den endelige *dataframen*.

For å innhente data av interesse for denne hypotesen, formes en ny *dataframe* som kun inneholder kolonnene **navn**, **dato** og **fra tidspunkt**.

Deretter filtreres det på rader hvor **dekningsgrad** (%) er lik 0.

Til slutt eksporteres denne *dataframen* til *Cassandra* tabellen **veiarbeid** som har blitt definert til å sortere innkommende data etter dato og klokkeslett.

5.5.6.2 Løsning

```
val df_veiarbeid = complete_df_raw
    .select(
        col("navn"),
        col("dato"),
        col("fra tidspunkt"),
    )
    .withColumnRenamed("fra tidspunkt", "fra")
    .filter(col("dekningsgrad (%)") === 0)

df_veiarbeid.write
    .format("org.apache.spark.sql.cassandra")
    .options(Map(
        "table" -> "veiarbeid",
        "keyspace" -> "capstone_ks"
    ))
    .mode("append")
    .save()
```

5.6 Resultat

5.6.1 Beskrivelse

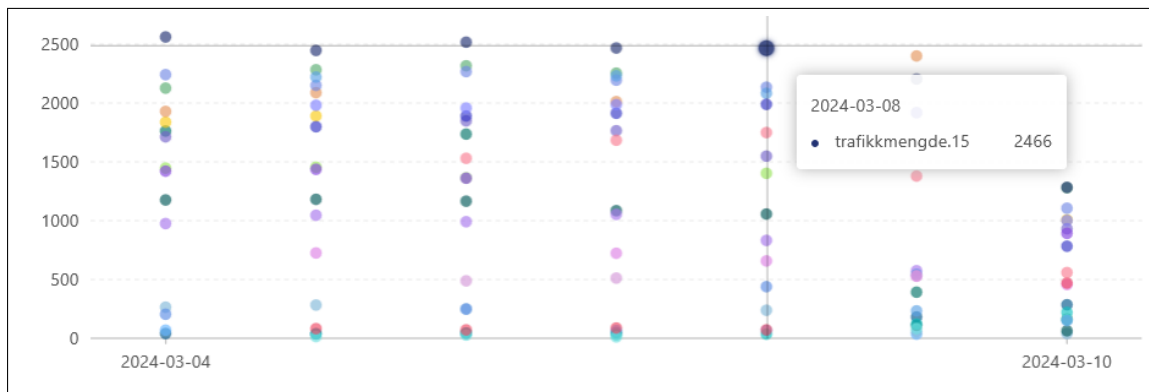
I denne seksjonen blir det utført analytisk utarbeiding av funn basert på spørringer fra [seksjon 5.3](#). Grafiske illustrasjoner er generert fra spørringene i [3_Capstone_Cassandra_Queries.zpln](#). Her blir hypoteser validert eller avkreftet basert på de innhentede resultatene.

5.6.2 Hypotese 1

Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 og 16 : 00.

5.6.2.1 Utdypelse

For å illustrere, samt vurdere resultatene fra denne spørringen, har det blitt generert et *plotdiagram* som viser trafikkmengden for hver påbegynte time over et døgn i løpet av perioden 2024 – 03 – 04 til 2024 – 03 – 10 ved registreringspunktet i *Glemmen*. Her kan vi se at den gjennomsnittlige trafikkmengden er høyest mellom 15 : 00 – 16 : 00 med unntak av lørdag 2024 – 03 – 09. Siden vi kun ser på hverdager i denne hypotesen, kan vi se bort i fra denne avvikende dagen.



Figur 5.9: Utklipp fra [3_Capstone_Cassandra_Queries.zpln](#) som viser spørringsresultat for hypotesen i et *pie-chart*.

5.6.2.2 Konklusjon

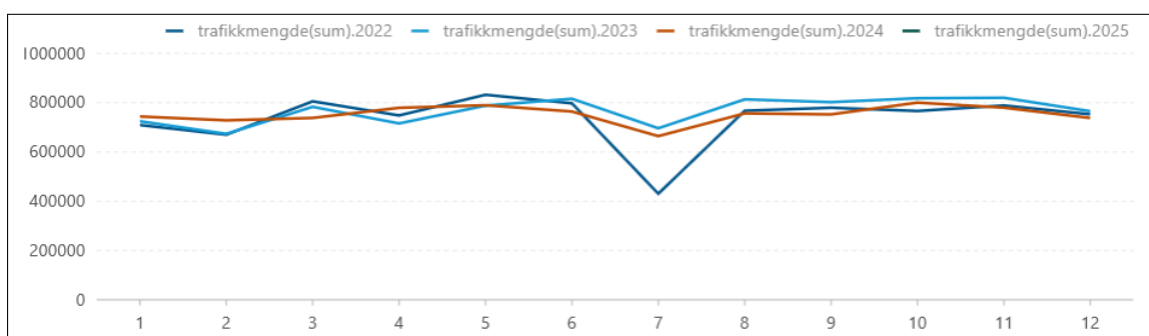
Basert på dette funnet kan det konkluderes med at hypotesen er sann, da den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 og 16 : 00.

5.6.3 Hypotese 2

Den totale trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.

5.6.3.1 Utdypelse

For å kunne evaluere vår hypotese har det blitt generert et *linjediagram* som viser den totale månedlige bil trafikkmengden ved registreringspunktet på *Glemmen* for alle år i perioden 2022 til 2025. Her kan vi se at trafikkmengden i juli måned får en nedgang sammenlignet med de andre månedene i året.



Figur 5.10: Utklipp fra [3_Capstone_Cassandra_Queries.zpln](#) som viser spørringsresultat for hypotesen i et *pie-chart*.

5.6.3.2 Konklusjon

Basert på dette funnet foreligger det sterke indikasjoner på at hypotesen er sann, da den totale trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.

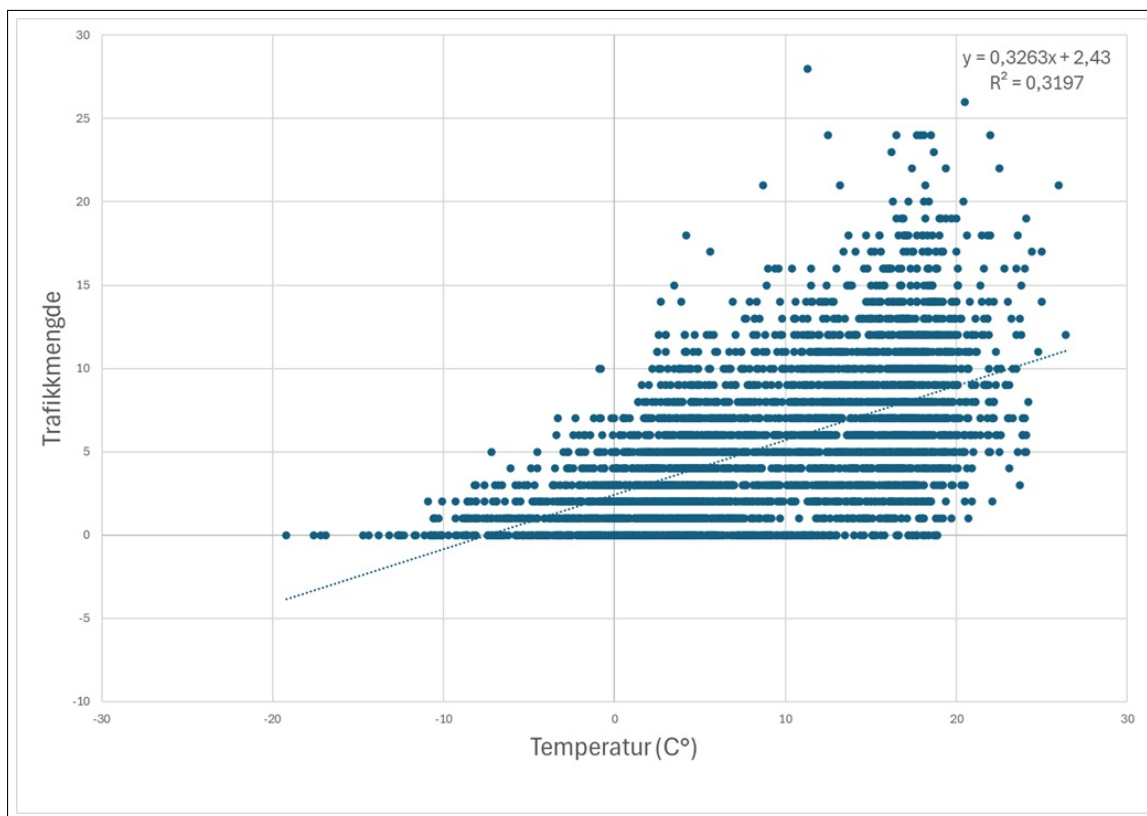
5.6.4 Hypotese 3

Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.6.4.1 Utdypelse

For å analysere denne hypotesen så har vi hentet ut data for målepunktet Wilbergjordet i perioden fra 2022-02-01 til 2025-10-01. Da Zeppelin ikke har muligheten til å generere de ønskede analysene, har vi eksportert verdiene til Excel. For å gjennomføre analysen har vi laget sprednings plott med trafikkmengde som y-verdier og temperatur som x-verdier, samt lagt til tilhørende trendlinje i plottet. Vi har også fjernet noen utligger verdier (5) for å få et mer oversiktlig plott da disse verdiene lå langt utenfor det normale verdi område i dataene.

Som vi kan se av figur 5.11 finner vi en svak positiv korrelasjon med en r^2 på 0,32 noen som bekrefter at det er et svakt, men positivt forhold mellom temperatur og sykkeltrafikk.



Figur 5.11: Bilde utklipp fra Excel som viser sprednings plott for trafikkmengde og temperatur med tilhørende trendlinje.

5.6.4.2 Konklusjon

Det foreligger en svak positiv korrelasjon på mellom temperatur og sykkeltrafikk og vi kan dermed bekrefte hypotesen validitet.

5.6.5 Hypotese 4

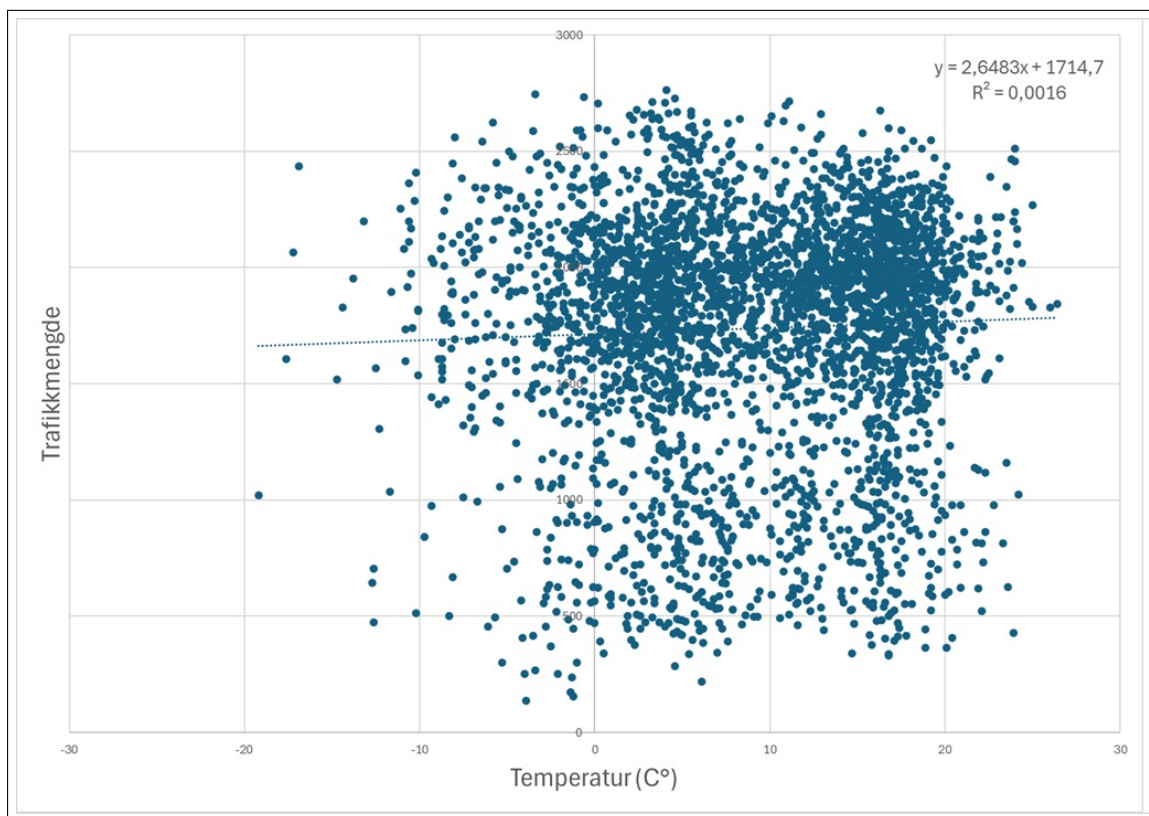
Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10 : 00 – 14 : 00.

5.6.5.1 Utdypelse

For å analysere denne hypotesen så har vi hentet ut data for målepunktet Stabburet i perioden fra 2022-02-01 til 2025-10-01. Akkurat som for hypotese 3, har vi eksportert dataene til Excel for å

gjennomføre analysen der. Vi har laget tilsvarende analyse for disse verdiene med et sprednings plott med trafikkmengde som y-verdier og temperatur som x-verdier, samt lagt til tilhørende trendlinje i plottet.

Ut i fra resultatene i figur 5.12 ser vi at vi ikke finner noen korrelasjon, med kun en r^2 verdi på 0,0016 noe som betyr at det ikke finnes en negativ korrelasjon mellom biltrafikk og temperatur for disse verdiene.



Figur 5.12: Bilde utklipp fra Excel som viser sprednings plott for trafikkmengde og temperatur med tilhørende trendlinje.

5.6.5.2 Konklusjon

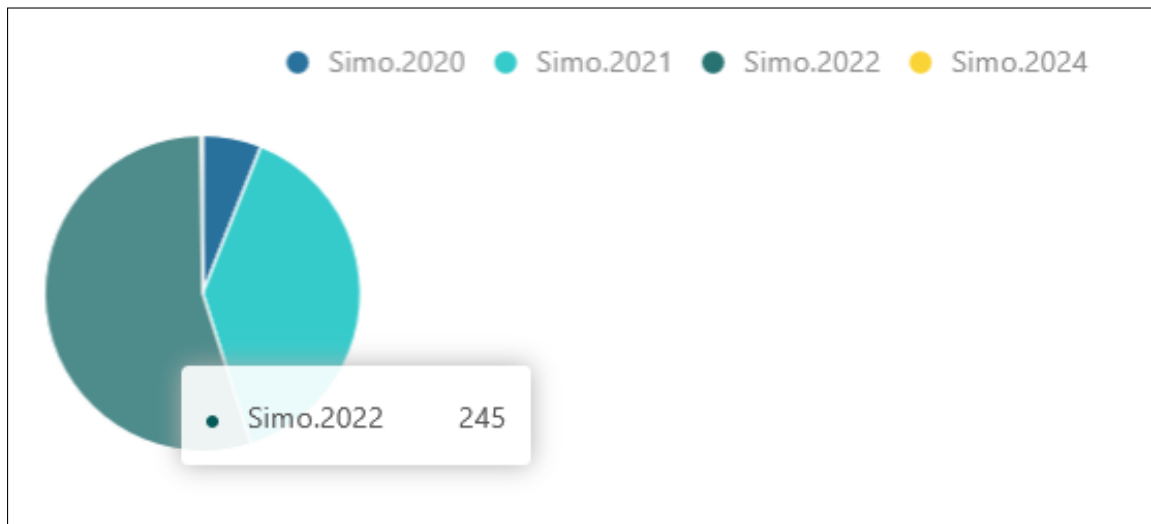
Det foreligger ingen korrelasjon mellom temperatur og biltrafikk og vi kan forkaste hypotesen.

5.6.6 Hypotese 5

Data kan benyttes til å forutsi om det er blitt utført veiarbeid eller vedlikehold på et gitt trafikk registreringspunkt basert på uregelmessigheter i trafikkmålinger.

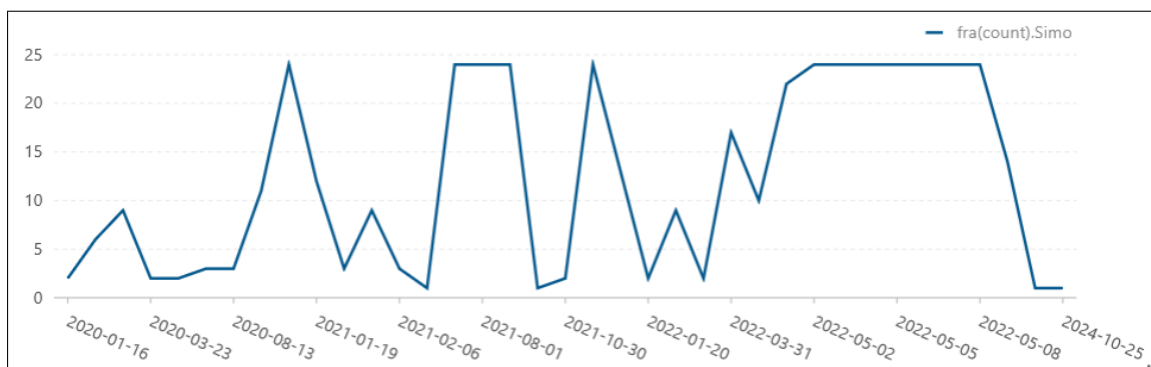
5.6.6.1 Utdypelse

Her vises to grafiske fremstillinger av spøringsresultatene for hypotesen. Den første illustrasjonen er et *pie-chart* som viser fordelingen av registreringer med *dekningsgrad*(%) = 0 i registreringspunkter *Simo*. Dette benyttes til å vurdere om det er noen årsperioder med høyere forekomst av manglende dekningsgrad. Vi kan se at året 2022 inneholder majoriteten av disse registreringene. Det kan dermed være interessant å se om det forekommer sammenhengende perioder innenfor dette året som kan indikere veiarbeid eller vedlikehold.



Figur 5.13: Utklipp fra [3_Capstone_Cassandra_Queries.zpln](#) som viser spøringsresultat for hypotesen i et *pie-chart*.

Den andre illustrasjonen er et *linjediagram* som viser en tidslinje av registreringer med *dekningsgrad*(%) = 0 ved *Simo* sitt trafikk registreringspunkt. Her kan vi se at det er en ukesperiode i mai 2022 hvor dekningsgraden er 0 hele døgnet igjennom. Dette kan indikere at det har blitt utført veiarbeid eller vedlikehold i dette tidsrommet.



Figur 5.14: Utklipp fra [3_Capstone_Cassandra_Queries.zpln](#) som viser spøringsresultat for hypotesen i et *linjediagram*.

5.6.6.2 Konklusjon

Basert på dette funnet kan det antas at dataene kan benyttes til å forutsi veiarbeid eller vedlikehold basert på uregelmessigheter i trafikkmålinger og at det foreligger sterke indikasjoner på at hypotesen er sann.

Retrospekt

6.1 Utfordringer

Denne seksjonen omhandler ulike utfordringer knyttet til prosjektet. Vi vil diskutere tekniske vanskeligheter, tidsstyring, samarbeid i teamet, og andre relevante problemstillinger som oppstod underveis i arbeidet. Målet er å gi en ærlig vurdering av de hindringene vi møtte, samt hvordan vi har forsøkt å overkomme dem.

6.1.1 Tekniske Utfordringer

Under utviklingen av prosjektet støtte vi på flere tekniske utfordringer. Et av de mest fremtredende problemene var at våre maskiner ikke hadde nok ytelse til å kjøre de nødvendige handlingene, til det punktet hvor systemer har kontinuerlig fryst og vi har måttet restarte maskinene flere ganger.

6.1.1.1 Docker på Windows Maskiner

Med denne løsningen er det forsøkt å kjøre Docker lokalt på maskinene våre med de instruksjoner og oppsett som er blitt utdelt av veileder (Konfigurasjons instruksjoner fra forelesning 6 presentasjon, samt miljø fra "Utviklingsmiljø Docker V3.zip").

Dette har fungert svært bra i henhold til *spark* relaterte oppgaver.

Alle i gruppen har uten problem klart å opprette *dataframes* og behandlet disse ut i fra de oppgaver som har blitt gitt.

Dessverre har det oppstått større utfordringer vedrørende *Cassandra/CQL* relaterte oppgaver. I flere tilfeller har det vist seg at *Cassandra* ikke har klart å starte opp som forventet. Dette har ført til at vi har måttet bruke mye tid på å feilsøke og prøve forskjellige løsninger for å få *Cassandra* til å fungere som ønsket.

Etter flere forsøk og feilsøk, har vi funnet ut at problemet kan skyldes begrenset maskinressurser på våre lokale maskiner. Denne konklusjonen er basert på at når vi har forsøkt å føre data inn i en *Cassandra* tabell fra en *DataFrame*, har systemet fryst og blitt uresponsivt.

Etter lengre venting får vi ofte tilbakemelding om at *Docker* containeren har mistet kontakt med *Cassandra* nodene. I andre tilfeller får vi tilbakemelding om at alle noder er opptatt og at vi må prøve igjen senere, til tross for at det ikke foreligger noen aktive jobber i *Cassandra* sin kø som vi har startet opp.

Selve utfordringens natur har variert fra maskin til maskin i gruppen, men utfallet har vært mye av det samme i form av at oppgaver relatert til *Cassandra/CQL* har vist seg å være svært tidkrevende og vanskelig å gjennomføre.

Enkelte har opplevd at miljøet får problemer når de skal opprette/endre et *keyspace* og tilhørende tabeller, mens andre har hatt problemer med å føre data inn i tabellene.

Dette har ført til at vi har måttet vurdere alternative løsninger for å kunne gjennomføre oppgavene

knyttet til *Cassandra/CQL*.

Dette er den siste versjonen av *docker-compose.yml* filen som vi har jobbet med på Windows maskiner:

Kode 6.1: Docker Compose fil - siste versjon på Windows 11

```
networks:
  bigdata:
    driver: bridge

volumes:
  cassandra-seed-data:
  cassandra-1-data:
  cassandra-2-data:
  zeppelin-data:

services:
  # -----
  # Spark
  # -----
  spark-master:
    image: apache/spark:3.5.6
    platform: "linux/amd64"
    container_name: spark-master
    environment:
      - SPARK_MASTER_HOST=spark-master
      - SPARK_NO_DAEMONIZE=true
      # Optional: tune Spark UI host if needed
      # - SPARK_MASTER_HOST=spark-master
    command: >
      bash -lc "/opt/spark/sbin/start-master.sh && tail -F /opt/spark/logs/*
        master*.out"
    volumes:
      - ./sparkio:/sparkio
    ports:
      - "7077:7077" # Spark master RPC
      - "8080:8080" # Spark master UI
    networks:
      - bigdata
    restart: unless-stopped

  spark-worker-1:
    image: apache/spark:3.5.6
    container_name: spark-worker-1
    command:
      - /bin/bash
      - -lc
      - >
        /opt/spark/sbin/start-worker.sh --cores 2 --memory 2G spark://spark-
        master:7077 &&
        tail -F /opt/spark/logs/*worker*.out
    volumes:
      - ./sparkio:/sparkio
    depends_on:
```

```

    - spark-master
ports:
  - "8081:8081" # Worker UI (worker-1)
networks:
  - bigdata
restart: unless-stopped

spark-worker-2:
  image: apache/spark:3.5.6
  platform: "linux/amd64"
  container_name: spark-worker-2
  command:
    - /bin/bash
    - -lc
    - >
      /opt/spark/sbin/start-worker.sh --cores 2 --memory 2G spark://spark-
        master:7077 &&
      tail -F /opt/spark/logs/*worker*.out
  volumes:
    - ./sparkio:/sparkio
  depends_on:
    - spark-master
  # UI is available within the network; map if you want it on host:
  # ports:
  # - "8083:8081"
  networks:
    - bigdata
  restart: unless-stopped

spark-worker-3:
  image: apache/spark:3.5.6
  platform: "linux/amd64"
  container_name: spark-worker-3
  command:
    - /bin/bash
    - -lc
    - >
      /opt/spark/sbin/start-worker.sh --cores 2 --memory 2G spark://spark-
        master:7077 &&
      tail -F /opt/spark/logs/*worker*.out
  volumes:
    - ./sparkio:/sparkio
  depends_on:
    - spark-master
  # ports:
  # - "8084:8081"
  networks:
    - bigdata
  restart: unless-stopped

# -----
# Cassandra (3-node cluster)
# -----

```

```
cassandra-seed:
  image: cassandra:4.1
  container_name: cassandra-seed
  hostname: cassandra-seed
  environment:
    - CASSANDRA_CLUSTER_NAME=demo-cluster
    - CASSANDRA_SEEDS=cassandra-seed
    - CASSANDRA_NUM_TOKENS=256
    - MAX_HEAP_SIZE=1G
    - HEAP_NEWSIZE=256M
  volumes:
    - cassandra-seed-data:/var/lib/cassandra
  ports:
    - "9042:9042" # CQL
    # (Optional) expose JMX if you need it:
    # - "7199:7199"
  networks:
    - bigdata
  healthcheck:
    test: ["CMD-SHELL", "cqlsh -e 'SHOW VERSION' 127.0.0.1 9042"]
    interval: 10s
    timeout: 5s
    retries: 30
  restart: unless-stopped

cassandra-1:
  image: cassandra:4.1
  container_name: cassandra-1
  hostname: cassandra-1
  environment:
    - CASSANDRA_CLUSTER_NAME=demo-cluster
    - CASSANDRA_SEEDS=cassandra-seed
    - CASSANDRA_NUM_TOKENS=256
    - MAX_HEAP_SIZE=1G
    - HEAP_NEWSIZE=256M
  volumes:
    - cassandra-1-data:/var/lib/cassandra
  depends_on:
    - cassandra-seed
  networks:
    - bigdata
  healthcheck:
    test: ["CMD-SHELL", "cqlsh -e 'SHOW VERSION' 127.0.0.1 9042"]
    interval: 10s
    timeout: 5s
    retries: 30
  restart: unless-stopped

cassandra-2:
  image: cassandra:4.1
  container_name: cassandra-2
  hostname: cassandra-2
  environment:
```

```

- CASSANDRA_CLUSTER_NAME=demo-cluster
- CASSANDRA_SEEDS=cassandra-seed
- CASSANDRA_NUM_TOKENS=256
- MAX_HEAP_SIZE=1G
- HEAP_NEWSIZE=256M
volumes:
- cassandra-2-data:/var/lib/cassandra
depends_on:
- cassandra-seed
networks:
- bigdata
healthcheck:
  test: ["CMD-SHELL", "cqlsh -e 'SHOW VERSION' 127.0.0.1 9042"]
  interval: 10s
  timeout: 5s
  retries: 30
restart: unless-stopped

# -----
# Zeppelin
# -----
zeppelin:
  image: apache/zeppelin:0.12.0
  environment:
    - MASTER=spark://spark-master:7077
    - SPARK_HOME=/opt/spark
    - SPARK_PUBLIC_DNS=localhost
  volumes:
    - ./sparkio:/sparkio
    - ./opt/spark-3.5.6-bin-hadoop3:/opt/spark:ro
    - zeppelin-data:/zeppelin
  ports:
    - "8082:8080" # Zeppelin UI
    - "4040-4050:4040-4050" # Spark app UIs from Zeppelin
  depends_on:
    spark-master:
      condition: service_started
    cassandra-seed:
      condition: service_healthy
    cassandra-1:
      condition: service_healthy
    cassandra-2:
      condition: service_healthy
  networks: ["bigdata"]

```

6.1.1.2 Docker på Linux Maskiner

Etter mange timer med feilsøking på den originale løsningen med Docker på Windows maskiner, valgte enkelte i gruppen å prøve en alternativ tilnærming ved å bruke Linux maskiner for å kjøre Docker. Dette virket som en lovende løsning, da Linux generelt sett har bedre støtte for Docker og

containerisering.

Første store utfordring etter at *Docker* var oppe og kjørte på Linux maskinene, var å få kjørt opp '*dockerdocker-compose.yml*' filen som vi hentet ned fra *Canvas* portalen.

Blandt annet måtte det foretas en del endringer for at docker skulle klare å hente ned '*images* til docker miljøet.

Her viste det seg å være utfordrende å finne relevant informasjon for feilhåndtering, så det har blitt foretatt ganske tungt bruk av *LLM* verktøy[61] for å finne mulige løsninger.

Første feilmelding: **Error response from daemon: manifest for bitnami/spark:3.5.6 not found: manifest unknown: manifest unknown.**

Denne feilmeldingen indikerte at den spesifikke versjonen av *Spark* som var angitt i *docker-compose.yml* filen ikke var tilgjengelig i *Docker Hub* sitt register. For å løse dette, måtte vi oppdatere *docker-compose.yml* filen til å bruke en tilgjengelig versjon av *Spark*.

Endring i yml;

– Spark:

'image: bitnami/spark:3.5.6' → 'image: apache/spark:3.5.6'

Etter å ha endret versjonen til en som eksisterte, klarte vi å laste ned *image* uten problemer.

Når miljøet var ferdig oppdatert ble det gjort et ny forsøk på å starte opp en sesjon med *Zeppelin Notebook*.

I starten virket dette svært lovende. Siden vi nå hadde tilnærmed dobbelt tilgjengelige resurser for miljøet, så var *spark* koding utrolig effektivt og vi klarte å lese millioner av rader fra *TLC data* inn i en *DataFrame* på under 5 sekunder.

Dessverre returnerte tidligere problemer når vi forsøkte å jobbe med *Cassandra/CQL* relaterte oppgaver. Når vi forøkte å føre data inn i en *Cassandra* tabell obeserverte vi at systemet brukte alle tilgjengelige resurser på maskinen (16CPU kjerner og 32GB RAM). Dette førte til at maskinen ble uresponsiv og vi måtte restarte den for å få den tilbake i normal tilstand.

Dette ledet til et lang sesjon med feilsøking og testing av forskjellige konfigurasjoner. Resultatet av dette arbeidet ledet til en **YAML** fil med en god del endringer fra dem originale. Det er blandt annet satt strengere begrensninger på hvor mye ressurser hver enkelt *container* kan bruke.

Dette viste seg å være en utfordring å implemnetere, da docker miljøet ikke samsvarte med våre forventninger i henhold til det nye oppsettet. Blandt annet så oppstod det en **SPARK** arbeider som hadde tilgang til hele maskinens ressurser, til tross for at denne arbeideren ikke var ført opp i *docker-compose.yml* filen.

Per dags dato har vi ikke klart å løse dette problemet, og det har ført til at vi fortsatt opplever at maskinene blir uresponsiv når vi forsøker å jobbe med *Cassandra/CQL* relaterte oppgaver.

Vi tar høyde for at disse konfigurasjonene kan inneholde feil, da vår erfaring med *Docker* og **YAML** filer er begrenset, og vi kommer til å fortsette dette arbeidet fremover.

Dette er oppsettet på siste versjon av *docker-compose.yml* filen vi har jobbet med:

Kode 6.2: Docker Compose fil - siste versjon på Linux

```
networks:
  bigdata:
    driver: bridge
```

```
volumes:
  cassandra-seed-data: null
  cassandra-1-data: null
  cassandra-2-data: null
  zeppelin-data: null
services:
  spark-master:
    image: apache/spark:3.5.6
    container_name: spark-master
    hostname: spark-master
    command:
      - /opt/spark/bin/spark-class
      - org.apache.spark.deploy.master.Master
      - --host
      - spark-master
      - --port
      - '7077'
      - --webui-port
      - '8080'
    environment:
      - SPARK_MODE=master
    cpus: '2.0'
    mem_limit: 2g
    ports:
      - 7077:7077
      - 8080:8080
    networks:
      - bigdata
    volumes:
      - ./sparkio:/sparkio:ro
  spark-worker-1:
    image: apache/spark:3.5.6
    container_name: spark-worker-1
    hostname: spark-worker-1
    depends_on:
      - spark-master
    command:
      - /opt/spark/bin/spark-class
      - org.apache.spark.deploy.worker.Worker
      - spark://spark-master:7077
      - --webui-port
      - '8081'
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_CORES=4
      - SPARK_WORKER_MEMORY=6g
      - SPARK_DAEMON_MEMORY=1g
    cpus: '4.0'
    mem_limit: 6g
    networks:
      - bigdata
  spark-worker-2:
```

```

    image: apache/spark:3.5.6
    container_name: spark-worker-2
    hostname: spark-worker-2
    depends_on:
      - spark-master
    command:
      - /opt/spark/bin/spark-class
      - org.apache.spark.deploy.worker.Worker
      - spark://spark-master:7077
      - --webui-port
      - '8082'
    networks:
      - bigdata
    volumes:
      - ./sparkio:/sparkio:ro
spark-worker-3:
    image: apache/spark:3.5.6
    container_name: spark-worker-3
    hostname: spark-worker-3
    depends_on:
      - spark-master
    command:
      - /opt/spark/bin/spark-class
      - org.apache.spark.deploy.worker.Worker
      - spark://spark-master:7077
      - --webui-port
      - '8083'
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_CORES=4
      - SPARK_WORKER_MEMORY=6g
      - SPARK_DAEMON_MEMORY=1g
    cpus: '4.0'
    mem_limit: 6g
    networks:
      - bigdata
    volumes:
      - ./sparkio:/sparkio:ro
cassandra-seed:
    image: cassandra:4.1
    container_name: cassandra-seed
    hostname: cassandra-seed
    environment:
      - CASSANDRA_CLUSTER_NAME=bd_cluster
      - CASSANDRA_SEEDS=cassandra-seed
      - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
      - CASSANDRA_DC=datacenter1
      - CASSANDRA_RACK=rack1
    ulimits:
      nofile:
        soft: 65536
        hard: 65536

```

```
ports:
- 9042:9042
volumes:
- /mnt/data/cassandra/seed:/var/lib/cassandra
networks:
- bigdata
cassandra-1:
  image: cassandra:4.1
  container_name: cassandra-1
  hostname: cassandra-1
  environment:
    - CASSANDRA_CLUSTER_NAME=bd_cluster
    - CASSANDRA_SEEDS=cassandra-seed
    - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
    - CASSANDRA_DC=datacenter1
    - CASSANDRA_RACK=rack1
  ulimits:
    nofile:
      soft: 65536
      hard: 65536
  depends_on:
    - cassandra-seed
  volumes:
    - /mnt/data/cassandra/node1:/var/lib/cassandra
  networks:
    - bigdata
cassandra-2:
  image: cassandra:4.1
  container_name: cassandra-2
  hostname: cassandra-2
  environment:
    - CASSANDRA_CLUSTER_NAME=bd_cluster
    - CASSANDRA_SEEDS=cassandra-seed
    - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
    - CASSANDRA_DC=datacenter1
    - CASSANDRA_RACK=rack1
  ulimits:
    nofile:
      soft: 65536
      hard: 65536
  depends_on:
    - cassandra-seed
  volumes:
    - /mnt/data/cassandra/node2:/var/lib/cassandra
  networks:
    - bigdata
zeppelin:
  image: apache/zeppelin:0.12.0
  container_name: bigdata-zeppelin-1
  environment:
    - MASTER=spark://spark-master:7077
    - SPARK_HOME=/opt/spark
    - ZEPPELIN_JAVA_OPTS=-Xms1g -Xmx2g
```

```
cpus: '2.0'
mem_limit: 3g
volumes:
- ./sparkio:/sparkio
- ./opt/spark-3.5.6-bin-hadoop3:/opt/spark:ro
- zeppelin-data:/zeppelin
ports:
- 8082:8080
- 4040-4050:4040-4050
depends_on:
- spark-master
networks:
- bigdata
spark-worker-big:
  image: apache/spark:3.5.6
  container_name: spark-worker-1-big
  hostname: spark-worker-1-big
  depends_on:
  - spark-master
  command:
  - /opt/spark/bin/spark-class
  - org.apache.spark.deploy.worker.Worker
  - spark://spark-master:7077
  - --webui-port
  - '8081'
  environment:
  - SPARK_MODE=worker
  - SPARK_MASTER_URL=spark://spark-master:7077
  - SPARK_WORKER_CORES=4
  - SPARK_WORKER_MEMORY=6g
  - SPARK_DAEMON_MEMORY=1g
  cpus: '4.0'
  mem_limit: 7g
  networks:
  - bigdata
spark-worker-small:
  image: apache/spark:3.5.6
  container_name: spark-worker-1-small
  hostname: spark-worker-1-small
  depends_on:
  - spark-master
  command:
  - /opt/spark/bin/spark-class
  - org.apache.spark.deploy.worker.Worker
  - spark://spark-master:7077
  - --webui-port
  - '8081'
  environment:
  - SPARK_MODE=worker
  - SPARK_MASTER_URL=spark://spark-master:7077
  - SPARK_WORKER_CORES=2
  - SPARK_WORKER_MEMORY=4g
  - SPARK_DAEMON_MEMORY=1g
```

```
cpus: '2.0'  
mem_limit: 5g  
networks:  
- bigdata
```

6.1.1.3 Cassandra, QCL

Under arbeidet med *Cassandra* og *CQL* har vi støtt på flere utfordringer knyttet til ytelse og stabilitet. For enkelte har det vært at *Cassandra* nodene har blitt uresponsive når det blir gjort en handling fra en *Spark DataFrame* mot en *Cassandra* tabell, som videre har ført til at innføring av data fra filer har blitt en ekstremt tidskrevende handling uten noe videre resultat.

Andre har opplevd inkonsis lagring av data i tabellene. Det er i dag uklart hva som er årsaken til dette, men vi har gjennom flere forsøk fått bekreftet at antall rader som blir skrevet til eller hentet fra en tabell ikke samsvarer med forventet antall rader. Her er det også verdt å nevne at samme spørring til samme tabell har gitt et varierende antall rader i retur.

Spark DataFrame til Cassandra Tabell

Her er et enkelt eksempel på hvordan vi har forsøkt å skrive data fra en *Spark DataFrame* til en *Cassandra* tabell.

I følge nettressurser[62] relatert til dette, virker dette som en standard fremgangsmåte for å utføre skriving fra en *DataFrame* til en *Cassandra* tabell og vi får ingen feilmeldinger vedrørende selve kodens syntaktiske oppbygning eller lignende feilmeldinger. I dette tilfellet har vi forsøkt å laste opp en *DataFrame* av minimal størrelse (tre rader) for å konkludere at problemet ikke er relatert til datamengde. Her møter vi på samme problemstilling som vi gjorde med store datamengder. Altså at **spark** koder som kjøres mot *Cassandra* miljøet ikke får kontakt med nodene(cassandra-seed,cassandra-1,cassandra-2).

Kode 6.3: Skrive DataFrame til Cassandra Tabell

```
%spark  
out.write  
  .format("org.apache.spark.sql.cassandra")  
  .options(Map("keyspace" -> "mock_tlc_ks", "table" -> "mock_tlc_raw"))  
  .mode("append")  
  .save()
```

6.1.1.4 Applikasjonsflyt

Angular i Zeppelin Notebook

I dette prosjektet har det blitt gjort forsøk på å lage dynamiske illustrasjoner av applikasjonsflyten i form av **html** orienterte *drop-down* menyer.

For å kunne implementere i *zeppelin* notebook, har vi benyttet oss av **angular** rammeverket for å skape interaktive elementer.

Dessverre har vi støtt på flere utfordringer knyttet til dette arbeidet.

Blandt annet har vi opplevd at variabler med lagrede verdier ikke blir fanget opp korrekt av *angular* sitt **scope**, noe som har ført til at uthentet data ikke blir vist i menyene som forventet.

Videre har det blitt observert en økt ustabilitet i *zeppelin* miljøet når vi har forsøkt å bygge celler som inneholder **html** og **angular** kode. Nøyaktig hva som forårsaker denne ustabiliteten er for tiden ukjent, da det ikke foreligger noen feilmeldinger eller lignende som kan gi en pekepinn på hva som går galt i docker sitt loggsystem.

Dette er de avvik vi har observert vedrørende dokumenter med **html** og **angular** kode:

- i. Notisblokk blir uresponsiv om vi forsøker å fjerne celle uavhengig av innhold.
- ii. Problemet vedvarer ved gjenoppretting av notisblokk.
- iii. Problemet oppstår ikke før celler med **html** og **angular** kode blir kjørt for første gang.

I sammenheng med dette arbeidet har vi funnet svært lite dokumentasjon eller lignende ressurser som kan hjelpe oss med å feilsøke og løse disse problemene.

Lese data fra Cassandra Tabell

En annen oppdagelse som ble gjort under dette arbeidet er periodiske kommunikasjonsproblemer mellom *Spark* og *Cassandra* miljøet. Det later til at miljøet vårt har problemer med å opprettholde en stabil forbindelse mellom *Spark* og *Cassandra* nodene over tid. Dette har ført til at enkelte *Spark* jobber som involverer *Cassandra* data, har feilet uten noen klar årsak.

Dette har hovedsaklig rammet jobber som innebærer utenting av data fra *Cassandra* tabeller til *Spark DataFrames*, da skrivning fra *DataFrames* til *Cassandra* tabeller har fungert uten å vise tegn til stabilitetsproblemer.

Ved nærmere undersøkelser av dette problemet har det blitt avdekket at problemet sannsynligvis er relatert til *Spark* ikke klarer å lese av data som *Cassandra* nodene *Broadcaster*.

Det er altså bekreftet at data eksisterer og *Spark* får kontakt med *Cassandra* nodene, men at data ikke blir overført som forventet.

På grunnlag av dette har det blitt svært implementere dynamiske applikasjonsflyter som involverer *Cassandra* data, da vi ikke har en stabil nok forbindelse til å kunne stole på at data blir hentet ut som forventet.

Forsøk på dropdown i Zeppelin

Til tross for at dette ikke er en løsning som har blitt implementert i det endelige prosjektet, var det likevel ønskelig å dokumentere hvordan dette er forsøkt løst.

Dette er et enkelt eksempel på hvordan vi har forsøkt å implementere en *dropdown* meny i *Zeppelin Notebook* ved hjelp av **angular** rammeverket:

– **Samle data:**

Først må vi samle inn de dataene som skal vises i *dropdown* menyen. Dette løste vi ved å lage en *Spark DataFrame* med de nødvendige verdiene, som videre ble serialisert til en liste i **JSON** format, slik at *angular* kunne prosessere dem.

Under her legger vi frem et eksempel på hvordan applikasjonsflyten er tenkt å fungere fra data blir samlet inn til de klargjort for å kunne brukes i en *dropdown* meny, samt koden vi har forsøkt å implementere for å oppnå dette:

Retning	Steg	Beskrivelse
↓	Cassandra	Etablerer kontakt med Cassandra noder.
↓	Spark DataFrame	Henter ut data og lagrer det som DataFrame variabler.
↓	Row limiting	Begrenser antall rader som blir hentet ut for å unngå overbelastning når data skal inn i dropdown tabell.
↓	JSON serialization (toJSON)	Omformer data til JSON format for å gjøre data mer lettleselig for <i>angular</i>
↓	Scala → Java collection conversion	Omformet data blir samlet til et object for å kunne ha et singulært referansepunkt.
↓	Zeppelin angularBind	Hentet data blir bunnet til et <i>angular</i> objekt, slik at data blir tilgjengelig i dropdown strukturen.

Tabell 6.1: Applikasjonsflyt for data innsamling til dropdown meny

I denne koden har vi laget funksjoner for å hente data fra forskjellige *Cassandra* tabeller, begrense antall rader som blir hentet ut, serialisere data til **JSON** format, og samle data i et **Java Map** objekt som kan bindes til *angular* i *Zeppelin Notebook*.

Det er blitt benyttet *HashMap* og *Java List* for å strukturere dataene på en måte som er kompatibel med *angular* sitt **scope**.

Det kommer ikke frem i dette eksempelet, men det er også gjort forsøk på å eksponere data ved å gjøre objekter globalt tilgjengelig i *Zeppelin Notebook* ved bruk av **z.angularBindGlobal**, men dette har heller ikke bidratt til å gjøre data tilgjengelige i *angular* sin kode.

Kode 6.4: Kode for data innsamling til dropdown meny

```
%spark
import org.apache.spark.sql.{DataFrame}
import org.apache.spark.sql.functions._
import scala.jdk.CollectionConverters._
import java.util.{Map => JMap, List => JList}
import java.util.HashMap

def dfToPayload(df: DataFrame, limitRows: Int = 200): JMap[String, Object] =
{
```

```

    val df2 = df.limit(limitRows)

    val columns: JList[String] =
        df2.columns.toSeq.asJava

    val rows: JList[String] =
        df2.toJSON.collect().toSeq.asJava // JSON strings (parse in %angular)

    val m = new HashMap[String, Object]()
    m.put("columns", columns.asInstanceOf[Object])
    m.put("rows", rows.asInstanceOf[Object])
    m
}

def makeDataset(key: String, title: String, df: DataFrame, limitRows: Int =
    200): JMap[String, Object] = {
    val m = new HashMap[String, Object]()
    m.put("key", key)
    m.put("title", title)
    m.putAll(dfToPayload(df, limitRows))
    m
}

// ---- Load from Cassandra ----
val rushtid = spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(Map("keyspace" -> "capstone_ks", "table" -> "rushtid"))
    .load()
    .where(col("navn") === "Glemmen" && col("dato") >= "2024-03-04" && col("
        dato") <= "2024-03-10")

val trafikkmengde_mnd = spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(Map("keyspace" -> "capstone_ks", "table" -> "trafikkmengde_mnd"))
    .load()
    .where(col("navn") === "Glemmen" && col("yr") >= 2022)

val sykkel_temp_corr = spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(Map("keyspace" -> "capstone_ks", "table" -> "sykkel_temp_corr"))
    .load()
    .select("navn", "trafikkmengde", "lufttemperatur", "dato")
    .where(col("navn") === "Wilbergjordet" && col("dato") >= "2022-02-01" &&
        col("dato") < "2025-10-01")

val bil_temp_corr = spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(Map("keyspace" -> "capstone_ks", "table" -> "bil_temp_corr"))
    .load()
    .select("navn", "trafikkmengde", "lufttemperatur", "dato")
    .where(col("navn") === "Stabburet" && col("dato") >= "2022-02-01" && col("
        dato") < "2025-10-01")

```

```

val veiarbeid = spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(Map("keyspace" -> "capstone_ks", "table" -> "veiarbeid"))
    .load()
    .where(col("navn") === "Simo")

// ---- Build list (Java list of Java maps) ----
val datasets: java.util.List[JMap[String, Object]] = Seq(
    makeDataset("rushtid", "rushtid (Glemmen, 2024-03-04..2024-03-10)",
        rushtid),
    makeDataset("trafikkmengde_mnd", "trafikkmengde_mnd (Glemmen, yr>=2022)",
        trafikkmengde_mnd),
    makeDataset("sykkel_temp_corr", "sykkel_temp_corr (Wilbergjordet,
        2022-02-01..2025-10-01)", sykkel_temp_corr),
    makeDataset("bil_temp_corr", "bil_temp_corr (Stabburet,
        2022-02-01..2025-10-01)", bil_temp_corr),
    makeDataset("veiarbeid", "veiarbeid (Simo)", veiarbeid)
).asJava

z.angularBind("datasets", datasets)
z.angularBind("selectedKey", "rushtid")

println("datasets bound: " + datasets.size)
println("first dataset keys: " + datasets.get(0).keySet())

```

– Implementere dropdown meny:

Etter at dataene var samlet inn og bundet til *angular* objekter i *Zeppelin Notebook*, var neste steg å implementere selve *dropdown* menyen ved hjelp av **html** og **angular** kode.

Her er et eksempel på hvordan vi har forsøkt å implementere et *UI* med *dropdown* meny med *angular* i *Zeppelin Notebook* med følgende løsninger:

1. Forventer at **datasets** objektet fra **spark** koden inneholder relevant data for å fylle ut menyen:
 - **key**: Unik ID for rullegardinvalg.
 - **title**: Tekst for visning i menyen.
 - **columns**: Liste over kolonnenavn.
 - **rows**: Rader enten som **JSON**-strenger eller objekter.
2. Viser en rullegardin for å velge hvilket datasett som skal vises (standard er *rushtid*).
3. Viser det valgte datasettet som en tabell ved å:
 - Gjengi kolonneoverskrifter fra kolonner.
 - Iterere gjennom radene i det valgte datasettet, parse hver rad fra **JSON** til et objekt, og vise verdiene i tabellceller.
 - Skrive ut hver celle basert på kolonnenavn for å sikre riktig rekkefølge.
 - Håndtere tilfeller der ingen datasett er valgt ved å vise en passende melding.

Kode 6.5: Forsøk på implementering av **angular** UI med dropdown meny

```

%angular
<div style="padding:10px"

```

```

ng-init="
  parseRow = function(x){ try { return (typeof x === 'string') ? JSON.
    parse(x) : x; } catch(e){ return {}; } };
  getSelected = function(){
    if(!datasets) return null;
    for (var i=0; i<datasets.length; i++) if (datasets[i].key ===
      selectedKey) return datasets[i];
    return null;
  };
">

<label style="font-weight:600">Choose dataset:</label>
<select
  ng-model="selectedKey"
  ng-init="selectedKey = selectedKey || 'rushtid'"
  ng-options="d.key as d.title for d in datasets"
  style="margin-left:10px; padding:4px 8px;">
</select>

<div style="margin-top:12px">
  <div ng-if="!getSelected()">No data bound yet. Run the %spark paragraph
    first.</div>

  <div ng-if="getSelected()">
    <h4 style="margin: 8px 0;">{{getSelected().title}}</h4>

    <table class="table table-striped table-condensed" style="width:100%;"
      >
      <thead>
        <tr>
          <th ng-repeat="c in getSelected().columns">{{c}}</th>
        </tr>
      </thead>

      <tbody>
        <tr ng-repeat="r in getSelected().rows track by $index">
          <td ng-repeat="c in getSelected().columns">
            {{ parseRow(r)[c] }}
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>
</div>

```

6.2 Løsninger for Docker Utfordringer

Etter noen døgn med feilsøking og testing av forskjellige konfigurasjoner, har vi endelig funnet en løsning som ser ut til å fungere tilfredsstillende.

Dette er for øyeblikket ikke en verifisert løsning for alle medlemmer i gruppen, da vi ikke har fått

testet oppsettet ferdig på mer enn en maskin.

Selve prosessen har ført til en dypere analyse av hvordan et *Docker* miljø fungerer på et Windows 11 system og hvilke fallgruver man burde være oppmerksom på ved en potensielt ny installasjon i fremtiden.

Grunnet at vi har funnet særdeles lite dokumentasjon vedrørende denne type feilsøking, samt at vi har begrenset erfaring med konfigurering av omhandlende løsninger, så har vi benyttet oss tungt av *LLM* verktøy[61][63][64] for å komme frem til denne løsningen.

6.2.1 Windows 11

Dette er løsningen vi har kommet frem til for å få et stabilt og funksjonelt *Docker* miljø på en Windows 11 maskin med *WSL2* backend.

Løsningen består av to hoveddeler:

- ◇ Endring av miljøets *docker-compose.yml* (YAML) fil med nødvendige konfigurasjoner for å etablere et stabilt miljø.
- ◇ Justering av ressursallokeringen til *WSL2* for å sikre at *Docker* har korrekt tilgang på ressurser for effektiv og stabil kjøring.

6.2.2 YAML Konfigurering

Det første steget i løsningen var å oppdatere *docker-compose.yml* filen med nødvendige konfigurasjoner for å etablere et stabilt miljø.

Hver ambisjonen å holde oss så nærme den originale filen som mulig, da vi ikke hadde en fundamental forståelse av hvordan de forskjellige endringene ville påvirke miljøet, hvilke parametere som var kritiske og hvordan de relative adressene til forskjellige '*volumes*' og '*networks*' fungerte.

Dette var en øvelse med mye prøving og feiling. Et av de største problemene vi støtte på var at enkelte '*containers*' ikke ville starte opp som forventet.

Blandt annet så hadde vi en *Cassandra node* som startet opp uten problemer, men som ikke var tilgjengelig for clusteret, da den ikke fikk tildelt en IP, til tross for at *Docker* miljøet inkluderte den i arbeidskøen. Dette førte til at hver gang denne noden fikk en jobb, så ble systemet låst og uresponsivt frem til noden feilet med en timeout feilmelding. Dette ble løst ved at vi implementerte flere spesifikasjoner i **environment:** seksjonen til hver *Cassandra node* for å sikre at nodene startet opp på en funksjonell måte. Det ble også lagt inn en valideringskontroll på disse nodene som sjekker at nodene responderer som forventet når miljøet blir kjørt i gang. Dette medførte at *Cassandra noder* bruker noe lengre tid på selve oppstarten, men dette har vist seg å bidra til et betraktelig mer stabilt oppsett etter at alle *containere* har startet opp.

Et annet problem vi støtte på når vi forsøkte å tilbakestille miljøet for å starte med blanke ark, så støttet vi på problemer med den oppførte kilden til **spark: 3.5.6** sitt '*image*'. Etter litt undersøkelser viste det seg at denne versjonen hadde blitt fjernet fra *bitnami* sine registre i slutten av september[65], og vi måtte dermed oppdatere **image:** seksjonen til *spark containere* til å hente ned **apache/spark: 3.5.6** i stedet.

Til slutt ble det lagt inn et lite command script på *spark* sine arbeidere som definerer følgende:

- ◇ `- /bin/bash - -lc`
Kjører et login(-l) **bash shell** som ber om å eksekvere en *command string*(-c). Ved å kjøre dette i et *login shell* sikrer vi at *bash* leser profil filer, slik at relative adresseringer peker til riktig lokasjon.
- ◇ `'>'`(*YAML folded block*)
Sørger for at vi kan skrive en commando over flere linjer, også 'bretter' *YAML* dette sammen til en enkel linje ved kjøring.
- ◇ `-cores 2`
Definerer hvor mange kjerner arbeideren får tilgang til å benytte.
- ◇ `-memory 2G`
Definerer hvor mye RAM arbeideren får tilgang til å bruke.
- ◇ `&&`
Binneledd for sekvensiell kjøring av kommandoer.
- ◇ `tail -F /opt/spark/logs/*worker*.out`
Følger (-F håndterer loggrotasjoner) arbeidsnoden sin loggfil i forgrunnen slik at containeren fortsetter å kjøre. Spark-skriptet start-worker.sh kjører arbeidsnoden i bakgrunnen, så uten en prosess i forgrunnen ville containeren potensielt ha avsluttet. Denne tail-kommandoen holder PID 1 i live og sender loggene videre til docker logs.

Her ser vi at det er mange små detaljer som må på plass for at et *Docker* miljø skal fungere optimalt, og det har vært en lærerik prosess å forstå hvordan disse konfigurasjonene påvirker miljøets stabilitet og ytelse.

Under er den endelige versjonen av *docker-compose.yml* filen(venstre) som vi har jobbet med på Windows maskiner, samt den originale filen(høyre) hentet fra *Canvas* portalen for sammenligning:

<pre> 1 networks: 2 bigdata: 3 driver: bridge 4 5 volumes: 6 cassandra-seed-data: 7 cassandra-1-data: 8 cassandra-2-data: 9 zeppelin-data: 10 11 services: 12 # ----- 13 # Spark 14 # ----- 15 spark-master: 16 image: apache/spark:3.5.6 17 platform: "linux/amd64" 18 container_name: spark-master 19 environment: 20 - SPARK_MASTER_HOST=spark-master 21 - SPARK_NO_DAEMONIZE=true </pre>	<pre> 1 networks: 2 bigdata: 3 driver: bridge 4 5 volumes: 6 cassandra-seed-data: 7 cassandra-1-data: 8 cassandra-2-data: 9 zeppelin-data: 10 11 services: 12 # ----- 13 # Spark 14 # ----- 15 spark-master: 16 image: bitnami/spark:3.5.6 17 platform: "linux/amd64" 18 container_name: spark-master 19 environment: 20 - SPARK_MODE=master 21 - SPARK_NO_DAEMONIZE=true </pre>
--	--

```

22     # Optional: tune Spark UI host if
23     needed
24     # - SPARK_MASTER_HOST=spark-
25     master
26     command: >
27     bash -lc "/opt/spark/sbin/start-
28     master.sh && tail -F /opt/spark/
29     logs/*master*.out"
30     volumes:
31     - ./sparkio:/sparkio
32     ports:
33     - "7077:7077" # Spark master
34     RPC
35     - "8080:8080" # Spark master UI
36     networks:
37     - bigdata
38     restart: unless-stopped
39
40 spark-worker-1:
41     image: apache/spark:3.5.6
42     container_name: spark-worker-1
43     command:
44     - /bin/bash
45     - -lc
46     - >
47     /opt/spark/sbin/start-
48     worker.sh --cores 2 --memory 2G
49     spark://spark-master:7077 &&
50     tail -F /opt/spark/logs/*
51     worker*.out
52     volumes:
53     - ./sparkio:/sparkio
54     depends_on:
55     - spark-master
56     ports:
57     - "8081:8081" # Worker UI (
58     worker-1)
59     networks:
60     - bigdata
61     restart: unless-stopped
62
63 spark-worker-2:
64     image: apache/spark:3.5.6
65     platform: "linux/amd64"
66     container_name: spark-worker-2
67     command:
68     - /bin/bash
69     - -lc
70     - >
71     /opt/spark/sbin/start-
72     worker.sh --cores 2 --memory 2G
73     spark://spark-master:7077 &&
74     tail -F /opt/spark/logs/*
75     worker*.out
76     volumes:
77     - ./sparkio:/sparkio

```

```

23     # Optional: tune Spark UI host if
24     needed
25     # - SPARK_MASTER_HOST=spark-
26     master
27     volumes:
28     - ./sparkio:/sparkio
29     ports:
30     - "7077:7077" # Spark master
31     RPC
32     - "8080:8080" # Spark master UI
33     networks:
34     - bigdata
35     restart: unless-stopped
36
37 spark-worker-1:
38     image: bitnami/spark:3.5.6
39     container_name: spark-worker-1
40     environment:
41     - SPARK_MODE=worker
42     - SPARK_MASTER_URL=spark://spark-
43     master:7077
44     - SPARK_WORKER_MEMORY=2G
45     - SPARK_WORKER_CORES=2
46     - SPARK_NO_DAEMONIZE=true
47     volumes:
48     - ./sparkio:/sparkio
49     depends_on:
50     - spark-master
51     ports:
52     - "8081:8081" # Worker UI (
53     worker-1)
54     networks:
55     - bigdata
56     restart: unless-stopped
57
58 spark-worker-2:
59     image: bitnami/spark:3.5.6
60     platform: "linux/amd64"
61     container_name: spark-worker-2
62     environment:
63     - SPARK_MODE=worker
64     - SPARK_MASTER_URL=spark://spark-
65     master:7077
66     - SPARK_WORKER_MEMORY=2G
67     - SPARK_WORKER_CORES=2
68     - SPARK_NO_DAEMONIZE=true
69     volumes:
70     - ./sparkio:/sparkio
71     depends_on:
72     - spark-master
73     # UI is available within the
74     network; map if you want it on host
75     :
76     # ports:
77     # - "8083:8081"
78     networks:

```

```

66     depends_on:
67         - spark-master
68         # UI is available within the
69         network; map if you want it on host
70         :
71         # ports:
72         # - "8083:8081"
73     networks:
74         - bigdata
75     restart: unless-stopped
76
77 spark-worker-3:
78     image: apache/spark:3.5.6
79     platform: "linux/amd64"
80     container_name: spark-worker-3
81     command:
82         - /bin/bash
83         - -lc
84         - >
85             /opt/spark/sbin/start-
86             worker.sh --cores 2 --memory 2G
87             spark://spark-master:7077 &&
88             tail -F /opt/spark/logs/*
89             worker*.out
90     volumes:
91         - ./sparkio:/sparkio
92     depends_on:
93         - spark-master
94     # ports:
95     # - "8084:8081"
96     networks:
97         - bigdata
98     restart: unless-stopped
99
100 # -----
101 # Cassandra (3-node cluster)
102 # -----
103
104 cassandra-seed:
105     image: cassandra:4.1
106     container_name: cassandra-seed
107     hostname: cassandra-seed
108     environment:
109         - CASSANDRA_CLUSTER_NAME=demo-
110         cluster
111         - CASSANDRA_DC=datacenter1
112         - CASSANDRA_RACK=rack1
113         - CASSANDRA_SEEDS=cassandra-seed
114         - CASSANDRA_NUM_TOKENS=256
115         - CASSANDRA_LISTEN_ADDRESS=auto
116         - CASSANDRA_BROADCAST_ADDRESS=
117         cassandra-seed
118         - CASSANDRA_BROADCAST_RPC_ADDRESS
119         =cassandra-seed
120         - CASSANDRA_RPC_ADDRESS=0.0.0.0
121
122         - bigdata
123     restart: unless-stopped
124
125 spark-worker-3:
126     image: bitnami/spark:3.5.6
127     platform: "linux/amd64"
128     container_name: spark-worker-3
129     environment:
130         - SPARK_MODE=worker
131         - SPARK_MASTER_URL=spark://spark-
132         master:7077
133         - SPARK_WORKER_MEMORY=2G
134         - SPARK_WORKER_CORES=2
135         - SPARK_NO_DAEMONIZE=true
136     volumes:
137         - ./sparkio:/sparkio
138     depends_on:
139         - spark-master
140     # ports:
141     # - "8084:8081"
142     networks:
143         - bigdata
144     restart: unless-stopped
145
146 # -----
147 # Cassandra (3-node cluster)
148 # -----
149
150 cassandra-seed:
151     image: cassandra:4.1
152     container_name: cassandra-seed
153     hostname: cassandra-seed
154     environment:
155         - CASSANDRA_CLUSTER_NAME=demo-
156         cluster
157         - CASSANDRA_SEEDS=cassandra-seed
158         - CASSANDRA_NUM_TOKENS=256
159         - MAX_HEAP_SIZE=1G
160         - HEAP_NEWSIZE=256M
161     volumes:
162         - cassandra-seed-data:/var/lib/
163         cassandra
164     ports:
165         - "9042:9042" # CQL
166         # (Optional) expose JMX if you
167         need it:
168         # - "7199:7199"
169     networks:
170         - bigdata
171     restart: unless-stopped
172
173 cassandra-1:
174     image: cassandra:4.1
175     container_name: cassandra-1
176     hostname: cassandra-1

```

```

112     - MAX_HEAP_SIZE=1G
113     - HEAP_NEWSIZE=256M
114 volumes:
115     - cassandra-seed-data:/var/lib/
116     cassandra
117 ports:
118     - "9042:9042" # CQL
119     # (Optional) expose JMX if you
120     need it:
121     # - "7199:7199"
122 networks:
123     - bigdata
124 healthcheck:
125     test: ["CMD-SHELL", "cqlsh -e '
126     SHOW VERSION' 127.0.0.1 9042"]
127     interval: 10s
128     timeout: 5s
129     retries: 30
130 restart: unless-stopped
131
132 cassandra-1:
133     image: cassandra:4.1
134     container_name: cassandra-1
135     hostname: cassandra-1
136     environment:
137         - CASSANDRA_CLUSTER_NAME=demo-
138         cluster
139         - CASSANDRA_SEEDS=cassandra-seed
140         - CASSANDRA_NUM_TOKENS=256
141         - MAX_HEAP_SIZE=1G
142         - HEAP_NEWSIZE=256M
143     volumes:
144         - cassandra-1-data:/var/lib/
145         cassandra
146     depends_on:
147         - cassandra-seed
148     networks:
149         - bigdata
150     healthcheck:
151         test: ["CMD-SHELL", "cqlsh -e '
152         SHOW VERSION' 127.0.0.1 9042"]
153         interval: 10s
154         timeout: 5s
155         retries: 30
156     restart: unless-stopped
157
158 cassandra-2:
159     image: cassandra:4.1
160     container_name: cassandra-2
161     hostname: cassandra-2
162     environment:
163         - CASSANDRA_CLUSTER_NAME=demo-
164         cluster
165         - CASSANDRA_SEEDS=cassandra-seed
166         - CASSANDRA_NUM_TOKENS=256
167         - MAX_HEAP_SIZE=1G
168         - HEAP_NEWSIZE=256M
169     volumes:
170         - cassandra-2-data:/var/lib/
171         cassandra
172     depends_on:
173         - cassandra-seed
174     networks:
175         - bigdata
176     restart: unless-stopped
177
178 # -----
179 # Zeppelin
180 # -----
181 zeppelin:
182     image: apache/zeppelin:0.12.0
183     environment:
184         - MASTER=spark://spark-master
185         :7077
186         - SPARK_HOME=/opt/spark
187         - SPARK_PUBLIC_DNS=localhost
188     volumes:
189         - ./sparkio:/sparkio
190         - ./opt/spark-3.5.6-bin-hadoop3:/
191         opt/spark:ro
192         - zeppelin-data:/zeppelin
193     ports:
194         - "8082:8080" # Zeppelin UI

```

```

159   image: cassandra:4.1
160   container_name: cassandra-2
161   hostname: cassandra-2
162   environment:
163     - CASSANDRA_CLUSTER_NAME=demo-
      cluster
164     - CASSANDRA_DC=datacenter1
165     - CASSANDRA_RACK=rack1
166     - CASSANDRA_SEEDS=cassandra-seed
167     - CASSANDRA_NUM_TOKENS=256
168     - CASSANDRA_LISTEN_ADDRESS=auto
169     - CASSANDRA_BROADCAST_ADDRESS=
      cassandra-2
170     - CASSANDRA_BROADCAST_RPC_ADDRESS
      =cassandra-2
171     - CASSANDRA_RPC_ADDRESS=0.0.0.0
172     - MAX_HEAP_SIZE=1G
173     - HEAP_NEWSIZE=256M
174   volumes:
175     - cassandra-2-data:/var/lib/
      cassandra
176   depends_on:
177     - cassandra-seed
178   networks:
179     - bigdata
180   healthcheck:
181     test: ["CMD-SHELL", "cqlsh -e '
      SHOW VERSION' 127.0.0.1 9042"]
182     interval: 10s
183     timeout: 5s
184     retries: 30
185     restart: unless-stopped
186
187   # -----
188   # Zeppelin
189   # -----
190   zeppelin:
191     image: apache/zeppelin:0.12.0
192     environment:
193       - MASTER=spark://spark-master
        :7077
194       - SPARK_HOME=/opt/spark
195       - SPARK_PUBLIC_DNS=localhost
196     volumes:
197       - ./sparkio:/sparkio
198       - ./opt/spark-3.5.6-bin-hadoop3:/
        opt/spark:ro
199       - zeppelin-data:/zeppelin
200     ports:
201       - "8082:8080"           # Zeppelin UI
202       - "4040-4050:4040-4050" # Spark
      app UIs from Zeppelin
203     depends_on:
204       spark-master:

```

```

167     - "4040-4050:4040-4050" # Spark
      app UIs from Zeppelin
168     depends_on: ["spark-master"]
169     networks: ["bigdata"]

```

Kode 6.6: Siste utgave av original YAML fil hentet fra Canvas

```
205         condition: service_started
206     cassandra-seed:
207         condition: service_healthy
208     cassandra-1:
209         condition: service_healthy
210     cassandra-2:
211         condition: service_healthy
212     networks: ["bigdata"]
```

Kode 6.6: Vår korrigerte YAML fil

6.2.3 Zeppelin Interperter Oppsett

I tillegg til endringene i *docker-compose.yml* filen, har det også blitt gjort noen justeringer i *Zeppelin* sin *interpreter* konfigurasjon for å sikre at *Spark* interpreteren er riktig satt opp til å kommunisere med *Spark* masteren i *Docker* miljøet.

Dette oppsettet er i all hovedsak basert på konfigurasjoner som har blitt utdelt av foreleser gjennom slides fra forelesninger.

6.2.3.1 Spark

Name	Value
SPARK_HOME	/opt/spark
spark.master	spark://spark-master:7077
spark.submit.deployMode	
spark.app.name	
spark.driver.cores	1
spark.driver.memory	1g
spark.executor.cores	1
spark.executor.memory	1g
spark.executor.instances	2
spark.files	
spark.jars	
spark.jars.packages	com.datastax.spark:spark-cassandra-connector-assembly_2.12:3.5.1
zeppelin.spark.useHiveContext	true
zeppelin.spark.run.asLoginUser	true
zeppelin.spark.printREPLOutput	true
zeppelin.spark.maxResult	1000
zeppelin.spark.enableSupportedVersionCheck	true
zeppelin.spark.uiWebUrl	
zeppelin.spark.ui.hidden	false
spark.webui.yarn.useProxy	false
zeppelin.spark.scala.color	true
zeppelin.spark.deprecatedMsg.show	true
zeppelin.spark.concurrentSQL	true
zeppelin.spark.concurrentSQL.max	10
zeppelin.spark.sql.stacktrace	true
zeppelin.spark.sql.interpolation	false
PYSPARK_PYTHON	python
PYSPARK_DRIVER_PYTHON	python
zeppelin.pyspark.usePython	true
zeppelin.R.knitr	true
zeppelin.R.cmd	R
zeppelin.R.image.width	100%
zeppelin.R.render.options	out.format='html',comment=NA,echo=FALSE
zeppelin.R.shiny.portRange	:
spark.cassandra.connection.host	cassandra-seed,cassandra-1,cassandra-2
spark.cassandra.connection.localDC	datacenter1
spark.driver.extraJavaOptions	-Ddatastax-java-driver.advanced.metadata.token-map.enabled=true
spark.executor.extraJavaOptions	-Ddatastax-java-driver.advanced.metadata.token-map.enabled=true

Tabell 6.2: Zeppelin Spark Interpreter Konfigurasjon

6.2.3.2 Alluxio

Name	Value
alluxio.master.hostname	localhost
alluxio.master.port	19998

Tabell 6.3: Zeppelin Alluxio Interpreter Konfigurasjon

6.2.3.3 bigquery

Name	Value
zeppelin.bigquery.project_id	
zeppelin.bigquery.wait_time	5000
zeppelin.bigquery.max_no_of_rows	100000
zeppelin.bigquery.sql_dialect	
zeppelin.bigquery.region	

Tabell 6.4: Zeppelin Angular Interpreter Konfigurasjon

6.2.3.4 Cassandra

Name	Value
cassandra.hosts	cassandra-seed,cassandra-1,cassandra-2
cassandra.native.port	9042
cassandra.protocol.version	DEFAULT
cassandra.cluster	demo-cluster
cassandra.keyspace	system
cassandra.compression.protocol	NONE
cassandra.credentials.username	none
cassandra.credentials.password	*****
cassandra.query.default.consistency	ONE
cassandra.query.default.fetchSize	5000
cassandra.socket.read.timeout.millisecs	12000
cassandra.ssl.enabled	false
cassandra.format.output	human
cassandra.format.locale	en_US
cassandra.format.timezone	UTC

Tabell 6.5: Zeppelin Cassandra Interpreter Konfigurasjon

6.2.3.5 Elasticsearch

Name	Value
elasticsearch.host	localhost
elasticsearch.port	9300
elasticsearch.client.type	transport
elasticsearch.cluster.name	elasticsearch
elasticsearch.result.size	10

Tabell 6.6: Zeppelin Elasticsearch Interpreter Konfigurasjon

6.2.3.6 File

Name	Value
hdfs.url	http://localhost:50070/webhdfs/v1/
hdfs.user	hdfs
hdfs.maxlength	1000

Tabell 6.7: Zeppelin File Interpreter Konfigurasjon

6.2.3.7 flink

Name	Value
FLINK_HOME	
HADOOP_CONF_DIR	
HIVE_CONF_DIR	
flink.execution.mode	local
flink.execution.remote.host	
flink.execution.remote.port	
jobmanager.memory.process.size	1024m
taskmanager.memory.process.size	1024m
taskmanager.numberOfTaskSlots	1
local.number-taskmanager	4
yarn.application.name	Zeppelin Flink Session
yarn.application.queue	default
zeppelin.flink.uiWebUrl	
zeppelin.flink.run.asLoginUser	true
flink.udf.jars	
flink.udf.jars.packages	
flink.execution.jars	
flink.execution.packages	
zeppelin.flink.scala.color	true
zeppelin.flink.scala.shell.tmp_dir	
zeppelin.flink.enableHive	false
zeppelin.flink.hive.version	2.3.7
zeppelin.flink.module.enableHive	false
zeppelin.flink.printREPLOutput	true
zeppelin.flink.maxResult	1000
zeppelin.pyflink.python	python
flink.interpreter.close.shutdown_cluster	true
zeppelin.interpreter.close.cancel_job	true
zeppelin.flink.job.check_interval	1000
zeppelin.flink.concurrentBatchSql.max	10
zeppelin.flink.concurrentStreamSql.max	10

Tabell 6.8: Zeppelin Flink Interpreter Konfigurasjon

6.2.3.8 flink-cmd

Name	Value
FLINK_HOME	

Tabell 6.9: Zeppelin Flink-CMD Interpreter Konfigurasjon

6.2.3.9 Groovy

Name	Value
GROOVY_CLASSES	

Tabell 6.10: Zeppelin Groovy Interpreter Konfigurasjon

6.2.3.10 HBase

Name	Value
hbase.home	/usr/lib/hbase/
hbase.ruby.sources	lib/ruby
zeppelin.hbase.test.mode	false

Tabell 6.11: Zeppelin HBase Interpreter Konfigurasjon

6.2.3.11 InfluxDB

Name	Value
influxdb.url	http://localhost:9999
influxdb.token	*****
influxdb.org	my-org
influxdb.logLevel	NONE

Tabell 6.12: Zeppelin InfluxDB Interpreter Konfigurasjon

6.2.3.12 Java

Name	Value
default.url	jdbc:postgresql://localhost:5432/
default.user	gpadmin
default.password	*****
default.driver	org.postgresql.Driver
default.completer.ttlInSeconds	120
default.completer.schemaFilters	
default.precode	
default.statementPrecode	
common.max_count	1000
zeppelin.jdbc.auth.type	
zeppelin.jdbc.auth.kerberos.proxy.enable	true
zeppelin.jdbc.concurrent.use	true
zeppelin.jdbc.concurrent.max_connection	10
zeppelin.jdbc.keytab.location	
zeppelin.jdbc.principal	
zeppelin.jdbc.interpolation	false
zeppelin.jdbc.maxConnLifetime	-1
zeppelin.jdbc.maxRows	1000
zeppelin.jdbc.hive.timeout.threshold	60000
zeppelin.jdbc.hive.monitor.query_interval	1000
zeppelin.jdbc.hive.engines.tag.enable	true
zeppelin.jdbc.kyuubi.timeout.threshold	60000
zeppelin.jdbc.kyuubi.monitor.query_interval	1000
zeppelin.jdbc.kyuubi.jobUrl.template	

Tabell 6.13: Zeppelin Java (JDBC) Interpreter Konfigurasjon

6.2.3.13 Jupyter

Name	Value
zeppelin.livy.url	http://localhost:8998
zeppelin.livy.session.create_timeout	120
livy.spark.driver.cores	
livy.spark.driver.memory	
livy.spark.executor.instances	
livy.spark.executor.cores	
livy.spark.executor.memory	
livy.spark.dynamicAllocation.enabled	false
livy.spark.dynamicAllocation.cachedExecutorIdleTimeout	
livy.spark.dynamicAllocation.minExecutors	
livy.spark.dynamicAllocation.initialExecutors	
livy.spark.dynamicAllocation.maxExecutors	
zeppelin.livy.principal	
zeppelin.livy.keytab	
zeppelin.livy.pull_status.interval.millis	1000
zeppelin.livy.maxLogLines	1000
livy.spark.jars.packages	
zeppelin.livy.displayAppInfo	true
zeppelin.livy.restart_dead_session	false
zeppelin.livy.spark.sql.maxResult	1000
zeppelin.livy.spark.sql.field.truncate	true
zeppelin.livy.concurrentSQL	false
zeppelin.livy.tableWithUTFCharacter	false

Tabell 6.14: Zeppelin Jupyter (Livy) Interpreter Konfigurasjon

6.2.3.14 Markdown

Name	Value
markdown.parser.type	flexmark

Tabell 6.15: Zeppelin Markdown Interpreter Konfigurasjon

6.2.3.15 MongoDB

Name	Value
mongo.shell.path	mongosh
mongo.shell.command.table.limit	1000
mongo.shell.command.timeout	60000
mongo.server.host	localhost
mongo.server.port	27017
mongo.server.database	test
mongo.server.authenticationDatabase	
mongo.server.username	
mongo.server.password	*****
mongo.interpreter.concurrency.max	10

Tabell 6.16: Zeppelin MongoDB Interpreter Konfigurasjon

6.2.3.16 Neo4j

Name	Value
neo4j.url	bolt://localhost:7687
neo4j.database	
neo4j.multi.statement	true
neo4j.auth.type	BASIC
neo4j.auth.user	
neo4j.auth.password	
neo4j.max.concurrency	50

Tabell 6.17: Zeppelin Neo4j Interpreter Konfigurasjon

6.2.3.17 Python

Name	Value
zeppelin.python	python
zeppelin.python.maxResult	1000
zeppelin.python.useIPython	true
zeppelin.ipython.launch.timeout	30000
zeppelin.ipython.grpc.message_size	33554432

Tabell 6.18: Zeppelin Python Interpreter Konfigurasjon

6.2.3.18 R

Name	Value
zeppelin.R.knitr	true
zeppelin.R.cmd	R
zeppelin.R.maxResult	1000
zeppelin.R.image.width	100%
zeppelin.R.render.options	out.format='html', comment=NA, echo=FALSE, results='asis', message=F, warning=F, fig.retina=2
zeppelin.R.shiny.portRange	:
zeppelin.R.shiny.iframe_width	100%
zeppelin.R.shiny.iframe_height	500px

Tabell 6.19: Zeppelin R Interpreter Konfigurasjon

6.2.3.19 sh

Name	Value
shell.command.timeout.millisecs	60000
shell.command.timeout.check.interval	60000
shell.working.directory.user.home	false
zeppelin.shell.auth.type	
zeppelin.shell.keytab.location	
zeppelin.shell.principal	
zeppelin.shell.interpolation	false
zeppelin.concurrency.max	10
zeppelin.terminal.ip.mapping	

Tabell 6.20: Zeppelin Shell Interpreter Konfigurasjon

6.2.3.20 Spark-Submit

Name	Value
SPARK_HOME	/opt/spark

Tabell 6.21: Zeppelin Spark-Submit Interpreter Konfigurasjon

6.2.3.21 SPARQL

Name	Value
sparql.engine	jena
sparql.endpoint	http://dbpedia.org/sparql
sparql.replaceURIs	true
sparql.removeDatatypes	true

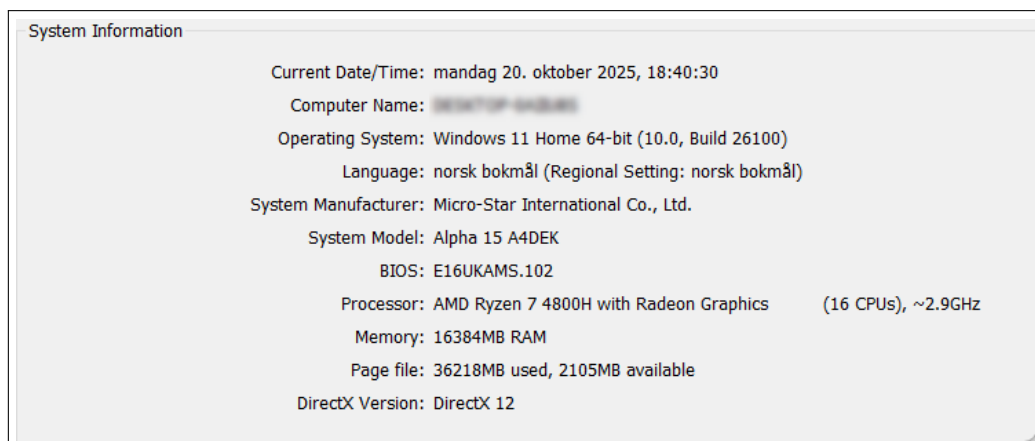
Tabell 6.22: Zeppelin SPARQL Interpreter Konfigurasjon

6.2.4 Operativ Systemet

En oppdagelse som ble gjort ganske sent i prosessen, var at Windows 11 har predefinerte begrensninger på hvor mye ressurser et *WSL2* miljø får tilgang til. Hva som er standard innstillinger er ikke helt avklart per dags dato, men dette viste seg å ikke være en optimal løsning for maskinen som kjørte *Docker* miljøet.

Docker sine standardkonfigurasjoner for *WSL2* miljøet på den gitt maskinen var satt til å benytte **16** kjerner og **8GB** RAM, så miljøet tillot ikke overskrivning av dette uavhengig av hvilke spesifikasjoner som var satt for de forskjellige *containerne* i **docker-compose.yml**.

I bildet under henviser vi til den omhandlede maskinens *hardware* spesifikasjoner:

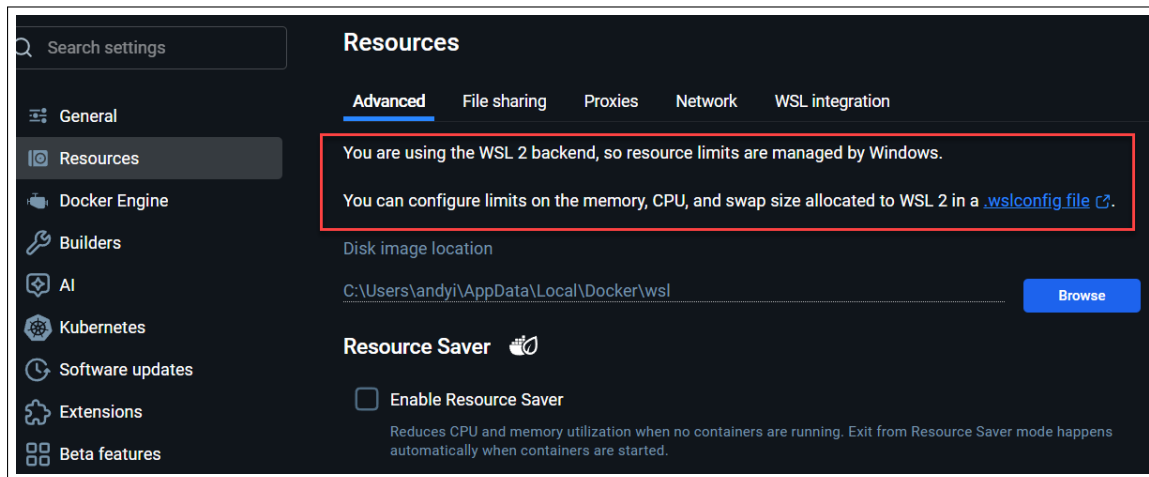
Figur 6.1: Utklipp av **dxdiag** til gjeldende maskin

Her ser vi at maskinen totalt har **16** kjerner og **16GB** RAM. Med det standardiserte oppsettet for *WSL2* miljøet ledet dette til mye ventetid og ustabilitet når vi forsøkte å utføre handlinger i *Zeppelin*.

Det første store problemet i henhold til effektivitet og ventetid var at alle kjerner i maskinen ble allokert til *Docker* miljøet. Dette ledet til at miljøets *containere* begynte å holde på kjerner allerede før vi begynte å skrive i *Zeppelin*. Når vi så forsøkte å utføre en handling uavhengig av språk eller kompleksitet, ble jobben sittende i kø da det skjeldent var ledige kjerner til å utføre jobben med en gang.

Det andre problemet var at *Docker* miljøet hadde tilgang til maksimalt **8GB** RAM av gangen. Dette førte til at maskinen satt med mye ledig minne som kunne bidratt betraktelig til ytelsen.

Første steg for å løse dette var å se om vi enkelt kunne justere ressursallokeringen direkte i *Docker* sine innstillinger. Her fant vi følgende informasjon under *Resources* fanen i *Docker Desktop* applikasjonens innstillinger:



Figur 6.2: Utklipp av Docker Desktop innstillinger - Resources

Det var altså ikke mulig å justere disse innstillingene direkte i *Docker*, men vi ble informert om at dette måtte gjøres via en `.wslconfig` fil i brukerens hjemmekatalog. Dermed opprettet vi en `.wslconfig` fil med ønskede spesifikasjoner for *WSL2* miljøet:

Kode 6.7: `.wslconfig` fil for justering av *WSL2* ressurser

```
# C:\Users\andyi\.wslconfig
[ws12]
memory=14GB # caps total RAM the WSL2 VM (incl. docker-desktop) can use
processors=10 # limit vCPUs
swap=4GB # optional but helps with Java spikes
localhostForwarding=true
```

Etter å ha lagret denne filen, måtte vi gjøre en omstart av *WSL2* miljøet for at endringene skulle tre i kraft. Dette ble gjort ved å kjøre følgende kommando i en *PowerShell* terminal med administrator rettigheter:

Kode 6.8: Kommando for å restarte *WSL2* miljøet

```
wsl --shutdown
```

Når vi nå startet opp *Docker* på nytt, kunne vi observere at ressursallokeringen hadde blitt oppdatert i henhold våre spesifikasjoner.

Dette førte til en betydelig forbedring i ytelsen til *Docker* miljøet. Jobber som tidligere kunne finne på å vente i helt opp til 15 minutter for de startet, ble nå utført nesten umiddelbart.

Det ble også observert at maskinen holdt seg mer responsiv under kjøring av jobber i *Zeppelin* og behovet for kontinuerlig omstart av *Docker* miljøet gikk kraftig ned.

6.2.5 Tidsfordeling

På grunnlag av de tekniske utfordringene vi har møtt, har tidsfordelingen i prosjektet blitt påvirket betydelig. Det faktumet at majoriteten av gruppemedlemmene ikke har kunne fullføre *Cassandra/CQL* relaterte oppgaver, har ledet til at målsetninger for denne innleveringen ikke har blitt møtt som forventet.

6.2.6 Samarbeid i Teamet

Til tross for de tekniske utfordringene, har samarbeidet i teamet vært sterkt. Vi har delt erfaringer, løsninger og utfordringer under arbeidet.

Siden Jon er den eneste som har oppnådd en relativ funksjonalitet med *Cassandra*, så har han jobbet mye med koblingen mellom *Spark* og *Cassandra*, men han har møt på utfordringer i form av at deler av datasettene/rader forsvinner ved skriving av til tabeller i *Cassandra*.

Andreas har fokusert på *Docker* oppsett og feilsøking i håp om å få et fungerende miljø for hele gruppen.

Torgrim sin hovedoppgave har vært å jobbe med innhenting og analyse av data relatert til vårt selvalgte *Capstone* prosjekt om trafikkanalyse.

Innføring av denne rapporten har dermed blitt fordelt i henhold til hvilket felt hvert individ har hatt sterkest innsikt i.

6.3 Performance Tuning

6.3.1 Salting

Salting er ofte brukt til å mitigere problemer ved dataskjevheter i datasettet. Datasettene som brukes i capstone prosjektet har gode inndelinger og lite skjevheter i partisjonsnøkklene så dette er ikke nødvendig å implementere. For mer informasjon om salting se [underseksjon 2.2.8](#)

6.3.2 Bucketing

Man bruker bucketing for å dele datasettet inn i et fast antall buckets, eller samlinger basert på verdiene i en eller flere kolonner med bruk av en hash funksjon. Dette for å optimalisere joins og redusere tid og ressursbruk. Dette gir en mer forutsigbar ytelse på større datasett med flere joins. Datasettet vi tar for oss har ikke noe stort behov for bucketing da vi kjører relativt lite join funksjoner på store datamengder. Ved utvidelse av datasettet for dette prosjektet kan det være hensiktsmessig å benytte bucketing.[66]

6.3.3 Repartitions og coalesce

Brukes til å endre antall partisjoner i et datasett. En partition er en logisk del av datasettet som behandles av en task. For mange partisjoner kan føre til høy overhead(mange tasks, men lite arbeid)

og for få partisjoner kan føre til dårlig utnyttelse av clustere og svak oppgavefordelingen. Repartisjoning brukes for å endre antallet partisjoner og fordele datasettet på nytt, men dette medfører en ressurskostbar shuffle operasjon. Dersom man kun ønsker å redusere eller slå sammen eksisterende partisjoner kan man istedet bruke coalesce. Denne metoden flytter minst mulig data og utløser vanligvis ikke en shuffle operasjon og er derfor mindre ressurskrevende.

I dette prosjektet har det ikke vært behov for å bruke repartisjoning eller coalesce og disse funksjonene er derfor ikke tatt i bruk.[59][67]

6.3.4 Serialisering

Ved repeterende skiving og lesing av filer er det ofte lønsomt å bruke filformater som inneholder mer metadata og som kan serialiseres raskere, for eksempel filtypen parquet. Slike formater gir ofte bedre utelse gjennom effektiv lagring, komprimering og innlesing av data. I capstone prosjektet skrives det ikke data til filer, og det er derfor ikke noe som kan implementeres i praksis.[68]

6.4 Møtereferater - Veileder

6.4.1 12.12.2025

Deltagere:

- Andreas
- Jon
- Torggrim
- Marius (Veileder)

Agenda:

- Spørsmål om Chebotko diagrammer.

Tilbakemeldinger:

- Siden vi har en enkel applikasjons-flyt kan vi vurdere å legge inn teoretisk utvidelse av dette for å vise forståelse for hvordan en mer kompleks applikasjons-flyt vil fungere.
- For Chebotko diagramme der vi ikke har behov for å sortere på alle primær nøkkelverdier for tabeller kan vi bare sette disse til std. stigende.

6.4.2 01.12.2025

Deltagere:

- Andreas
- Jon
- Torggrim
- Marius (Veileder)

Agenda:

- veiledning etter milestone 3.
- Tilbakemeldinger for innlevering

Tilbakemeldinger:

- Design og CQL database design.
- implementere Chebotko diagrammer.
- Demonstrere hvordan man bygger en database med flere nivåer. Vise forståelse av design.
- Diskutere ytelse, nevne dette mer i rapporten. Salting, Bucketing, optimalisering av spark.
- Redusere datamengde og tunge operasjoner. Trenger vi Regex i replace deler, kan vi bruke en vanlig replace istedet? Kanskje ved innlesning av data"
- Felles avsluttende refleksjonsøkt. Hva vi kunne gjort annerledes, Refleksjon over eget arbeid, Hva har fungert bra, Kunne vi angrepet dette på en annen måte?
- Neste veiledning fredag 12 desember 11.00

6.4.3 24.11.2025**Deltagere:**

- Andreas
- Jon
- Torggrim
- Marius (Veileder)

Agenda:

- Overordnet evaluering før siste innlevering.
- Få tilbakemelding på zeppelin rapport og tilhørende struktur.

Tilbakemeldinger:

- Veileder anbefalte oss mer verbose forklaringer i selve Zeppelin dokumentene, fremfor å kun ha dette innført i rapport.
- Vi ble informert om hvordan vi bedre kunne representere grafiske data ved å justere størrelser i zeppelin.

6.4.4 10.11.2025**Deltagere:**

- Andreas
- Jon
- Torgrim
- Marius (Veileder)

Agenda:

- Praktiske spørsmål knyttet til prosjekt presentasjon
- Spørsmål om hva vi bør fokusere på i presentasjonen.

Tilbakemeldinger:

- Tilbakemelding om at vi bør ha fokus på milestone3, hypoteser + spørsmål.
- Eventuelle utfordringer bør ha en positiv vinkling.
- Bruk Powerpoint til fremføringen.
- Bruk konkrete spørninger.

6.4.5 27.10.2025**Deltagere:**

- Andreas
- Jon
- Torgrim
- Marius (Veileder)

Agenda:

- Få en avklaring på forberedte spørsmål fra veileder.
- Tilbakemelding på milestone1 innlevering.

Tilbakemeldinger:

- Angående presentasjonen har vi fått tilbakemelding om at vi bør fokusere mer på våre løsninger og strukturelle implementasjon fremfor å legge dette frem som et endelig produkt.
- Når det kommer til behandling av data, blir vi anbefalt å benytte spark for å utføre behandling og strukturering av spørring tabeller, og heller bruke Cassandra tabeller som et start/sluttpunkt i prosessen. Dette er grunnet at det foreligger bedre løsninger for behandling av data i spark, da Cassandra mangler ADHOC funksjonalitet, som vi kunne skape problemer når man skal prøve å gjøre filtreringer mellom en hovedtabell og sub tabeller.
- Det er kommet frem at av de resterende gruppene i dette faget, så er vår gruppe den eneste som har møtt på store tekniske utfordringer med oppsett av docker miljøet og Cassandra containere.
- Veileder har gitt tilbakemelding på vår konfigurerte YAML og sier at denne inneholder gode løsninger som ligner mer på hvordan en produksjonsklar konfigurasjons fil ville ha sett ut ved implementasjon.

6.4.6 13.10.2025**Deltagere:**

- Andreas
- Jon
- Torgrim
- Marius (Veileder)

Agenda:

- Få en avklaring på forberedte spørsmål fra veileder.
- Tilbakemelding på milestone1 innlevering.

Tilbakemeldinger:

- Dato for endelig prosjekt innlevering: 15.12.2025

- Rapport struktur: Bra.
- Vedrørende sitering format, så ble vi anbefalt å se nærmere på IEEE referanse struktur, men nåværende siteringer var også godkjent for denne oppgaven(men ikke nødvendigvis for en bachelor oppgave).
- Veileder hadde ingen forslag til alternative datakilder for vårt prosjekt.
- Fikk tilbakemelding om at Fredrikstad området hadde tilstrekkelig med data, men at vi heller burde fokusere på økning av punkter og potensielt øke tidsperioden.
- I henhold til datavolum fikk vi beskjed om at relevans er høyere prioriter fremfor volum.
- Ble anbefalt å flytte prosjektet fra OneDrive til et git repo, grunnet hvordan OneDrive kan ha uforventet endring av datafiler.
- Vi ble anbefalt å starte med lokale dynamiske data for å senere fokusere på mer dynamiske løsninger(API).
- Rapport burde justeres for bedre format (cropping av bilder, etc.).
- Generalisere strukturen slik at det enkelt kan implementeres større mengder med data uten å måtte utvikle nye spørringer.
- Det ble forspurt en mer detaljert dataanalyse i forhold til hva data faktisk representerer (f.eks. nedbør pr. time, temperatur format (C, k, f?), titemålinger(Hvilken del av timen blir representert i verdien?).

6.4.7 29.09.2025

Deltagere:

- Andreas
- Jon
- Torgrim
- Marius (Veileder)

Agenda:

- Praktiske spørsmål i forhold til innlevering av milestone1.
- Spørsmål knyttet til beskrivelse av verdier i datasettene til capstone prosjekt.

Tilbakemeldinger:

- Spesifiser data og databeskrivelse.

- Beskrive hvordan dataene ble samlet inn og eventuelle begrensninger.
- Forklare hva dataene er og hvor de er hentet fra.
- Dokumenter forståelse, forstå dataene i datasettet.
- Opprette tabeller med forklaringer på hva datasettene inneholder og hvilke datatyper det inneholder.
- Forklaringer på hva dataene representerer
- Utvide intro datasett.
- Øke størrelse på bilder.
- Bruke describe funksjonen i spark og dokumentere hva den gir av informasjon om datasettet.

6.4.8 22.09.2025

Deltagere:

- Andreas
- Jon
- Torgrim
- Marius (Veileder)

Agenda:

- Gjennomgang av milestone1.
- Tips til potensielle data sett for capstone prosjektet.

Tilbakemeldinger:

- Foreleser anbefalte oss en bred problemstilling i henhold til capstone.
- Anbefalte oss å bruke datasett av personlig interesse.

Bibliografi

- [1] Apache Spark Project. *SparkSession.implicits* — *Scala API (Spark 3.5.5)*. Official documentation. URL: [https://spark.apache.org/docs/3.5.5/api/scala/org/apache/spark/sql/SparkSession\\$implicits\\$.html](https://spark.apache.org/docs/3.5.5/api/scala/org/apache/spark/sql/SparkSession$implicits$.html) (sjekket 26.09.2025).
- [2] Apache Spark Project. *types* — *Scala API (latest)*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/types/index.html> (sjekket 26.09.2025).
- [3] Apache Spark Project. *org.apache.spark.sql.expressions* — *Scala API (latest)*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/expressions/index.html> (sjekket 26.09.2025).
- [4] Apache Spark Project. *org.apache.spark.sql.functions* — *Scala API (latest)*. Official documentation. URL: [https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/functions\\$.html](https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/functions$.html) (sjekket 26.09.2025).
- [5] mukunku. *ParquetViewer* — *Parquet file viewer for Windows*. GitHub repository. URL: <https://github.com/mukunku/ParquetViewer> (sjekket 29.10.2025).
- [6] Scala Documentation Team. *A First Look at Types* — *Type Casting (Scala 3 Book)*. Official documentation. URL: <https://docs.scala-lang.org/scala3/book/first-look-at-types.html#type-casting> (sjekket 29.10.2025).
- [7] ChatGPT (GPT-5). *Conversation with ChatGPT (shared link)*. Conversation with the author. URL: <https://chatgpt.com/share/68d6c93d-cd44-800e-8d79-690b13c7f663> (sjekket 26.09.2025).
- [8] Apache Spark Project. *pyspark.sql.functions.date-format* — *PySpark API reference*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.date_format.html (sjekket 25.09.2025).
- [9] Apache Spark Project. *Datetime patterns for date/time formatting and parsing*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-ref-datetime-pattern.html> (sjekket 25.09.2025).
- [10] Apache Spark Project. *pyspark.sql.functions.dayofweek* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.dayofweek.html> (sjekket 25.09.2025).
- [11] Apache Spark Project. *pyspark.sql.functions.to-date* — *PySpark API reference*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.to_date.html (sjekket 25.09.2025).
- [12] Apache Spark Project. *pyspark.sql.DataFrame.agg* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.agg.html> (sjekket 25.09.2025).
- [13] Spark By Examples. *PySpark orderBy and sort Explained*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-orderby-and-sort-explained/> (sjekket 25.09.2025).
- [14] Apache Spark Project. *pyspark.sql.functions.date_trunc* — *PySpark API reference*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.date_trunc.html (sjekket 27.09.2025).

- [15] Apache Spark Project. *pyspark.sql.functions.median* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.median.html> (sjekket 25.09.2025).
- [16] Spark By Examples. *PySpark groupBy and agg (aggregate) Explained*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-groupby-agg-aggregate-explained/> (sjekket 25.09.2025).
- [17] Apache Spark Project. *pyspark.sql.DataFrame.schema* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.schema.html> (sjekket 25.09.2025).
- [18] Apache Spark Project. *pyspark.sql.DataFrame.join* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.join.html> (sjekket 25.09.2025).
- [19] Apache Spark Project. *pyspark.sql.DataFrame.alias* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.alias.html> (sjekket 25.09.2025).
- [20] Apache Spark Project. *pyspark.sql.functions.sum* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.sum.html> (sjekket 27.09.2025).
- [21] Apache Spark Project. *pyspark.sql.functions.avg* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.avg.html> (sjekket 25.09.2025).
- [22] Apache Spark Project. *org.apache.spark.sql.expressions.Window* — *Scala API (latest)*. Official documentation. URL: [https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/expressions/Window\\$.html](https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/expressions/Window$.html) (sjekket 27.09.2025).
- [23] Apache Spark Project. *pyspark.sql.functions.dense_rank* — *PySpark API reference*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.dense_rank.html (sjekket 25.09.2025).
- [24] Apache Spark Project. *pyspark.sql.Column.over* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.Column.over.html> (sjekket 27.09.2025).
- [25] ChatGPT (GPT-5). *Conversation with ChatGPT, scala join error fix*. Conversation with the author. URL: <https://chatgpt.com/share/68da5b83-ae3c-800e-8cce-56ccbf006505> (sjekket 29.09.2025).
- [26] Bill Chambers og Matei Zaharia. *Spark: The Definitive Guide: Big Data Processing Made Simple*. O'Reilly Media, 28. feb. 2018. ISBN: 9781491912218. URL: <https://isbnsearch.org/isbn/9781491912218> (sjekket 25.09.2025).
- [27] Apache Spark Project. *Performance Tuning* — *Spark SQL, DataFrames and Datasets*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-performance-tuning.html> (sjekket 25.09.2025).
- [28] Apache Spark. *pyspark.sql.DataFrame*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.html> (sjekket 12.12.2025).

- [29] Scala Lang. *scala.math*. Official documentation. URL: <https://www.scala-lang.org/api/3.2.1/scala/math.html#> (sjekket 28.09.2025).
- [30] Apache Spark. *Generic Load/Save Functions*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-data-sources-load-save-functions.html> (sjekket 28.09.2025).
- [31] Apache Spark. *pyspark.sql.DataFrame.show*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.show.html> (sjekket 28.09.2025).
- [32] Apache Spark. *pyspark.sql.DataFrame.count*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.count.html> (sjekket 28.09.2025).
- [33] Apache Spark. *pyspark.sql.DataFrame.withColumn*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.withColumn.html> (sjekket 28.09.2025).
- [34] Apache Spark. *pyspark.sql.DataFrame.from_unixtime*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.from_unixtime.html?highlight=from%20unixtime#pyspark.sql.functions.from_unixtime (sjekket 28.09.2025).
- [35] Apache Spark. *pyspark.sql.DataFrame.to_date*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.to_date.html?highlight=to_date#pyspark-sql-functions-to-date (sjekket 28.09.2025).
- [36] Apache Spark. *pyspark.sql.DataFrame.drop*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.drop.html> (sjekket 28.09.2025).
- [37] Apache Spark Project. *pyspark.sql.DataFrame.where* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.where.html> (sjekket 25.09.2025).
- [38] Apache Spark. *pyspark.sql.DataFrame.select*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.select.html> (sjekket 28.09.2025).
- [39] Apache Spark. *pyspark.sql.functions.format_number*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.format_number.html?highlight=format_number#pyspark.sql.functions.format_number (sjekket 28.09.2025).
- [40] Apache Spark. *pyspark.sql.DataFrame.limit*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.limit.html?highlight=limit#pyspark-sql-dataframe-limit> (sjekket 28.09.2025).
- [41] Scala Lang. *StringOps*. Official documentation. URL: <https://www.scala-lang.org/api/3.2.1/scala/collection/StringOps.html#> (sjekket 28.09.2025).
- [42] ChatGPT (GPT-5). *Conversation with ChatGPT, how can i create a rolling 3 month count of count in this sample*. Conversation with the author. URL: <https://chatgpt.com/share/68daf1bb-94a0-800e-9a54-69a08673b1a9> (sjekket 29.09.2025).

- [43] Apache Spark Project. *pyspark.sql.Window.rowsBetween* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.Window.rowsBetween.html> (sjekket 27.09.2025).
- [44] Apache Spark Project. *pyspark.sql.Column.cast* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.Column.cast.html> (sjekket 25.09.2025).
- [45] Apache Spark. *pyspark.sql.DataFrame.first*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.first.html?highlight=first#pyspark.sql.DataFrame.first> (sjekket 28.09.2025).
- [46] Apache Spark. *Interface Row*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/java/org/apache/spark/sql/Row.html> (sjekket 28.09.2025).
- [47] GridGain Systems. *Cassandra Ad Hoc Queries — Database Performance Solutions*. Technical article. URL: <https://www.gridgain.com/experience/solutions/database-performance/cassandra-ad-hoc-queries> (sjekket 01.11.2025).
- [48] Baeldung. *Ad Hoc Programming — Definition and Examples*. Tutorial article. URL: <https://www.baeldung.com/cs/ad-hoc-programming> (sjekket 01.11.2025).
- [49] Baeldung. *Cassandra Partition Key, Composite Key, and Clustering Key*. Tutorial article. URL: <https://www.baeldung.com/cassandra-keys> (sjekket 29.09.2025).
- [50] Harshavardhan Achyuta. *Mastering DataFrame Aggregations in Apache Spark*. Tutorial article. URL: <https://medium.com/@harshavardhan.achyuta/mastering-dataframe-aggregations-in-apache-spark-507b4e12b6e9> (sjekket 23.07.2023).
- [51] Statens Vegvesen. *Eksporter data*. Official data export. URL: <https://trafikdata.atlas.vegvesen.no/eksport?datatype=HOUR&from=2025-10-20&to=2025-10-21&trpids=29444V971469&zoom=12&lat=59.2275&lon=10.9416®istrationFrequencies=>.
- [52] Statens Vegvesen. *Eksporter data*. Official data export. URL: <https://trafikdata.atlas.vegvesen.no/eksport?datatype=HOUR&from=2025-10-20&to=2025-10-21&trpids=61523V971803&zoom=12&lat=59.2275&lon=10.9416®istrationFrequencies=>.
- [53] Statens Vegvesen. *Eksporter data*. Official data export. URL: <https://trafikdata.atlas.vegvesen.no/eksport?datatype=HOUR&from=2025-10-20&to=2025-10-21&trpids=29937V971442&zoom=12&lat=59.2275&lon=10.9416®istrationFrequencies=>.
- [54] timeanddate.com. *What Is ISO 8601?* Explainer article on the ISO 8601 date and time format. URL: <https://www.timeanddate.com/calendar/what-is-iso-8601.html> (sjekket 11.11.2025).
- [55] Wikipedia contributors. *ISO 8601*. Wikipedia, The Free Encyclopedia. URL: https://en.wikipedia.org/wiki/ISO_8601 (sjekket 11.11.2025).
- [56] Apache Spark Project. *pyspark.sql.DataFrame.dropDuplicates* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropDuplicates.html> (sjekket 25.09.2025).
- [57] Spark Code Hub. *Spark DataFrame Union*. Tutorial article. URL: <https://www.sparkcodehub.com/spark/dataframe/spark-union> (sjekket 11.11.2025).

- [58] Spark By Examples. *Spark regexp_replace — Replace String Value*. Tutorial article. URL: <https://sparkbyexamples.com/spark/spark-regexp-replace-replace-string-value/> (sjekket 11.11.2025).
- [59] Apache Spark Project. *RDD Programming Guide*. Official documentation. URL: <https://spark.apache.org/docs/latest/rdd-programming-guide.html> (sjekket 25.09.2025).
- [60] Apache Spark Project. *org.apache.spark.sql.DataFrameWriter — Scala API (latest)*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/DataFrameWriter.html> (sjekket 19.11.2025).
- [61] ChatGPT (GPT-5). *Conversation with ChatGPT (Install Docker on Mint)*. Conversation with the author. URL: <https://chatgpt.com/share/68f40cba-3890-800e-ba8c-98900cabed5a> (sjekket 19.10.2025).
- [62] DataStax. *Data Frames — Apache Spark Integration (DSE 6.9): TTL*. Official documentation. URL: <https://docs.datastax.com/en/dse/6.9/spark/data-frames.html#ttl> (sjekket 20.10.2025).
- [63] ChatGPT (GPT-5). *Conversation with ChatGPT (Docker desktop cache clearing)*. Conversation with the author. URL: <https://chatgpt.com/share/68f66e4b-5434-800e-a4fb-c057e7500a38> (sjekket 20.10.2025).
- [64] ChatGPT (GPT-5). *Conversation with ChatGPT (Spark container restart fix)*. Conversation with the author. URL: <https://chatgpt.com/share/68f66e09-9114-800e-8871-823918f4227f> (sjekket 20.10.2025).
- [65] Bitnami. *Upcoming changes to the Bitnami catalog (effective August 28th, 2025) — Issue #83267*. GitHub issue on bitnami/containers (closed). 16. jul. 2025. URL: <https://github.com/bitnami/containers/issues/83267> (sjekket 20.10.2025).
- [66] Spark Code Hub. *SQL Bucketing — Spark SQL Performance*. Tutorial article. URL: <https://www.sparkcodehub.com/spark/performance/sql-bucketing> (sjekket 15.12.2025).
- [67] Spark By Examples. *Spark Performance Tuning*. Tutorial article. URL: <https://sparkbyexamples.com/spark/spark-performance-tuning/> (sjekket 15.12.2025).
- [68] Apache Spark Project. *Tuning Spark — Performance Guide*. Official documentation. URL: <https://spark.apache.org/docs/latest/tuning.html> (sjekket 15.12.2025).
- [69] Apache Spark Project. *pyspark.sql.functions.to-timestamp-ntz — PySpark API reference*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.to_timestamp_ntz.html (sjekket 25.09.2025).
- [70] Apache Spark Project. *pyspark.sql.functions.to-timestamp — PySpark API reference*. Official documentation. URL: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.to_timestamp.html (sjekket 25.09.2025).
- [71] Apache Spark Project. *pyspark.Broadcast — Broadcast variable API (anchored link)*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.Broadcast.html?highlight=broadcast#pyspark.Broadcast> (sjekket 25.09.2025).
- [72] Spark By Examples. *PySpark Broadcast Join with Example*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-broadcast-join-with-example/> (sjekket 25.09.2025).

- [73] Apache Spark Project. *pyspark.Broadcast* — *Broadcast variable API*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.Broadcast.html> (sjekket 25.09.2025).
- [74] Apache Spark Project. *pyspark.sql.DataFrame.sort* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.sort.html> (sjekket 25.09.2025).
- [75] Philipp Brunenberg. *YouTube video on Spark/PySpark (Philipp Brunenberg)*. Online video. URL: https://www.youtube.com/watch?v=C4lX7GvJ9K0&ab_channel=PhilippBrunenberg (sjekket 25.09.2025).
- [76] Spark By Examples. *PySpark partitionByExample*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-partitionby-example/> (sjekket 25.09.2025).
- [77] Apache Spark Project. *pyspark.sql.DataFrameWriter.partitionBy* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameWriter.partitionBy.html> (sjekket 25.09.2025).
- [78] Stack Overflow. *What is the difference between the mean and the avg methods?* Q&A discussion. URL: <https://stackoverflow.com/questions/74026229/what-is-the-difference-between-the-mean-and-the-avg-methods> (sjekket 25.09.2025).
- [79] Apache Spark Project. *pyspark.sql.DataFrame.filter* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.filter.html> (sjekket 25.09.2025).
- [80] Apache Spark Project. *pyspark.sql.Column.alias* — *PySpark API reference (v3.5.6)*. Official documentation. URL: <https://spark.apache.org/docs/3.5.6/api/python/reference/pyspark.sql/api/pyspark.sql.Column.alias.html> (sjekket 25.09.2025).
- [81] Apache Spark Project. *pyspark.sql.DataFrameReader.csv* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameReader.csv.html> (sjekket 25.09.2025).
- [82] Spark By Examples. *Spark Schema Explained with Examples*. Tutorial article. URL: <https://sparkbyexamples.com/spark/spark-schema-explained-with-examples/> (sjekket 25.09.2025).
- [83] Apache Spark Project. *CSV Files* — *Spark SQL data sources*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-data-sources-csv.html> (sjekket 25.09.2025).
- [84] Apache Spark Project. *pyspark.sql.DataFrameReader.csv* — *PySpark API reference (anchored link)*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameReader.csv.html?highlight=csv#pyspark.sql.DataFrameReader.csv> (sjekket 25.09.2025).
- [85] Spark By Examples. *Find Median and Quantiles using Spark*. Tutorial article. URL: <https://sparkbyexamples.com/spark/find-median-and-quantiles-using-spark/> (sjekket 25.09.2025).
- [86] Apache Spark Project. *pyspark.sql.functions.day* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.day.html> (sjekket 25.09.2025).

- [87] Apache Spark Project. *pyspark.sql.DataFrame.drop* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.drop.html> (sjekket 25.09.2025).
- [88] Apache Spark Project. *pyspark.sql.functions.when* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.when.html> (sjekket 25.09.2025).
- [89] Stack Overflow. *Spark equivalent of if-then-else*. Q&A discussion. URL: <https://stackoverflow.com/questions/39048229/spark-equivalent-of-if-then-else> (sjekket 25.09.2025).
- [90] Apache Spark Project. *CASE WHEN* — *Spark SQL syntax reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-ref-syntax-qry-select-case.html> (sjekket 25.09.2025).
- [91] Spark By Examples. *PySpark where vs filter*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-where-filter/> (sjekket 25.09.2025).
- [92] Apache Spark Project. *pyspark.sql.DataFrame.dropna* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropna.html> (sjekket 25.09.2025).
- [93] Spark By Examples. *PySpark — Drop rows with null values*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-drop-rows-with-null-values/> (sjekket 25.09.2025).
- [94] Stack Overflow. *Remove rows from DataFrame based on condition in PySpark*. Q&A discussion. URL: <https://stackoverflow.com/questions/52395986/remove-rows-from-dataframe-based-on-condition-in-pyspark> (sjekket 25.09.2025).
- [95] Apache Spark Project. *pyspark.sql.DataFrame.distinct* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.distinct.html> (sjekket 25.09.2025).
- [96] Apache Spark Project. *pyspark.sql.DataFrame.dropDuplicatesWithinWatermark* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropDuplicatesWithinWatermark.html> (sjekket 25.09.2025).
- [97] Apache Spark Project. *pyspark.pandas.DataFrame.drop* — *pandas API on Spark*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.pandas/api/pyspark.pandas.DataFrame.drop.html?highlight=drop#pyspark.pandas.DataFrame.drop> (sjekket 25.09.2025).
- [98] Spark By Examples. *Spark Drop vs Delete vs Truncate — Differences*. Tutorial article. URL: <https://sparkbyexamples.com/spark/spark-drop-delete-truncate-differences/> (sjekket 25.09.2025).
- [99] Spark By Examples. *PySpark — Add New Column to DataFrame*. Tutorial article. URL: <https://sparkbyexamples.com/pyspark/pyspark-add-new-column-to-dataframe/> (sjekket 25.09.2025).
- [100] Apache Spark Project. *pyspark.sql.DataFrame.foreach* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.foreach.html> (sjekket 25.09.2025).

- [101] Apache Spark Project. *pyspark.sql.types.CharType* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.types.CharType.html> (sjekket 25.09.2025).
- [102] Apache Spark Project. *pyspark.sql.types.IntegerType* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.types.IntegerType.html> (sjekket 25.09.2025).
- [103] Apache Spark Project. *Parquet Files* — *Schema merging*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging> (sjekket 25.09.2025).
- [104] Apache Spark Project. *pyspark.sql.DataFrame.groupBy* — *PySpark API reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.groupBy.html> (sjekket 25.09.2025).
- [105] Apache Spark Project. *SHOW PARTITIONS* — *SQL auxiliary statements*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-ref-syntax-aux-show-partitions.html> (sjekket 25.09.2025).
- [106] DataNinjaGo. *Databricks Deep Dive #4 — Vectorised Parquet Reading*. Technical blog post. URL: <https://dataninjablog.com/2021/12/12/databricks-deep-dive-4-vectorised-parquet-reading/> (sjekket 25.09.2025).
- [107] Apache Spark Project. *JOIN* — *Spark SQL syntax reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-ref-syntax-qry-select-join.html> (sjekket 25.09.2025).
- [108] Apache Spark Project. *Data Types* — *Spark SQL reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-ref-datatypes.html> (sjekket 25.09.2025).
- [109] Spark By Examples. *Spark Read Options* — *Examples*. Tutorial article. URL: <https://sparkbyexamples.com/spark/spark-read-options/> (sjekket 25.09.2025).
- [110] Apache Spark Project. *JSON Files* — *Spark SQL data sources*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-data-sources-json.html> (sjekket 25.09.2025).
- [111] Apache Spark Project. *Data Sources* — *Overview*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-data-sources.html> (sjekket 25.09.2025).
- [112] Apache Spark Project. *Getting Started* — *Spark SQL, DataFrames and Datasets*. Official documentation. URL: <https://spark.apache.org/docs/latest/sql-getting-started.html> (sjekket 25.09.2025).
- [113] DataStax. *CQL Data Types* — *Cassandra Query Language (OSS 3.x)*. Official documentation. URL: https://docs.datastax.com/en/cql-oss/3.x/cql/cql_reference/cql_data_types_c.html (sjekket 05.10.2025).
- [114] DataStax. *CQL counter type* — *Cassandra Query Language (OSS 3.x)*. Official documentation. URL: https://docs.datastax.com/en/cql-oss/3.x/cql/cql_reference/counter_type.html (sjekket 05.10.2025).
- [115] GeeksforGeeks. *Useful CQL Queries in Cassandra*. Tutorial article. URL: <https://www.geeksforgeeks.org/dbms/useful-cql-query-in-cassandra/> (sjekket 01.11.2025).

- [116] Apache Cassandra Project. *CQL Commands — Reference (Table of Contents)*. Official documentation. URL: <https://cassandra.apache.org/doc/latest/cassandra/reference/cql-commands/commands-toc.html> (sjekket 01.11.2025).
- [117] Baeldung. *Cassandra Query Cheat Sheet*. Tutorial article. URL: <https://www.baeldung.com/cassandra-query-cheat-sheet> (sjekket 01.11.2025).
- [118] Apache Cassandra Project. *Data Manipulation (DML) — CQL Reference*. Official documentation. URL: <https://cassandra.apache.org/doc/stable/cassandra/developing/cql/dml.html> (sjekket 02.11.2025).
- [119] DataStax. *CREATE KEYSPACE — Cassandra Query Language (OSS 3.3)*. Official documentation. URL: https://docs.datastax.com/en/cql-oss/3.3/cql/cql_reference/cqlCreateKeyspace.html (sjekket 02.11.2025).
- [120] Stack Overflow. *WHERE and ORDER BY clauses in Cassandra CQL*. Q&A discussion. URL: <https://stackoverflow.com/questions/35708118/where-and-order-by-clauses-in-cassandra-cql> (sjekket 02.11.2025).
- [121] Apache Spark Project. *filter — SparkR DataFrame API Reference*. Official documentation. URL: <https://spark.apache.org/docs/latest/api/R/reference/filter.html> (sjekket 02.11.2025).
- [122] Stack Overflow. *Usage of {Spark} DataFrame as method*. Q&A discussion. URL: <https://stackoverflow.com/questions/31537420/usage-of-spark-dataframe-as-method> (sjekket 02.11.2025).
- [123] Spark By Examples. *Convert String to Date Format — Spark SQL*. Tutorial article. URL: <https://sparkbyexamples.com/spark/convert-string-to-date-format-spark-sql/> (sjekket 02.11.2025).